



NebulaGraph

NebulaGraph Database 手册

v3.5.0

梁振亚，黄凤仙

2023 vesoft Inc.

Table of contents

1. 欢迎阅读NebulaGraph 3.5.0 文档	6
1.1 快速开始	6
1.2 最新发布	6
1.3 其他资料	6
1.4 图例说明	6
1.5 修改文档中的错误	7
2. 简介	8
2.1 图	8
2.2 图数据库的市场概况	23
2.3 相关技术	34
2.4 什么是NebulaGraph	47
2.5 数据模型	50
2.6 路径	52
2.7 点 VID	54
2.8 服务架构	56
3. 快速入门	72
3.1 基于 Docker 快速部署	73
3.2 从云开始（免费试用）	0
3.3 本地部署	0
3.4 nGQL 命令汇总	0
4. nGQL 指南	0
4.1 nGQL 概述	0
4.2 数据类型	0
4.3 变量和复合查询	0
4.4 运算符	0
4.5 函数和表达式	0
4.6 通用查询语句	0
4.7 子句和选项	0
4.8 图空间语句	0
4.9 Tag 语句	0
4.10 Edge type 语句	0
4.11 点语句	0
4.12 边语句	0
4.13 原生索引	0
4.14 全文索引	0

4.15	子图和路径	0
4.16	查询调优与终止	0
4.17	作业管理	0
5.	安装部署	0
5.1	准备编译、本地安装和运行NebulaGraph的环境	0
5.2	编译安装	0
5.3	本地单机安装	0
5.4	使用 RPM/DEB 包部署NebulaGraph多机集群	0
5.5	使用 Docker Compose 部署NebulaGraph	0
5.6	使用生态工具安装NebulaGraph	0
5.7	管理NebulaGraph服务	0
5.8	连接NebulaGraph服务	0
5.9	管理 Storage 主机	0
5.10	升级NebulaGraph 至 3.5.0 版本	0
5.11	卸载NebulaGraph	0
6.	配置与日志	0
6.1	配置	0
6.2	日志	0
7.	监控	0
7.1	查询NebulaGraph监控指标	0
7.2	RocksDB 统计数据	0
8.	数据安全	0
8.1	验证和授权	0
8.2	SSL 加密	0
9.	备份与恢复	0
9.1	NebulaGraph BR（社区版）	0
9.2	管理快照	0
10.	同步与迁移	0
10.1	负载均衡	0
11.	最佳实践	0
11.1	Compaction	0
11.2	Storage 负载均衡	0
11.3	图建模设计	0
11.4	系统设计建议	0
11.5	执行计划	0
11.6	超级顶点（稠密点）处理	0
11.7	启用 AutoFDO	0
11.8	实践案例	0

12. 客户端	0
12.1 客户端介绍	0
12.2 NebulaGraph Console	0
12.3 NebulaGraph CPP	0
12.4 NebulaGraph Java	0
12.5 NebulaGraph Python	0
12.6 NebulaGraph Go	0
13. NebulaGraph Studio	0
13.1 认识 NebulaGraph Studio	0
13.2 安装与登录	0
13.3 快速开始	0
13.4 故障排查	0
14. NebulaGraph Dashboard（社区版）	0
14.1 什么是 NebulaGraph Dashboard（社区版）	0
14.2 部署 Dashboard 社区版	0
14.3 连接 Dashboard	0
14.4 Dashboard 页面介绍	0
14.5 监控指标说明	0
15. NebulaGraph Importer	0
15.1 功能	0
15.2 优势	0
15.3 版本兼容性	0
15.4 更新说明	0
15.5 前提条件	0
15.6 操作步骤	0
15.7 配置文件说明	0
16. NebulaGraph Exchange	0
16.1 认识 NebulaGraph Exchange	0
16.2 获取 NebulaGraph Exchange	0
16.3 参数说明	0
16.4 使用 NebulaGraph Exchange	0
16.5 Exchange 常见问题	0
17. NebulaGraph Operator	0
17.1 什么是 NebulaGraph Operator	0
17.2 使用流程	0
17.3 部署 NebulaGraph Operator	0
17.4 部署 NebulaGraph	0
17.5 通过 Nebular Operator 连接NebulaGraph	0

17.6 配置 NebulaGraph	0
17.7 故障自愈	0
17.8 常见问题	0
18. 图计算	0
18.1 NebulaGraph Algorithm	0
19. NebulaGraph Spark Connector	0
19.1 版本兼容性	0
19.2 适用场景	0
19.3 特性	0
19.4 更新说明	0
19.5 获取 NebulaGraph Spark Connector	0
19.6 使用方法	0
20. NebulaGraph Flink Connector	0
20.1 适用场景	0
20.2 更新说明	0
21. NebulaGraph Bench	0
21.1 适用场景	0
21.2 更新说明	0
22. 常见问题 FAQ	0
22.1 关于本手册	0
22.2 关于历史兼容性	0
22.3 关于执行报错	0
22.4 关于设计与功能	0
22.5 关于运维	0
22.6 关于连接	0
22.7 关于扩容、缩容	0
23. 附录	0
23.1 Release Note	0
23.2 NebulaGraph学习路径	0
23.3 生态工具概览	0
23.4 导入工具选择	0
23.5 产品端口全集	0
23.6 如何贡献代码和文档	0
23.7 NebulaGraph年表	0
23.8 思维导图	0
23.9 错误码	0

1. 欢迎阅读NebulaGraph 3.5.0 文档

**Note**

本文档更新时间2026-8-18, GitHub commit [a4013f9a](#)。该版本主色系为"胶蓝", 色号为 #66CCFF。

1.1 快速开始

- [快速开始](#)
- [部署要求](#)
- [nGQL 命令汇总](#)
- [FAQ](#)
- [生态工具](#)
- [Academy 课程](#)

1.2 最新发布

- [NebulaGraph 3.5.0](#)
- [Studio](#)
- [Dashboard](#)

1.3 其他资料

- [学习路径](#)
- [引用 NebulaGraph](#)
- [论坛](#)
- [主页](#)
- [系列视频](#)
- [英文文档](#)

1.4 图例说明

**Note**

额外的信息或者操作相关的提醒等。

**Caution**

需要严格遵守的注意事项。不遵守 **caution** 可能导致系统故障、数据丢失、安全问题等。



Danger

会引发危险的事项。不遵守 **danger** 必定会导致系统故障、数据丢失、安全问题等。



Performance

性能调优时需要注意的事项。



Faq

常见问题。



Compatibility

nGQL 与 openCypher 的兼容性或 nGQL 当前版本与历史版本的兼容性。



Enterpriseonly

描述社区版和企业版的差异。

1.5 修改文档中的错误

NebulaGraph文档以 **Markdown** 语言编写。单击文档标题右上侧的铅笔图标即可提交修改建议。

最后更新: 2026年8月18日

2. 简介

- **第一课：图的概念** (03 分 45 秒)

- **第二课：图的结构** (02 分 24 秒)

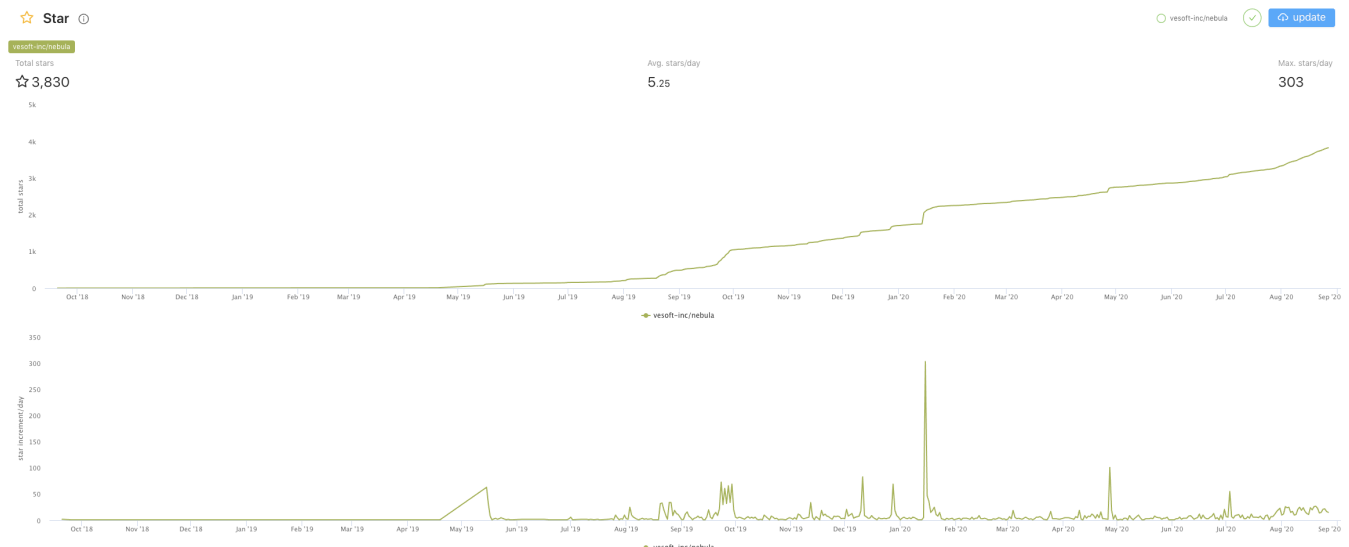
2.1 图

当前，从计算机行业巨头（例如 **Amazon** 和 **Facebook**）到小型研究团队，都投入了大量的资源探索图数据库在解决各种数据关系问题上的潜力。当然你也可以选择像他们这样进行尝试，现在可供选择的数据库有很多。那么图数据库究竟是什么？它可以做些什么？作为一类数据库，它在数据库领域里处于什么位置呢？要回答这些问题，我们首先得了解图。

图是计算机科学研究的主要领域之一。图能够高效地解决目前存在的诸多问题。本章将从图说起，继而说明图数据库的优点及其在现代应用程序开发中的巨大潜力，然后介绍分布式图数据库的区别和几种其他类型的数据库。

2.1.1 图、图片与图论

图无处不在。当听到图这个词时，很多人都会想到条形图或折线图，因为有时候我们确实会把它们称作图。从传统意义上来说，图是用来展示两个或多个数据系统之间的联系。最简单的例子如下图，下图展示了 **NebulaGraph** GitHub 仓库星星数量随时间推移的变化。

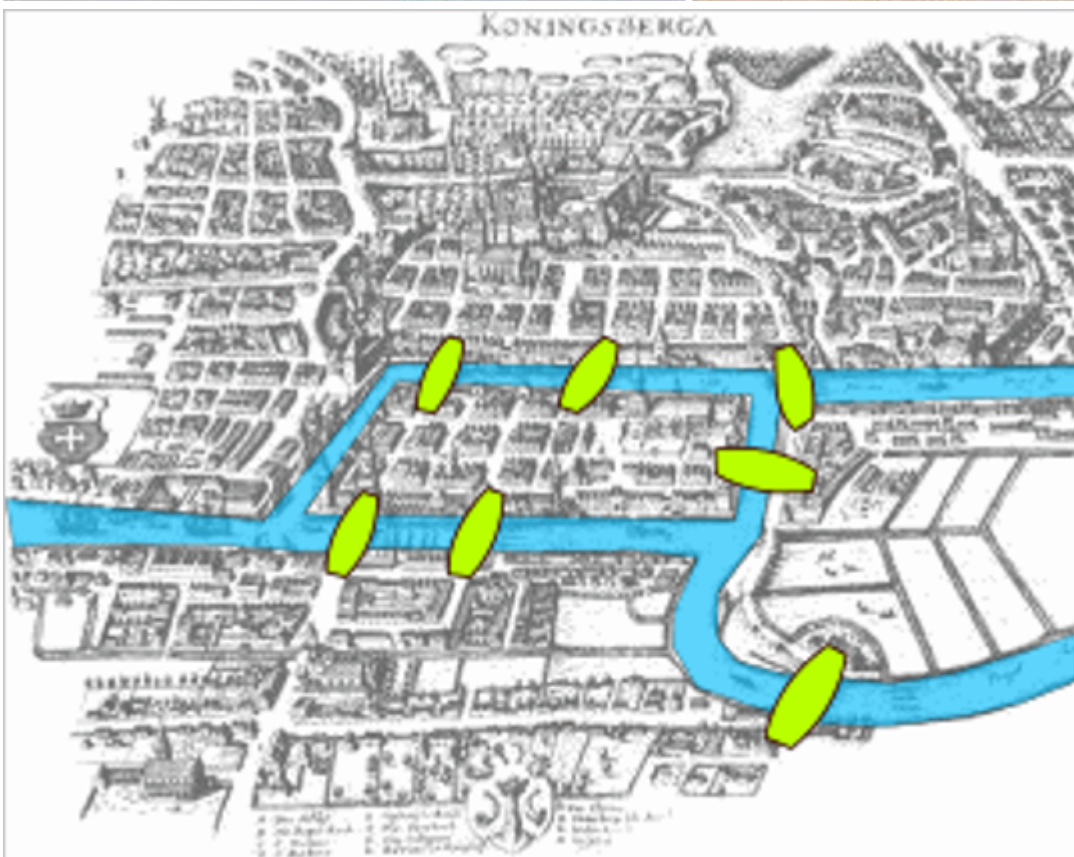


这是相对比较简单的一种图，横轴为时间，纵轴为星星数量。可以看到，星星数量是随着时间推移而上升的。这种类型的图通常称为折线图。折线图可以显示随时间（根据常用比例设置）而变化的连续数据。此处我们只给出了折线图的例子。当然图的形式有多种，比如饼图、条形图等。

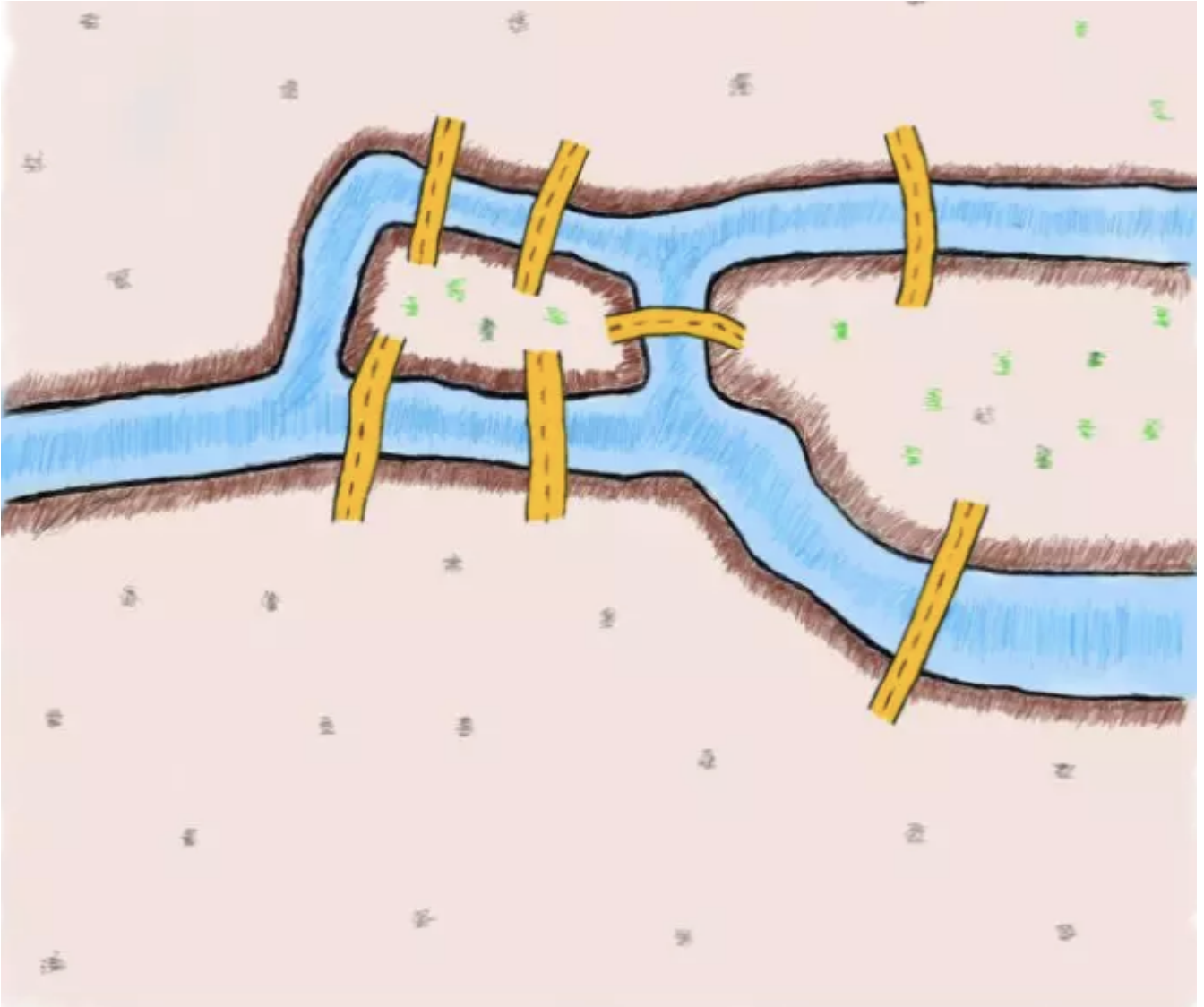
还有一种“图”在日常口语中会更多的被提及，例如，“图像识别”，“美图秀”，“修图”等。这些“图”都是指图片。

但是——总会有但是——我们在本书中讨论的图是另外一个概念——“图论”中的图。

图论始于 18 世纪初期的柯尼斯堡七桥问题。柯尼斯堡当时是普鲁士的城市，普雷格尔河穿过柯尼斯堡，不仅把柯尼斯堡分成了两部分，而且还在河中间形成了两个小岛。这就将整个城市分割成了四个区域，各区域由七座桥连接。在所有的桥都只能走一遍的前提下，如何能把这个地方所有的桥都走一遍呢？

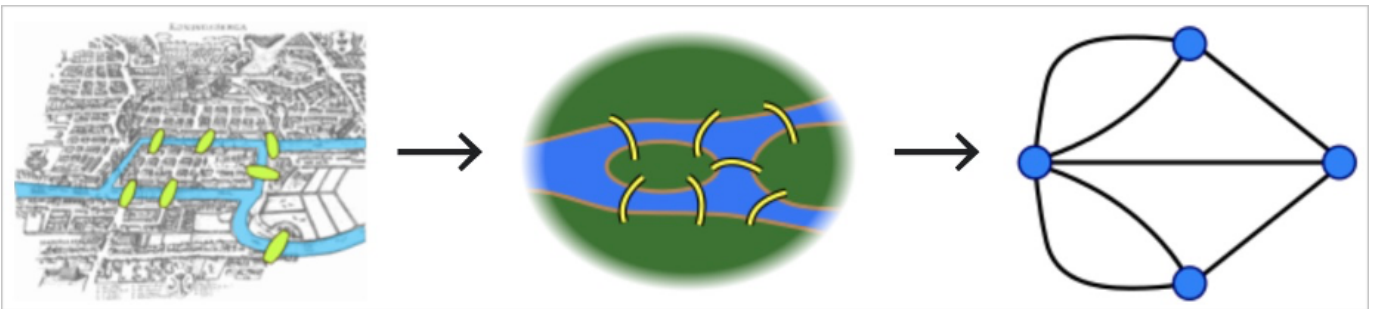


有兴趣的话可以试试寻找这个小游戏的答案¹。下图为柯尼斯堡七座桥的简化图。



大数学家欧拉于 1735 年提出，并没有方法能圆满解决这个问题，并在第二年发表论文，证明符合条件的走法并不存在。这篇论文在圣彼得堡科学院发表，成为图论史上第一篇重要文献。

在论文中，欧拉把实际的抽象问题简化为平面上的点与边组合，将城市的四个区域抽象成点，将连接城市的七座桥抽象成连接点的边。



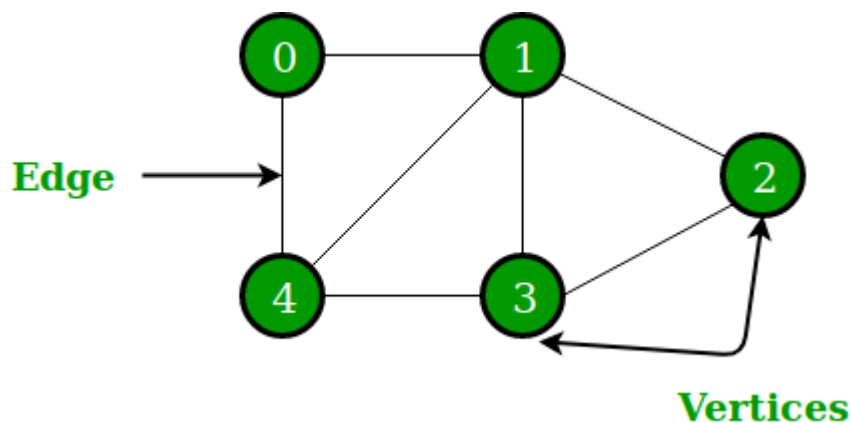
图中四个点代表柯尼斯堡的四个区域，点之间的线代表连接四个区域的七座桥。从图中不难看出，偶数座桥连接的区域可以轻松通过，因为来去可以选择不同的路线。奇数座桥连接的区域只能作为起点或者终点，因为同样的路线只能走一次。和节点相关联的边的条数称为节点度。现在可以证明，只有两个节点有奇数度，另外节点有偶数度时，也即两个区域必须有偶数座桥，剩下的区域有奇数座桥时，柯尼斯堡问题才能解决。欧拉论述了，由于柯尼斯堡七桥问题中所有区域都为奇数度，它无法实现符合题意的遍历。

欧拉发表的相关论文被认为是图论领域的第一篇文章，因此普遍认为欧拉是图论的创始人。

图论中的相关概念，我们将在后面的学习中提到。简单来说，图论就是研究图的学问。图是基本研究对象，用于表示实体与实体之间的关系。

在数学的分支图论中，图（Graph）是基本研究对象。在中文中，强调为“拓扑图”、“网络图”等。这一名词最早由西尔维斯特在 1878 年提出。他是著名的英国数学家、牛津大学几何教授，用图来表示数学和化学分子结构之间的关系。

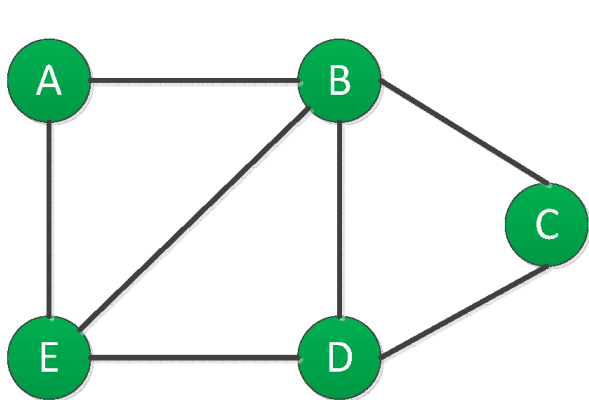
一张图由一些小圆点（称为顶点或节点，即 **Vertex**）和连接这些圆点的直线或曲线（称为边，即 **Edge**）组成。



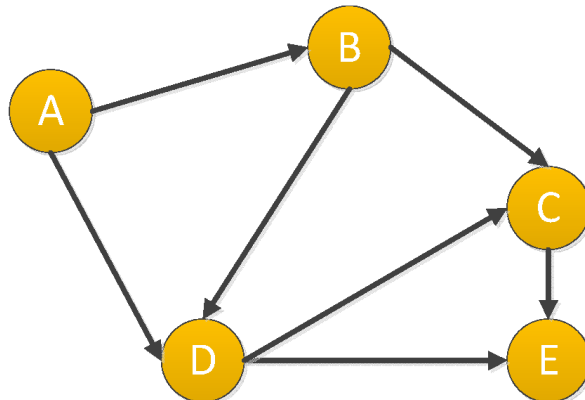
2.1.2 属性图

从数学角度来说，图论是研究建模对象之间关系结构的学科。但是从工业界使用的角度，通常会对基础的图模型进行扩展，称为属性图模型。属性图通常由以下几部分组成：

- 节点，即对象或实体。在本书中，通常简称为点（**Vertex**）。
- 节点之间的关系，在本书中，通常简称为边（**Edge**）。通常边是有方向或者无方向的，以表示两个实体之间有持续的关系。



A. 无向图

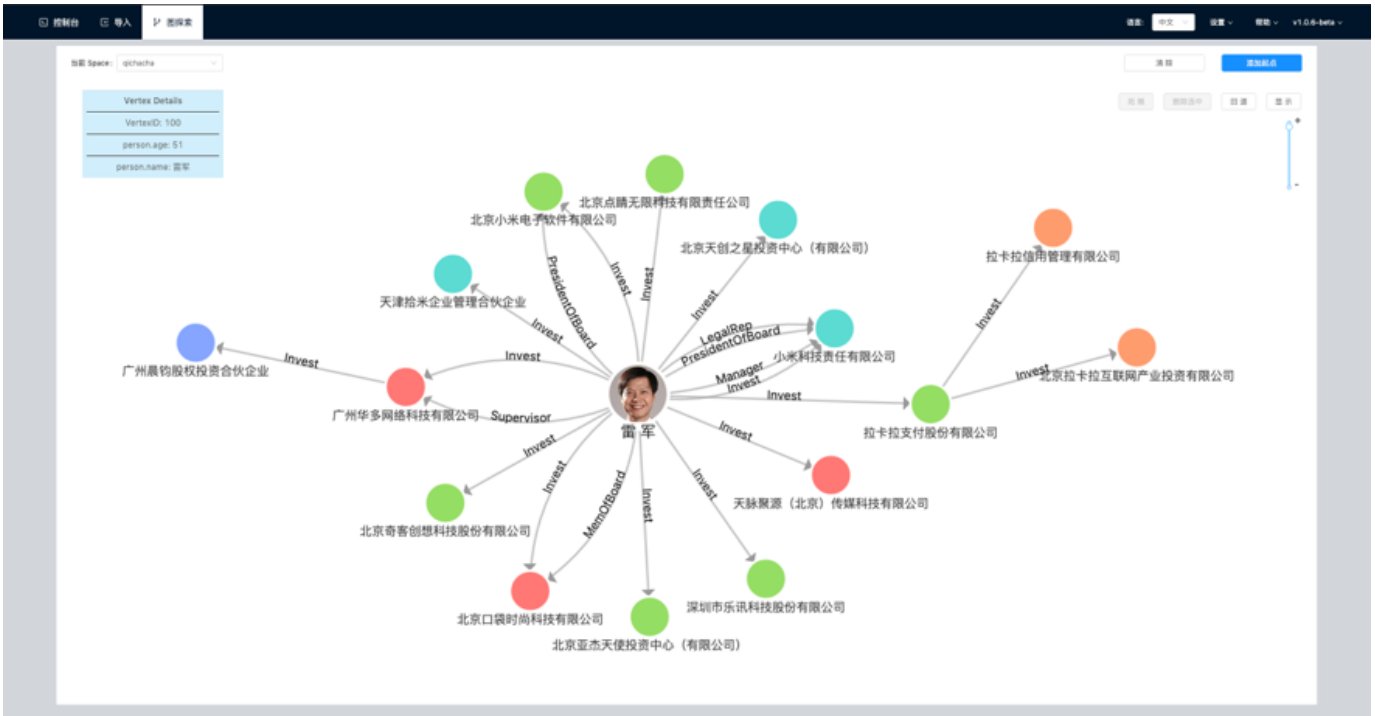


B. 有向图

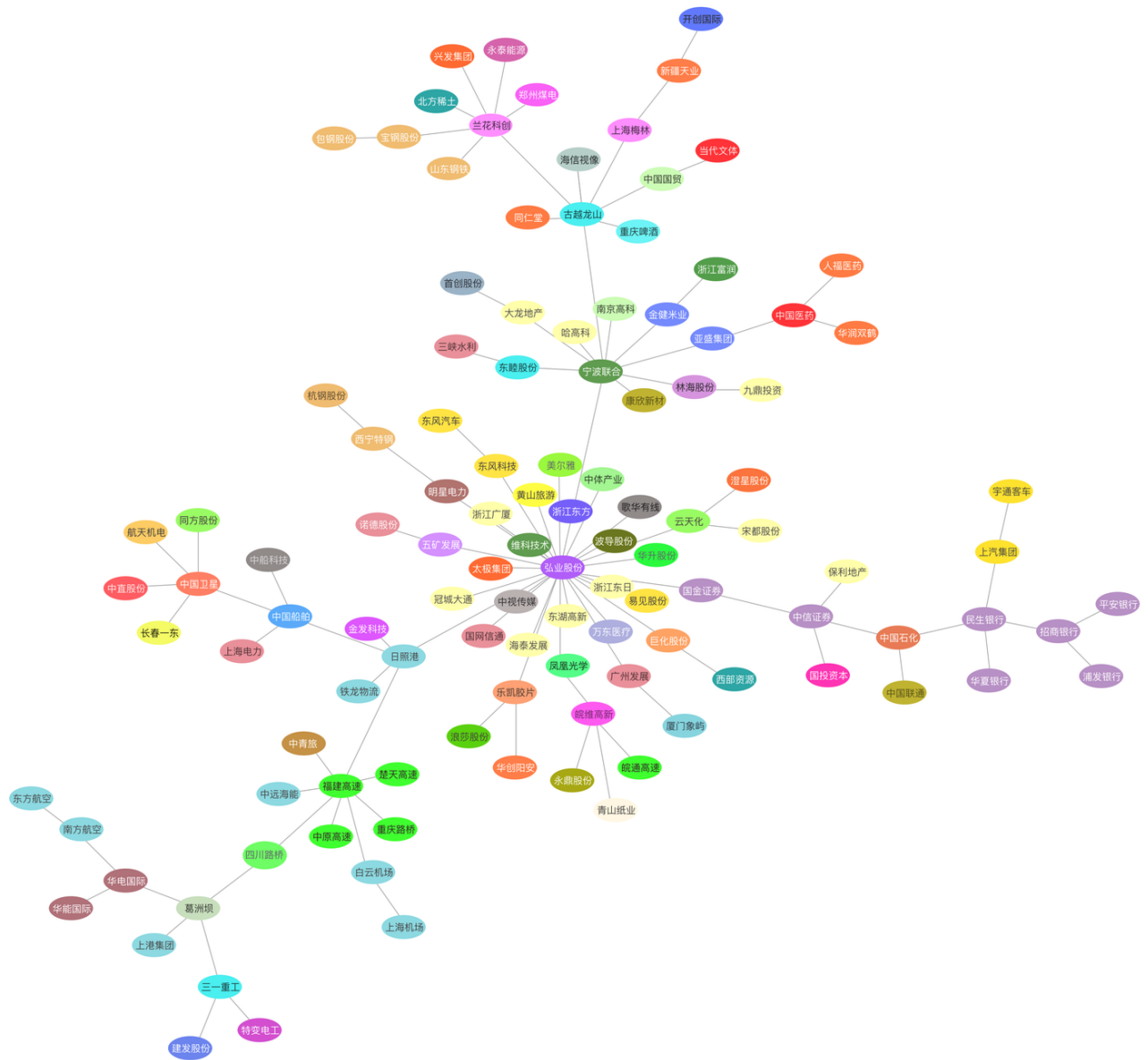
- 此外，在节点和边上，还可以有属性（**properties**）。

在现实生活中，有很多属性图的例子。

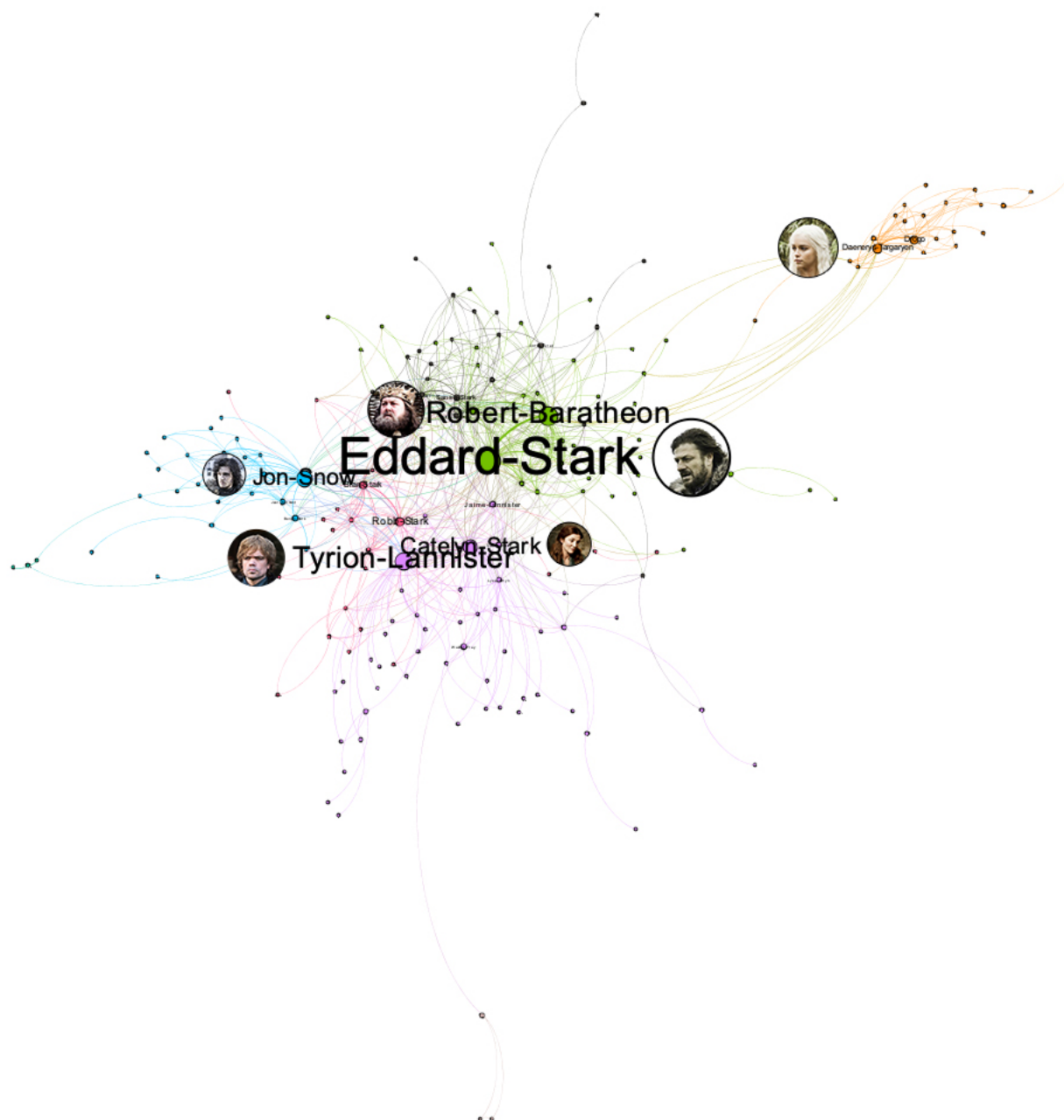
在网络理论中，图可以用来做可视化的社会网络分析，研究社会实体之间的关系结构。例如企查查或者 BOSS 直聘这类的公司，用图来建模商业股权关系网络。这个网络中，点通常是一个自然人或者是一家企业，边通常是某自然人与某企业之间的股权关系。点上的属性可以是自然人姓名、年龄、身份证号等。边上的属性可以是投资金额、投资时间、董监高等职位关系。



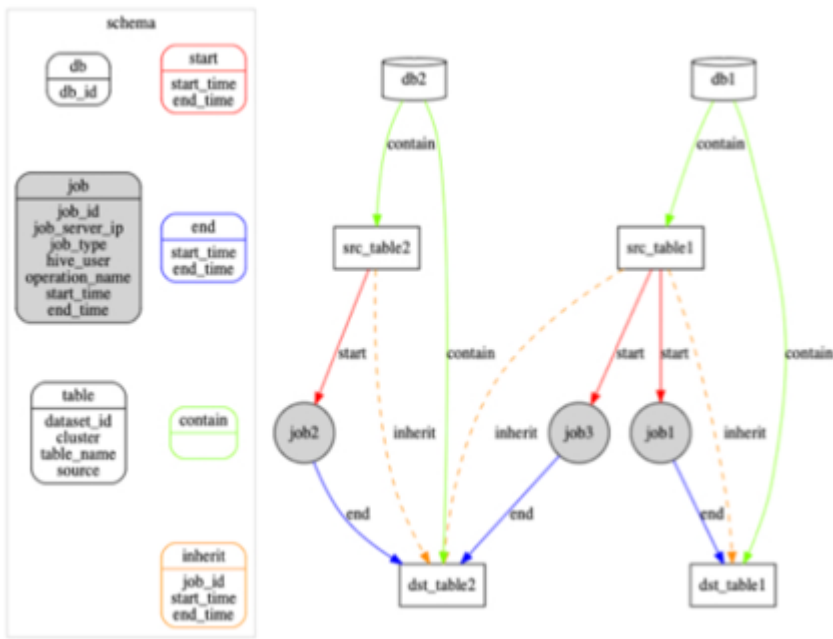
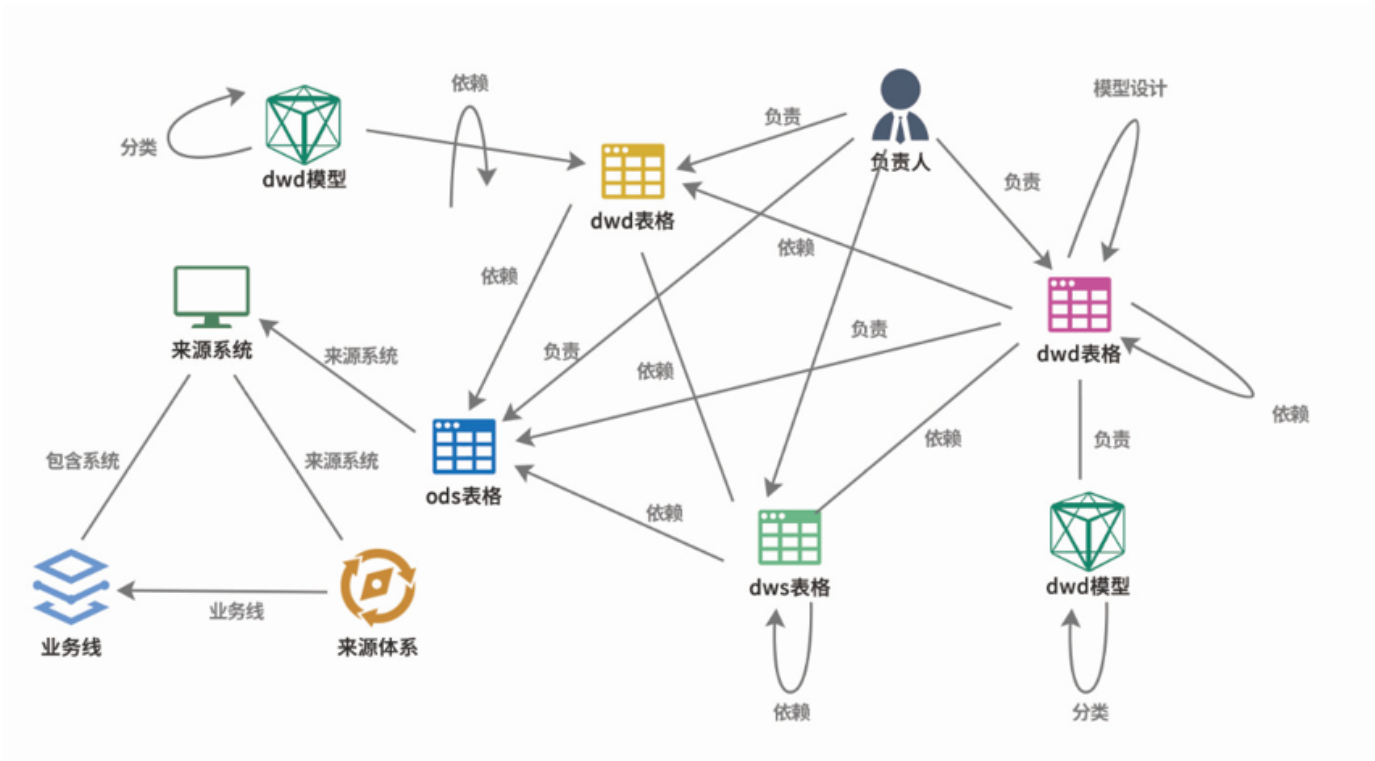
在一个股票市场里面，点可以是一家上市公司，边可以是上市公司之间的相关性。点的属性可以为股票代码、简称、市值、板块等；边的属性可以为股价的时间序列相关性系数²。



图关系还可以是类似《权力的游戏》这样电视剧中的人物关系网³：点为人物，边为人物之间的互动关系；点的属性为人物姓名、年龄、阵营等，边的属性（距离）为两个人物之间的互动次数，互动越频繁距离越近。



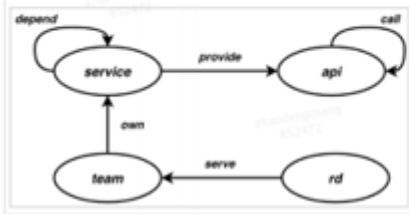
图也可以用于 IT 系统内部的治理。例如，对于像微众银行这样的公司，通常有着非常庞大的数据仓库，以及相应的数仓管理工具。这些管理工具记录了数仓内 **Hive** 表之间通过 **Job** 实现的 **ETL** 关系⁴，这样的 **ETL** 关系，可以非常方便的用图的形式呈现和管理，当出现问题时也可以非常方便地追溯根源。



图也可以用于记录一个大型 IT 系统内部错综复杂的微服务之间的调用关系⁵，运维团队用其进行服务治理。这里每个点表示一个微服务，边表示两个微服务之间的调用关系；这样，运维人员可以方便地寻找可用性低于阈值 (99.99%) 的调用链路，或者发现那些出故障会影响面特别大的微服务节点。

服务治理

- 图谱数据
 - 将RPC服务调用关系写入图谱
 - 包含service、api、team等4类实体及5类关系
 - 点边数量在百万级别，实时写入
- 用于服务链路治理和告警优化



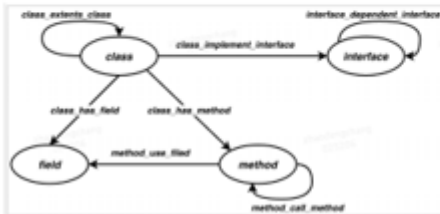
```
//查找API com.sankuai.ia.search.api:SearchControllerV2.search过去
七天可用率低于99.99%的链路的thrift调用，最大图遍历深度为10
GO 1 TO 10 STEPS FROM
hash("com.sankuai.ia.search.api:SearchControllerV2.
search") OVER call WHERE call.availability>0 AND
call.availability<1000000 AND
$$api.type=="mtthrift" YIELD call._src,call._dst
```

```
//查找所有java类型服务提供的API，并统计其会影响的上游API的数
量，从高到低排序着影响次数大小（调用的可用率小于4个9）
LOOKUP ON service WHERE service.type=="java"
| GO FROM $-.VertexID OVER provider YIELD
provider._dst AS java_api_id
| GO FROM $-.java_api_id OVER call REVERSELY WHERE
call.availability>0 AND call.availability<1000000
YIELD call._src AS api_src, call._dst AS api_dst
| GROUP BY $-.api_src YIELD $-.api_src AS api_id,
count(1) AS call_cnt
| ORDER BY call_cnt DESC
| FETCH PROP ON api $-.api_id YIELD
api.appkey,api.method,$-.call_cnt
```

图也可以用于提升代码开发效率。用图存放代码之间的函数调用关系⁵，可以提升研发团队审查和测试代码的效率。在这样的图中，每个点是代码中的一个函数或者变量，每个边是函数或者变量之间的调用关系。当有新提交的代码之时，人们可以更方便的看到可能会受到影响到其他接口，这样可以帮助测试人员更好的评估潜在的上线风险。

代码依赖分析

- 图谱数据
 - 将公司代码库中代码的依赖关系写入图谱
 - 包含method、field、class、interface等4类顶点、7类关系
 - 点边数量在千万级别，实时写入
- 用于QA精准测试
 - RD向代码仓库提交PR后，能查询出所修改代码能影响到的外部接口，并展示调用路径

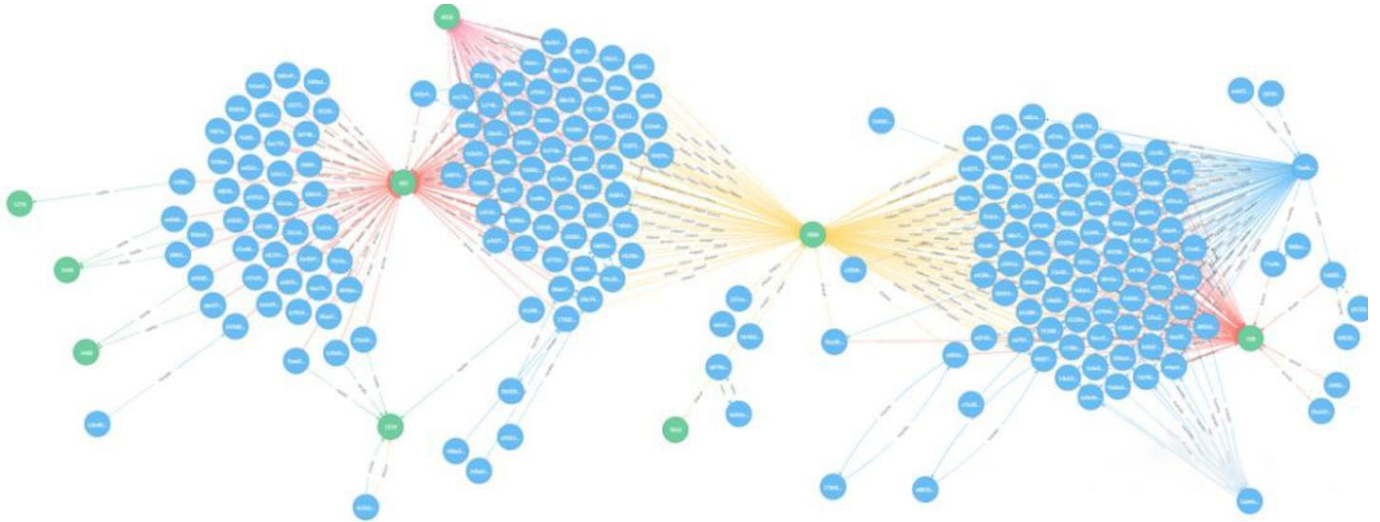


```
//查找最外层method到某个method的所有无环路径
GO 1 TO 30 STEPS FROM 2946345526231222882 OVER
method_call_method REVERSELY YIELD DISTINCT
method_call_method._dst AS id
MINUS
GO 1 TO 30 STEPS FROM 2946345526231222882 OVER
method_call_method REVERSELY YIELD DISTINCT
method_call_method._src as id )
| FIND NOLOOP path FROM $-.id TO
2946345526231222882 OVER method_call_method UPTO
30 STEPS
```

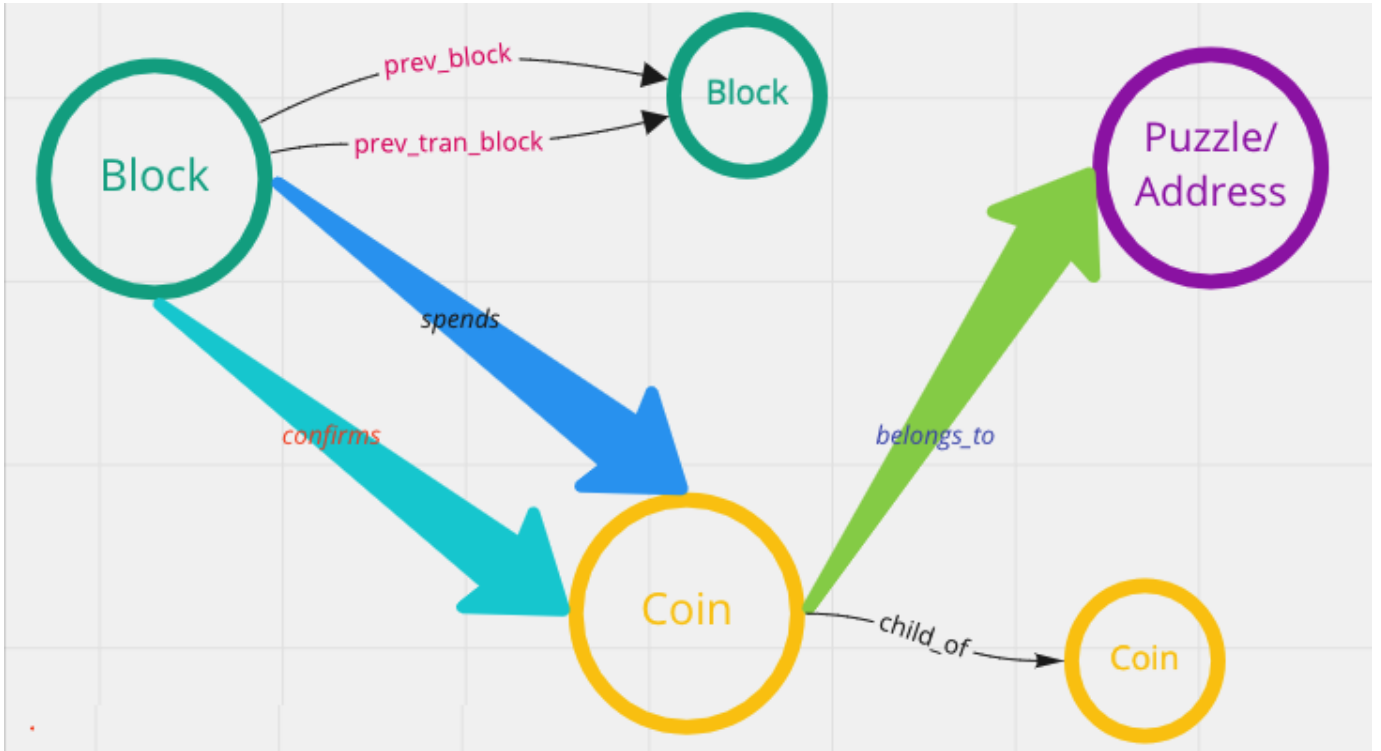
```
//确认两个method间是否有路径
FIND SINGLE SHORTEST PATH FROM hash("method1")
TO hash("method2") OVER method_call_method UPTO
30 STEPS
```

此外，相对于静态不发生变化的属性图，我们还可以通过增加一些时间信息，发掘出更多的使用场景。

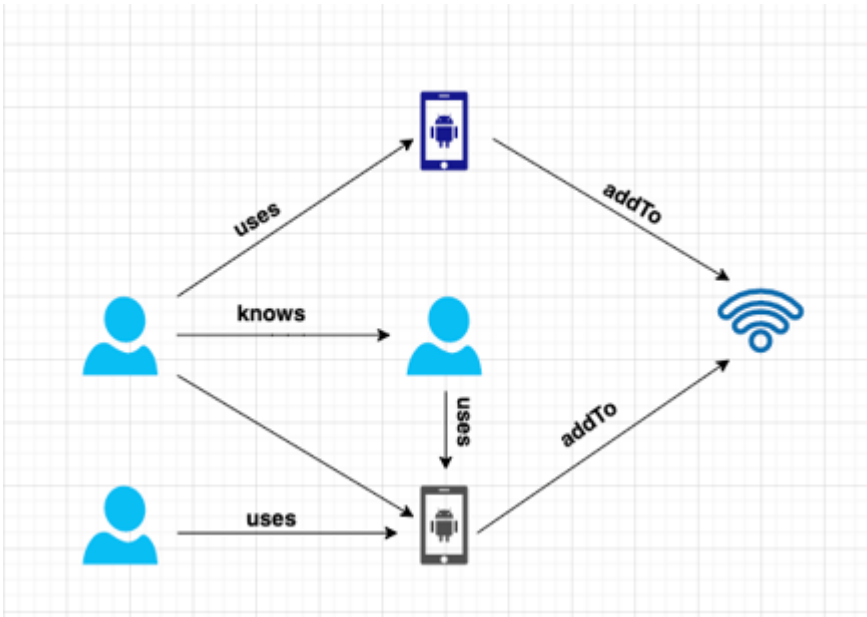
例如，在一个银行间账户资金流向网络里面⁶，点是账户，边是账户之间的转账记录。边属性记录了转账的时间、金额等。同盾、邦盛、半云科技等公司采用图技术，可以方便地通过图的方式探索发现明显的资金挪用、“以贷还贷”、“团伙贷款”等现象。



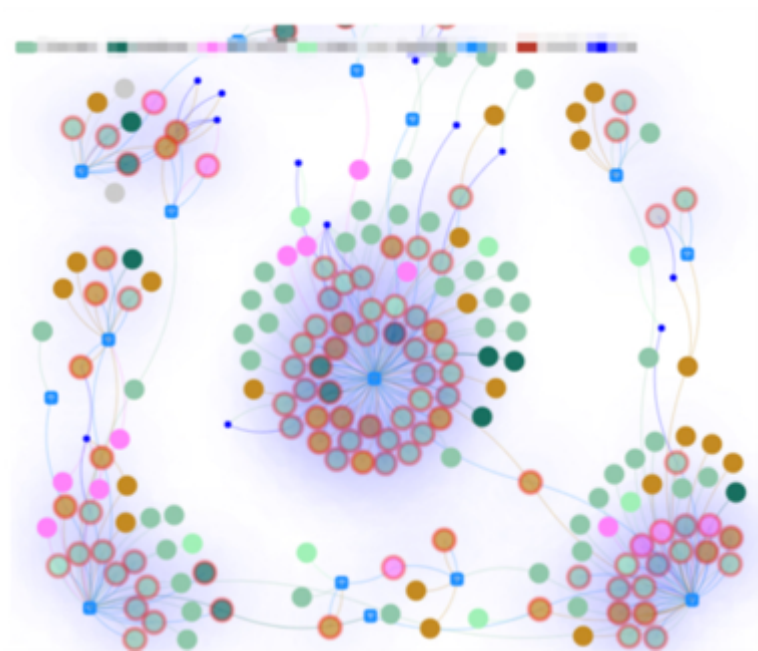
同样的方法也可以用于探索发现加密货币的流向。



在一个黑产账户和设备网络中⁷，其中的点可以是账户、手机设备和 WIFI 网络，边是这些账户与手机设备之间的登录关系，以及手机设备和 WIFI 网络之间的接入关系。



这些登录记录的网络构成了黑产群体网络的团伙作案特征。360 数科⁷、快手⁸、微信⁹、知乎¹⁰、携程金融这些公司都通过图技术实时（毫秒级的）识别超过百万个的黑产社群。

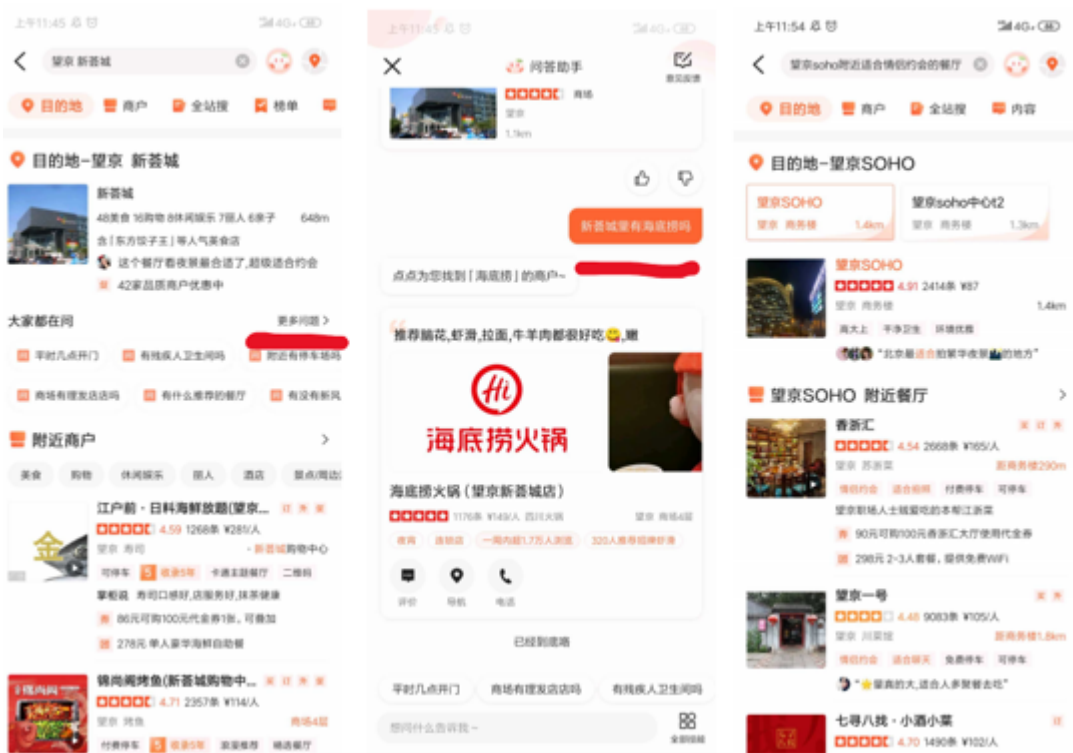
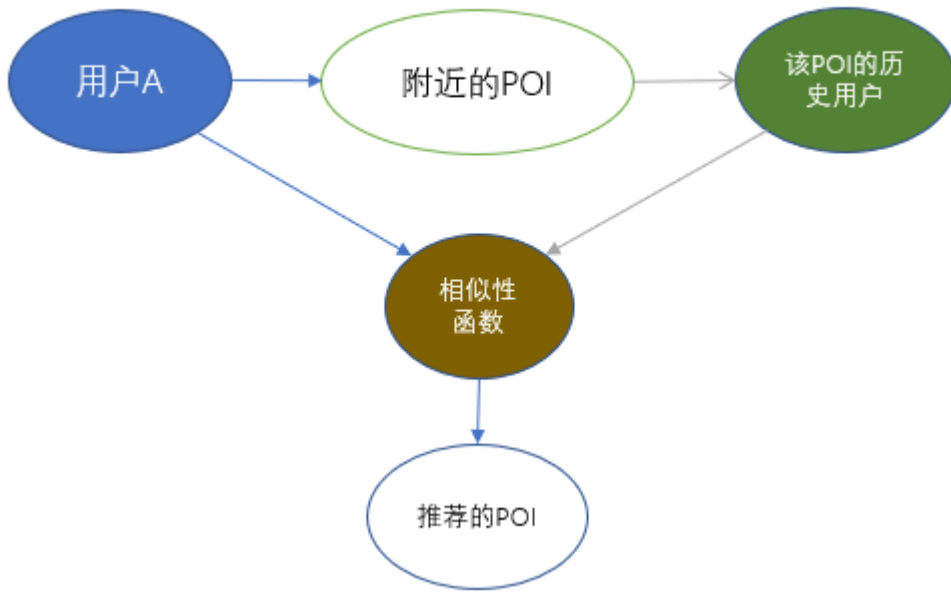


更进一步，除了时间这个维度外，我们通过添加一些地理空间信息，还能发现属性图更多的应用场景。

例如新冠病毒的流行病学溯源¹¹，点是人物，边是人与人之间的接触；点属性为人物的身份证、发病时间等信息，边属性为人物之间发生密切接触的时间和地理空间等。为卫生防疫部门快速识别高风险人群和其行为轨迹提供帮助。

image

地理空间与图的结合也可以用于一些 O2O 的场景，例如基于 POI（Point-of-Interest）的实时美食推荐¹²，使得美团这类本地生活服务平台公司能在消费者在打开 APP 的时候，实时推荐出更为合适的商家。



图还可以用于更深度的知识推理，华为、vivo、OPPO、微信、美团等公司，将图用于表征底层知识关系的数据模型。

由此可见，图可用于在物理、生物、社会和信息系统中建模许多类型的关系和过程，许多实际问题可以用图来表示。因此，图论成为运筹学、控制论、信息论、网络理论、社会科学、语言学、计算机科学等众多学科强有力的数学工具。

2.1.3 为什么要使用图数据库

虽然关系型数据库与 XML/JSON 等半结构类型的数据库，都可以用来描述图结构的数据模型，但是，图（数据库）不仅可以描述图结构与存储数据本身，更着眼于处理数据之间的关联（拓扑）关系。具体来说，图（数据库）有这么几个优点：

- 图是一种更直观、更符合人脑思考直觉的知识表示方式。这使得我们在抽象业务问题时，可以着眼于“业务问题本身”，而不是“如何将问题描述为数据库的某种特定结构（例如表格结构）”。
- 图更容易展现数据的特征，例如转账的路径、近邻的社区。例如，如果要分析《权力的游戏》中的人物派别关系和人物重要性，表的组织方式如下：

image

这显然不如下方图的组织方式直观：

image

特别是当某些中心节点被删除：

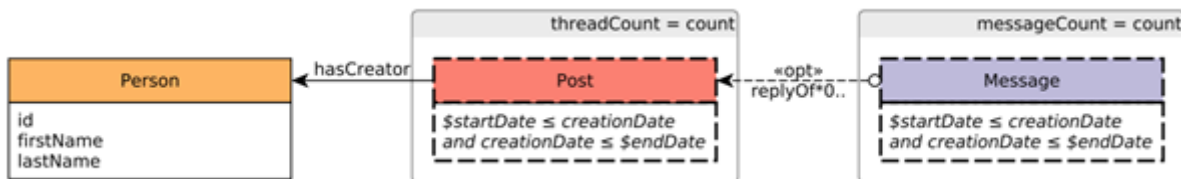
image

或者，增加一条边，可以彻底地改变整个图拓扑：

image

虽然只是个别数据的细微改变，图可以比表更直观地表现其中的重要而系统的信息。

- 图查询语言是针对图结构访问设计的，可以更加直观。例如，下面是一个 LDBC 中的查询示例，要求：查找某人（Person）在社交网络上发布的帖子（Posts）；查找相应的回复（Message，回复本身还会被多次回复）；发帖时间、回帖时间都满足一定条件；根据回帖数量对结果排序。



如果使用 PostgreSQL 编写查询语句：

```
--PostgreSQL
WITH RECURSIVE post_all(psa_threadid
, psa_thread_creatorid, psa_messageid
, psa_creationdate, psa_messagetype
) AS (
SELECT m_messageid AS psa_threadid
, m_creatorid AS psa_thread_creatorid
, m_messageid AS psa_messageid
, m_creationdate, 'Post'
FROM message
WHERE 1=1 AND m_c_replyof IS NULL -- post, not comment
AND m_creationdate BETWEEN :startDate AND :endDate
UNION ALL
SELECT psa.psa_threadid AS psa_threadid
, psa.psa_thread_creatorid AS psa_thread_creatorid
, m_messageid, m_creationdate, 'Comment'
FROM message p, post_all psa
WHERE 1=1 AND p.m_c_replyof = psa.psa_messageid
AND m_creationdate BETWEEN :startDate AND :endDate
)
SELECT p.p_personid AS "person.id"
, p.p_firstname AS "person.firstName"
, p.p_lastname AS "person.lastName"
, count(DISTINCT psa.psa_threadid) AS threadCount
END) AS messageCount
, count(DISTINCT psa.psa_messageid) AS messageCount
FROM person p left join post_all psa on (
1=1 AND p.p_personid = psa.psa_thread_creatorid
AND psa_creationdate BETWEEN :startDate AND :endDate
)
GROUP BY p.p_personid, p.p_firstname, p.p_lastname
ORDER BY messageCount DESC, p.p_personid
LIMIT 100;
```

如果使用为图专门设计的图语言 Cypher 编写查询语句：

```
--Cypher
MATCH (person:Person)-[:HAS_CREATOR]-(post:Post)-[:REPLY_OF*0..]-(reply:Message)
WHERE post.creationDate >= $startDate AND post.creationDate <= $endDate
AND reply.creationDate >= $startDate AND reply.creationDate <= $endDate
RETURN
person.id, person.firstName, person.lastName, count(DISTINCT post) AS threadCount,
count(DISTINCT reply) AS messageCount
ORDER BY
messageCount DESC, person.id ASC
LIMIT 100
```

- 由于存储引擎和查询引擎可以针对图的结构专门设计，图的遍历（对应 SQL 中的 join）要高效得多。下图是知名产品 Neo4j 所做的一个对比¹²。

深度	关系型数据库的执行时间(s)	Neo4j的执行时间(s)	返回的记录条数
2	0.016	0.01	~2500
3	30.267	0.168	~110000
4	1543.505	1.359	~600000
5	未完成	2.132	~800000

关系数据库 vs 图数据库(多跳查询)

- 图数据库具有广泛的适用场景。例如数据集成（知识图谱）、个性化推荐、欺诈与威胁检测、风险分析与合规、身份（与控制权）验证、IT 基础设施管理、供应链与物流、社交网络研究等。
- 根据文献¹³的统计，使用图技术最多的领域，依次是：信息技术(IT)、学术界研究、金融、工业界实验室、政府、医疗健康、国防、制药业、零售与电子商务、交通运输、电信、保险。
- 2019 年，根据 Gartner 的问卷调查，27% 的客户（500 组）在使用图数据库，20% 有计划使用。

2.1.4 RDF

受篇幅所限，本章不讨论 RDF 数据模型。

-
1. 图片来源 <https://medium.freecodecamp.org/i-dont-understand-graph-theory-1c96572a1401>. ↩
 2. <https://nebula-graph.com.cn/posts/stock-interrelation-analysis-jgraph-nebula-graph/> ↩
 3. <https://nebula-graph.com.cn/posts/game-of-thrones-relationship-networkx-gephi-nebula-graph/> ↩
 4. <https://nebula-graph.com.cn/posts/practicing-nebula-graph-webank/> ↩
 5. <https://nebula-graph.com.cn/posts/meituan-graph-database-platform-practice/> ↩↩
 6. <https://zhuanlan.zhihu.com/p/90635957> ↩
 7. <https://nebula-graph.com.cn/posts/graph-database-data-connections-insight/> ↩↩
 8. <https://nebula-graph.com.cn/posts/kuaishou-security-intelligence-platform-with-nebula-graph/> ↩
 9. <https://nebula-graph.com.cn/posts/nebula-graph-for-social-networking/> ↩
 10. <https://mp.weixin.qq.com/s/K2QinpR5Rplw1teHpHtf4w> ↩
 11. <https://nebula-graph.com.cn/posts/detect-corona-virus-spreading-with-graph-database/> ↩
 12. <https://nebula-graph.com.cn/posts/meituan-graph-database-platform-practice/> ↩↩
 13. <https://arxiv.org/abs/1709.03188> ↩
-

最后更新: 2026年8月18日

2.2 图数据库的市场概况

既然已经讨论了什么是图，接下来让我们进一步认识基于图论和属性图模型发展起来的图数据库。

不同的图数据库在术语方面可能会略有不同，但是归根结底都是在讲点、边和属性。至于更多的功能，例如标签、索引、约束、TTL、长任务、存储过程和UDF等这些高级功能，在不同图数据库中，会存在明显的差异。

图数据库用图来存储数据，而图是最接近高度灵活、高性能的数据结构之一。图数据库是一种专门用于存储和检索庞大信息网的存储引擎，它能够高效地将数据存储为点和边，并允许对这些点边结构进行高性能的检索和查询。我们也可以为这些点和边添加属性。

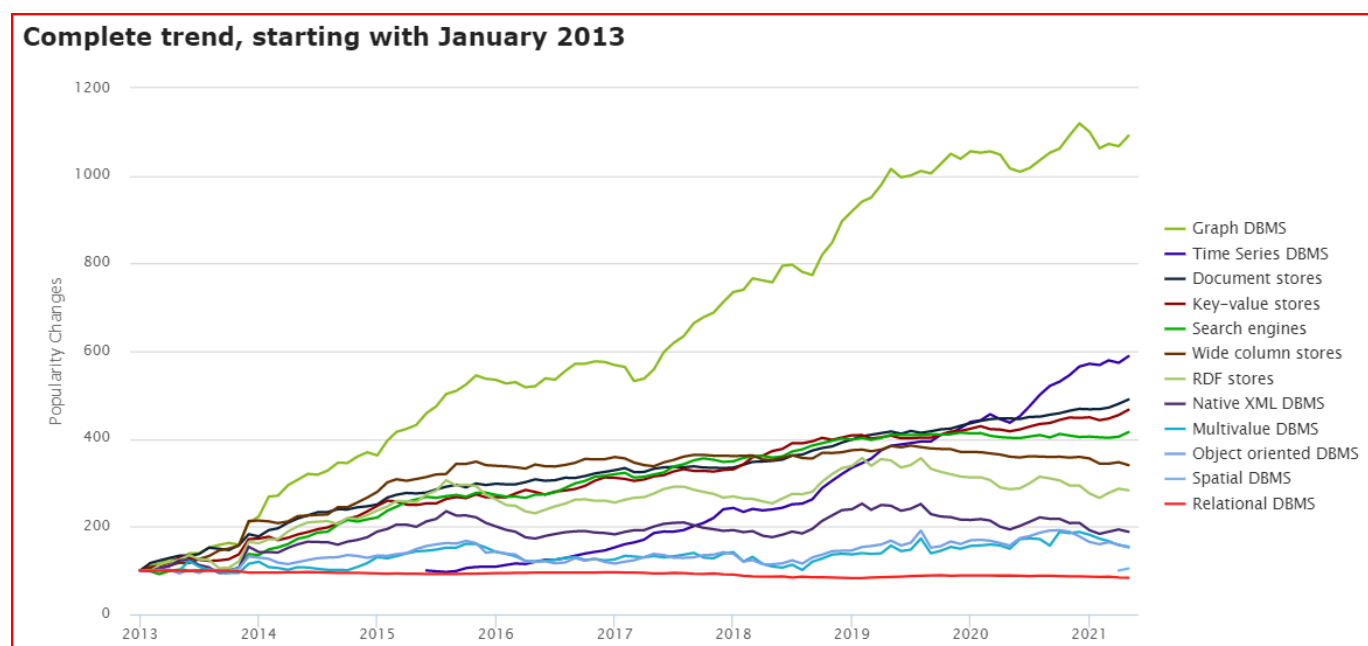
图数据库几乎适用于存储所有领域的数据。因为在几乎所有领域中，事物之间都是由某种相关联的。图数据库支持存储实体之间的丰富关系，并且能够将这些关系完美地呈现出来，而无需像其他建模方式那样，将关系也当成实体存储。因此图数据库能够以最接近对数据直观认知的形式存储数据。

2.2.1 三方机构的统计和预测

DB-Engines 的统计

根据世界知名的数据库排名网站DB-Engines.com的统计，图数据库至2013年以来，一直是“增速最快”的数据库类别¹。

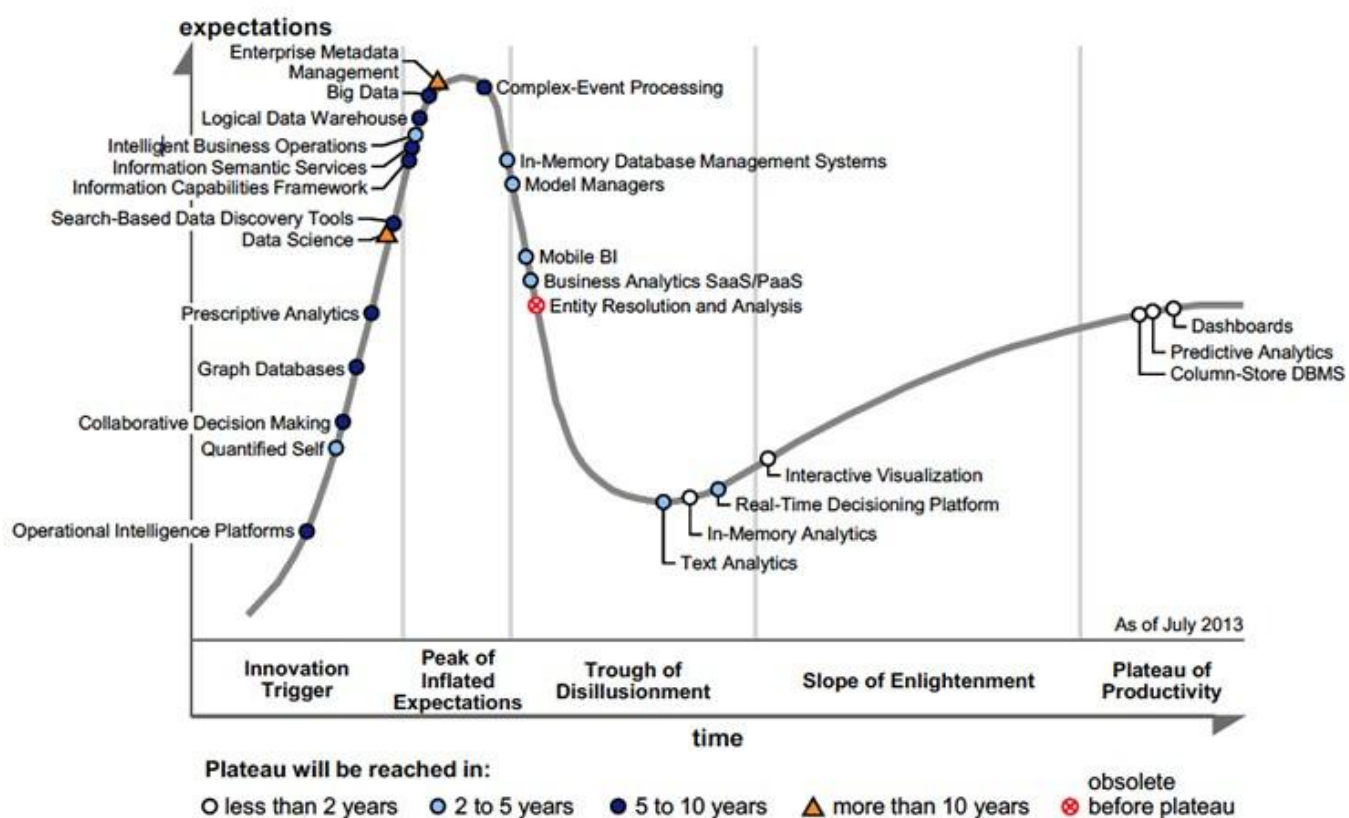
该网站根据一些指标来统计每种类别的数据库的流行度变化趋势，这些指标包括基于Google等搜索引擎的收录和趋势情况、主要IT技术论坛和社交网站上讨论的技术话题、招聘网站的职位变化等。该网站共收录了371种数据库产品，并分为12个类别。这12个类别中，图数据库这种类别的增速远远快于其他任何的类别。



Gartner 的预测

世界顶级智库Gartner早在2013年之前²，就将图数据库作为主要的“商业智能与分析技术趋势”。在那个时候，Big Data正火热的如日中天，数据科学家更是炙手可热的职位。

Figure 1. Hype Cycle for Business Intelligence and Analytics, 2013



BI = business intelligence; DBMS = database management system; SaaS = software as a service; PaaS = platform as a service

直到最近，图数据库及相关的图技术依旧是"2021年十大数据与分析趋势"³：

Gartner Top 10 Data and Analytics Trends, 2021



Accelerating Change

- 1** Smarter, Responsible, Scalable AI
- 2** Composable Data and Analytics
- 3** Data Fabric Is the Foundation
- 4** From Big to Small and Wide Data



Operationalizing Business Value

- 5** XOps
- 6** Engineering Decision Intelligence
- 7** D&A as a Core Business Function



Distributed Everything

- 8** Graph Relates Everything
- 9** The Rise of the Augmented Consumer
- 10** D&A at the Edge

gartner.com/SmarterWithGartner

Source: Gartner
© 2021 Gartner, Inc. All rights reserved. CTMKT_1164473

Gartner

趋势八：图技术使一切产生关联（Graph Relates Everything）

图技术已成为许多现代数据和分析能力的基础，能够在不同的数据资产中发现人、地点、事物、事件和位置之间的关系。数据和分析领导者依靠图技术快速回答需要在了解情况并理解多个实体之间的联系和优势的性质的复杂业务问题。

Gartner预测，到2025年图技术在数据和分析创新中的占比将从2021年的10%上升到80%。该技术将促进整个企业机构的快速决策。

可以注意到，Gartner 的预测比较好的吻合了 DB-Engines 的统计结论。技术的进步并不是完全线性的，通常会有一段快速发展的泡沫期，然后进入一段平台期，之后由于新的技术的出现产生新一轮的泡沫期，再经历一段平台期。以此往复螺旋形的循环发展。

对于市场规模的预测

根据 [verifiedmarketresearch⁴](#), [fnfresearch⁵](#), [marketsandmarkets⁶](#), 以及 [gartner⁷](#) 等智库的统计和预测, 图数据库市场 (包括云服务) 规模在2019年大约是8亿美元, 将在未来6年保持25%左右的年复合增长(CAGR)至 30-40 亿美元, 这大约对应于全球数据库市场 5-10% 的市场份额。

Image

2.2.2 市场参与者

(第一代) 图数据库的先行者 **Neo4j**

虽然在 1970 年代, 人们已经提出了一些类似于“图”的数据模型和产品原型 (例如 [CODASYL⁸](#))和相应的图语言 G/G+ 语言⁹。但真正能够让“图数据库”这个概念流行起来, 不得不说到这个市场最主要的先行者 **Neo4j**, 甚至(标签)属性图和图数据库这两个主要术语就是 **Neo4j** 最早提出并实践的。

本小节关于**Neo4j**和其创造的图查询语言**Cypher**的历史内容主要摘录自 **ISO WG3** 的工作论文“**An overview of the recent history of Graph Query Languages**”¹⁰ 和⁹, 本书作者根据最新两年的进展有删减和更新。

关于图查询语言 (Graph Query Language, GQL) 和国际标准的制定

熟悉数据库的读者可能都知道结构化查询语言SQL。通过使用SQL, 人们以接近自然语言的方式访问数据库。在 SQL 被广泛采用和标准化之前, 关系型数据库的市场是非常碎片和割裂的——各家厂商的产品都有完全不同的接入访问方式, 数据库产品自身的开发人员、数据库产品周边工具的开发人员、数据库最终的使用人员, 都不得不学习各个厂商的完全不同的产品, 在不同产品之间迁移极其困难。当1989年SQL-89标准被制定后, 整个关系型数据库的市场快速收敛到SQL-89上。这大大降低了上述各种人员的学习曲线。

类似的, 在图数据库领域, 图语言(GQL)承担了类似于SQL的作用, 是一种用户与图数据库主要的交互方式。但不同于SQL-89这种国际标准, GQL还没有任何国际标准。目前有两种主流的图语言:

Neo4j的Cypher (及其后续——ISO正在制定过程中的 GQL-standard 草案)和Apache TinkerPop的Gremlin。前者通常被称为声明式语言 (Declarative query language)——也即用户只需要告诉系统“要什么”, 而不管“怎么做”; 后者通常被称为命令式语言 (Imperative query language), 用户会显式地指定系统的操作。

GQL国际标准正在制定过程中。

年表简述

- 2000 年, Neo4j 的创始人产生将数据建模成网络 (network) 的想法。
- 2001 年, Neo4j 开发了最早的核心部分代码。
- 2007 年, Neo4j 开始以一个公司的方式运作。
- 2009 年, Neo4j 团队借鉴 XPath 作为图查询语言, Gremlin¹¹最初也是基于这个想法。
- 2010 年, Neo4j 的员工 Marko Rodriguez 采用术语属性图 (Property Graph) 来描述 Neo4j 和 Tinkerpop / Gremlin 的数据模型。
- 2011 年, 第一个公开发行业版本 Neo4j 1.4; 并发布了Cypher的第一个版本。
- 2012 年, Neo4j 1.8 为 Cypher 增加写入图的能力。Neo4j 2.0 增加了标签和索引, Cypher 成为一种声明式的语言。
- 2015 年, Neo4j 将 Cypher 开源为 openCypher。
- 2017 年, ISO WG3 工作组开始讨论如何将属性图查询能力引入 SQL。
- 2018 年 12 月, 从 Neo4j 3.5 开始其核心部分转为闭源。
- 2019 年, ISO 正式立项两个项目 (ISO/IEC JTC 1 N 14279 和 ISO/IEC JTC 1/SC 32 N 3228), 启动关于图数据库语言国际标准的制定工作。
- 2021 年, Neo4j 完成 F 轮 3.25 亿美元的融资, 是整个数据库 (包括关系型) 历史上最大一轮融资。

NEO4J 的早期历史

Neo4j 和属性图这种数据模型, 最早构想于 2000 年。Neo4j 的创始人当时在开发一个媒体管理系统, 所使用的数据库的 schema 经常会发生重大变化。为了支持这种灵活性, Neo4j 的联合创始人 Peter Neubauer, 受到 Informix Cocoon 的启发, 希望将系统能够建模为一种概念相互连接的网络。印度理工学院孟买分校的一群研究生们实现了最早的原型。Neo4j 的联合创始人 Emil Eifrem 和这些学生们花了一周的时间, 将 Peter

最初的想法扩展成为一个更抽象的模型：节点通过关系连接，**key-value** 作为节点和关系的属性。这群人开发了一个 **Java API** 来和这种数据模型交互，并在关系型数据库之上实现了一个抽象层。

虽然这种网络模型极大的提高了生产力，但是性能一直很差。所以 **Neo4j** 联合创始人 **Johan Svensson** 花了不少精力，为这种网络模型实现了一个原生的数据管理系统。这个就成为了 **Neo4j**。在最初的几年，**Neo4j** 作为一个内部产品很成功。在 2007 年，**Neo4j** 的知识产权转移给了一家独立的数据库公司。

在 **Neo4j** 的第一个公开发行版中（**Neo4j 1.4**, 2011 年），数据模型由节点和有类型的边构成，节点和边都有 **key-value** 组成的属性。**Neo4j** 的早期版本没有任何的索引，应用程序只能从根节点开始自己构造查询结构（**search structure**）。因为这样对于应用程序非常笨重，**Neo4j 2.0**（2013.12）引入了一个新概念——点上的标签（**label**）。基于点标签，**Neo4j** 可以为一些预定义的节点属性建立索引。

"节点"、"关系"、"属性"、"关系只能有一个标签"、"节点可以有零个或者多个标签"，以上这些概念构成了 **Neo4j** 属性图的数据模型定义。随着后来增加的索引功能，让 **Cypher** 成为了与 **Neo4j** 交互的主要方式。因为这样应用程序开发者只需要关注于数据本身，而不是上段提到的那个开发者自己构建的查询结构（**search structure**）。

GREMLIN 的创造

Gremlin是基于**Apache TinkerPop**开发的图语言，其风格接近于一连串的函数（过程）调用。最初 **Neo4j** 的查询方式是通过 **Java API**。应用程序可以将查询引擎作为库(library)嵌入到应用程序中，然后使用 **API** 来查询图。

就在这段时间，**NoSQL** 这个概念开始出现。**NoSQL** 型的数据库引擎一般用 **REST** 和 **HTTP** 来交互和查询。**Neo4j** 的早期员工 **Tobias Lindaaker**、**Ivarsson**、**Peter Neubauer**、**Marko Rodriguez**用 **XPath** 作为图查询，**Groovy** 提供循环结构，分支和计算（等图灵完毕的功能）。这个就是 **Gremlin** 最初的原型。2009 年 11 月发布了第一个版本。

后来，**Marko** 发现同时用两种不同的解析器（**XPath** 和 **Groovy**）有很多问题，就将 **Gremlin** 改为基于 **Groovy** 的一种领域特定语言（**DSL**）。

CYPHER 的创造

Gremlin 和 **Neo4j** 的 **Java API** 一样，最初用于表达如何查询数据库的一种过程（**Procedural**）。它允许更短的语法来表达查询，也允许通过网络远程访问数据库。**Gremlin** 这种过程式的特性，需要用户知道如何采用最好的办法查询结果，这样对于应用程序开发人员来说仍旧有负担。与此同时，在过去30年中，声明式语言 **SQL** 取得了极大的成功：**SQL** 可以将“获取数据的声明方式”和“引擎如何获取数据”相分开，所以 **Neo4j** 的工程师们希望开发一种声明式的图查询语言。

2010 年，**Andrés Taylor** 作为工程师加入 **Neo4j**。受 **SQL** 启发，他启动了一个项目来开发图查询语言，而这种新语言于 2011 年 **Neo4j 1.4** 发布，这种新语言就是如今大多数图查询语言的先祖——**Cypher**。

Cypher 的语法基础，是用 "ASCII艺术(ASCII art)" 来描述图模式。这种方式最初来源于 **Neo4j** 工程师团队在源代码中评注如何描述图模式。可以看看下图的例子：

Image

ASCII art 简单说，就是如何用可打印文本来描述点和边。**Cypher** 文本用 **()** 表示点，**-[]->** 表示边。**(query)-[modeled as]->(drawing)** 来表示 起点 query，终点 drawing，边 modeled as，这样一个最简单的图关系(也可以称为图模式)。

Cypher 第一个版本实现了对图的读取，但是需要用户说明从哪些节点开始查询。只有从这些节点开始，才可以支持图的模式匹配。

在后面的版本，2012 年 10 月发布的 **Neo4j 1.8** 中，**Cypher** 增加了修改图的能力。但查询还是需要指明从哪些节点开始。

2013 年 12 月，**Neo4j 2.0** 引入了 **label** 的概念，**label** 本质上是索引。这样，查询引擎就可以利用索引，来选择模式所匹配到的节点，而不需要用户指定开始查询的节点。

随着 **Neo4j** 的普及，**Cypher** 有着广泛的开发者群体，在各行各业的得到广泛的使用。至今仍是最受欢迎的图查询语言。

2015 年 9 月，**Neo4j** 发起成立了 **openCypher Implementors Group**（**oCIG**），将 **Cypher** 开放为 **openCypher**，通过开源的方式来治理和推进语言自身的演化。

后续

Cypher 启发了一系列后续的图查询语言，包括

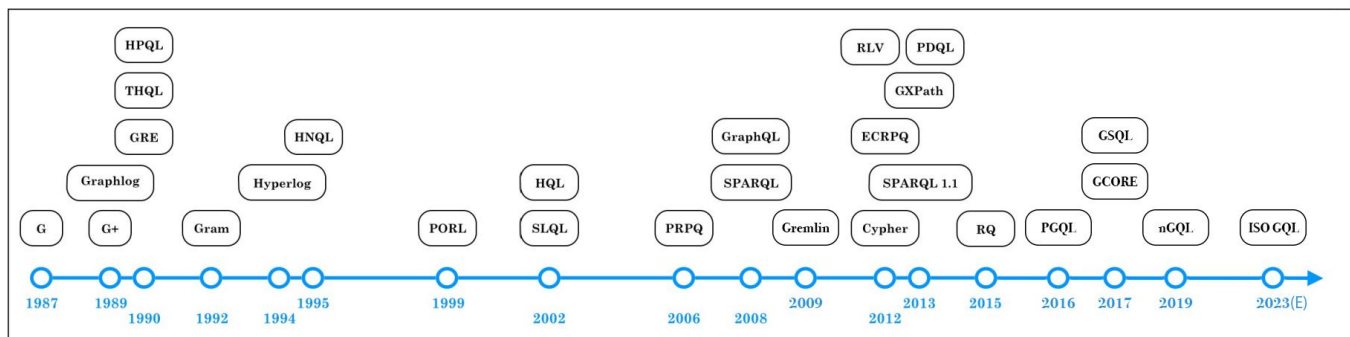
2015 年，**Oracle** 发布图引擎**PGX**使用的图语言**PGQL**。

2016 年，**Linked Data Benchmarking Council**, **LDBC** 是一个行业知名的图性能基准评测机构。**LDBC** 发布 **G-CORE**

2018 年，基于 Redis 的图库(library) RedisGraph 采用 Cypher 作为其图语言

2019 年，国际标准组织 ISO 启动两个项目，基于 openCypher, PGQL, GSQL¹², and G-CORE 等现有业界成果，启动图语言国际标准的制定过程

2019 年，NebulaGraph 以 openCypher 为基础发布其扩展的图语言 NebulaGraph Query Language, nGQL。



分布式图数据库

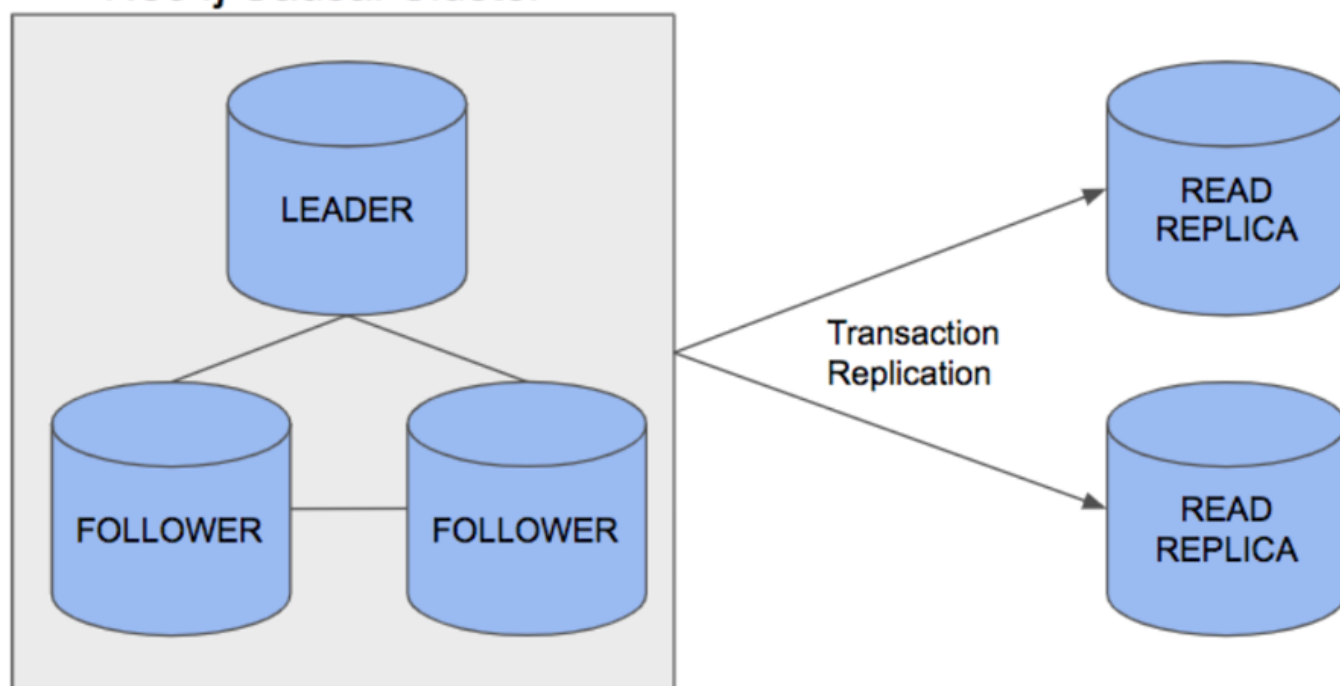
大约 2005-2010 年，随着 Google 云计算"三驾马车"的发布，各种分布式的架构开始越来越流行，其中就包括以开源方式运作的 Hadoop 和 Cassandra 等。这里包括几个方面的影响：

1. 由于数据量和计算量越来越大，相比于单机(例如 Neo4j)或者小型机这种方案，分布式系统的技术和成本优势更加明显；而同时，分布式系统使得应用程序在访问这成千上万台机器时，就如同访问本地的系统一样，不需要在代码层面进行过多改造；
2. 开源方式使得更多的人（包括代码开发者、数据科学家、产品经理等）以更加低成本和有效的方式参与新兴的技术，并反馈给社区。

严格说，Neo4j 也提供了不少的分布式的能力，但都和业界意义上的（对等、分片的）分布式系统有较大的不同：

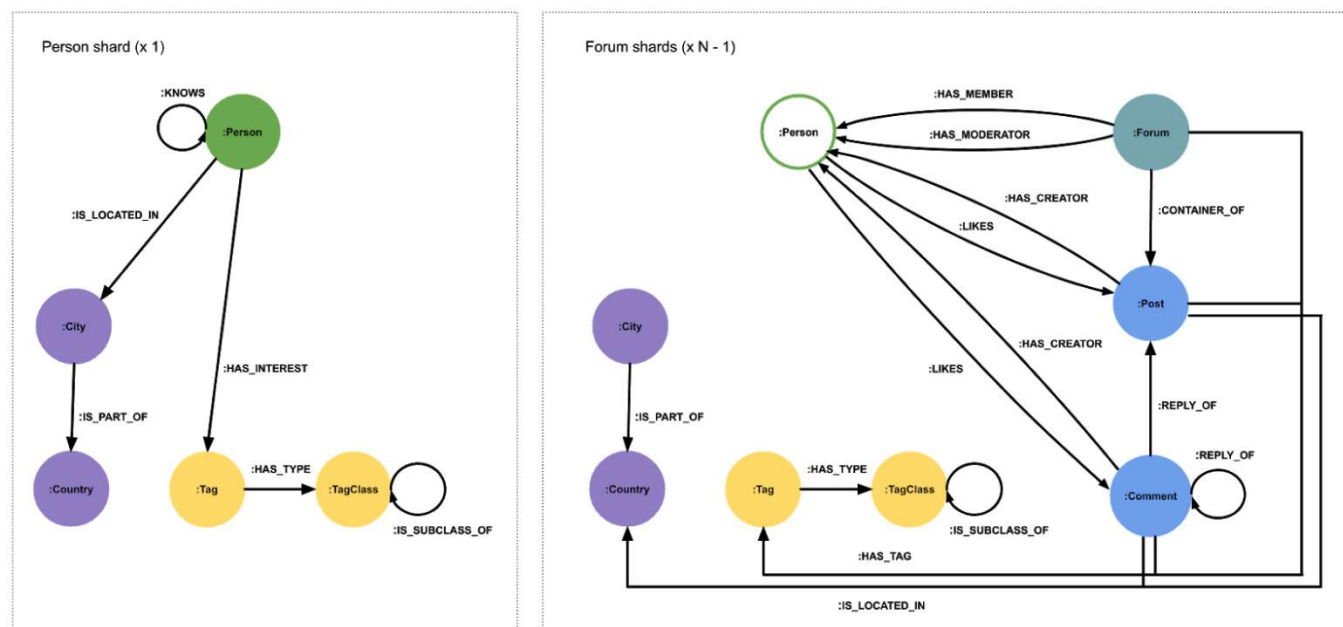
- Neo4j 3.X 要求全量数据必须存放在单机中。虽然其也提供多机之间(Master-slave/slave)做全量复制和高可用，但数据不可切分为不同子图存放。

Neo4j Causal Cluster



Cluster architecture

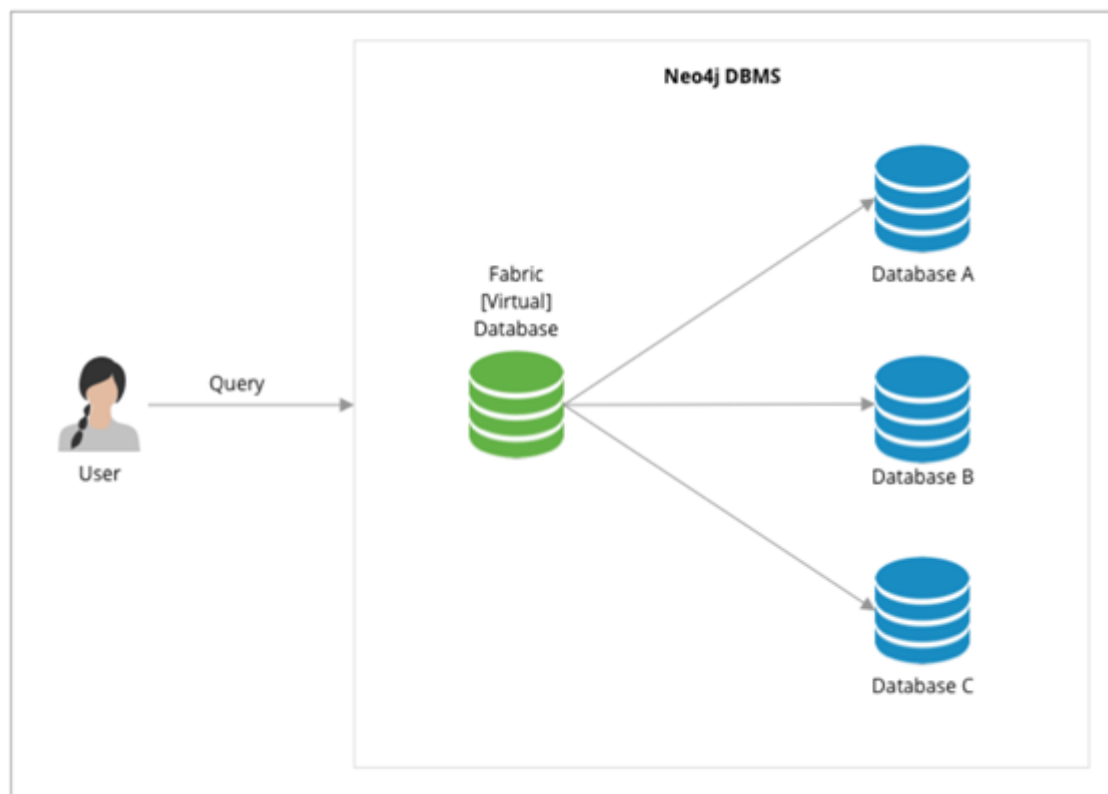
- Neo4j 4.X 允许在不同机器上各存放一部分数据（子图），然后在应用层需通过一定方式拼装后(其称为编织 Fabric)¹³，将读写分发到各个机器上。这种做法需要应用层代码有大量的参与和工作。例如，设计如何把不同子图应该放置在哪些机器上，如何将各机器获取的部分结果重新编织为最终的结果。



其语法风格大体是

```
USE graphA # S1.1 从 Shard A 读
MATCH (movie:Movie)
Return movie.title AS title
UNION # S2. 在代理服务器 Join 结果
USE graphB # S1.2 从 Shard B 读
```

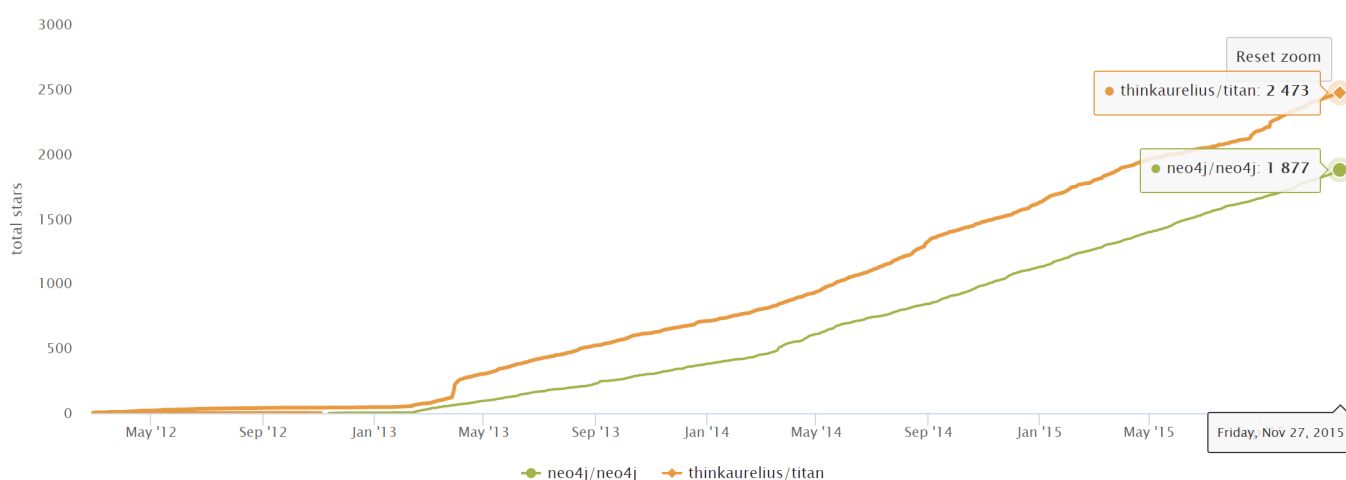
```
MATCH (move:Movie)
RETURN movie.title AS title
```



第二代（分布式）图数据库：TITAN 和其后继者 JANUSGRAPH

2011 年，Aurelius 公司成立，致力于开发一个开源的分布式图数据库 Titan¹⁴。到 2015 年 Titan 的第一个正式版发布，Titan 后端可以支持多种主流的分布式存储架构（例如 Cassandra, HBase, Elasticsearch, BerkeleyDB），并可以复用 Hadoop 生态的诸多便利，前端以 Gremlin 为统一的查询语言。对于程序员使用、开发和社区参与都很方便。大规模的图，可以分片后存放在 HBase 或者 Cassarndar 上(这些当时都已经是相对成熟的分布式存储方案)，Gremlin 语言虽然略微冗长但相对功能完备。整个方案在当时(2011-2015)体现了不错的竞争力。

下图显示了 2012 年 - 2015 年，Titan 和 Neo4j 在 GitHub.com 上 star 的增长情况。

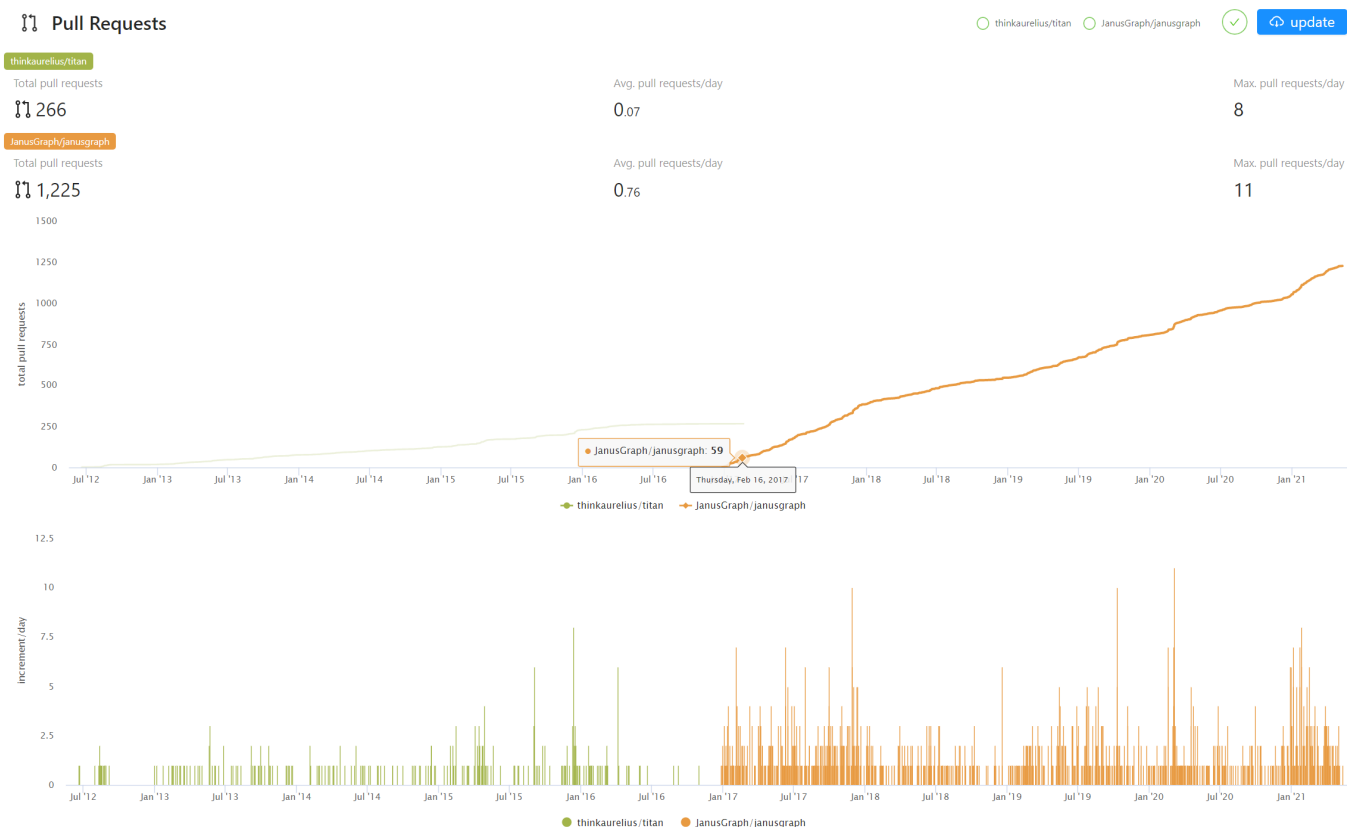


2015 年 Aurelius(Titan) 被 DataStax 收购，这之后 Titan 逐渐转变为一个闭源的商业产品 (DataStax Enterprise Graph)。

在 Aurelius(Titan) 被收购后，市场对于开源分布式的图数据库一直仍有比较强烈的需求，而当时市场上成熟和活跃的产品并不多。大数据时代，数据仍在远快于摩尔定律的速度，源源不断的产生。Linux 基金会以及一些技术巨头(Expero, Google, GRAKN.AI, Hortonworks, IBM and Amazon) 在2017年，复制并分叉(fork)了原有的Titan项目，并启动为一个新项目 JanusGraph¹⁵。之后大多数的社区工作，包括开发、测试、发布和推广都逐步转移到了新的 JanusGraph。

下图显示了两个项目2012-2021年日常代码提交(pull request)的变化情况，可以观察到几点：

1. 即使 Aurelius(Titan) 2015 年被收购后，其开源代码仍有一定的活跃度(左侧)，但增速已经明显放缓。这体现了社区的力量。
2. 新项目 JanusGraph 项目在 2017 年 1 月启动后，其社区迅速活跃起来，短短一年时间就超越了 Titan 过去 5 年累计的 pull request 数量。而与此同时，Titan 开源项目就此停滞。



同期知名产品 **ORIENTDB**, **TIGERGRAPH**, **ARANGODB**, 和 **DGRAPH**

此后更多的厂商加入整个市场，除了由Linux基金会托管的 **JanusGraph**，还有一些由商业公司主导开发的分布式图数据，各方采用的数据模型和访问方式也有明显的不同。本文不做一一介绍，仅简单列出主要区别。

厂商名	创立时间	核心产品名	开源协议	数据模型	查询语言
OrientDB LTD (2017 年 被 SAP 收购)	2011	OrientDB	开源	文档 + KV + 图	OrientDB SQL (基于SQL扩展的图能力)
GraphSQL (后改名 TigerGraph)	2012	TigerGraph	商业版本	图(分析)	GraphSQL (类 SQL风格)
ArangoDB GmbH	2014	ArangoDB	Apache License 2.0	文档 + KV + 图	AQL (同时操作文档, KV 和图)
DGraph Labs	2016	DGraph	Apache Public License 2.0 + Dgraph Community License	原 RDF, 后改为 GraphQL	GraphQL+-

传统巨头微软、亚马逊和甲骨文纷纷入场

除了聚焦于图产品的厂商外，传统巨头也纷纷进入这个领域。

Microsoft Azure Cosmos DB¹⁶ 是一个在微软云上的多模数据库云服务，可以提供SQL、文档、图、key-value等多种能力；**Amazon AWS Neptune¹⁷** 是一种由 AWS 提供图数据库云服务，可以提供属性图和 RDF 两种数据模型；**Oracle graph¹⁸** 是关系型数据库巨头 Oracle 在图技术与图数据库方向的产品。

分布式图数据库**NEBULAGRAPH**

在下一章，我们将正式介绍新一代分布式图数据库**NebulaGraph**。

-
1. https://db-engines.com/en/ranking_categories ↩
 2. <https://www.yellowfinbi.com/blog/2014/06/yfcommunitynews-big-data-analytics-the-need-for-pragmatism-tangible-benefits-and-real-world-case-165305> ↩
 3. <https://www.gartner.com/smarterwithgartner/gartner-top-10-data-and-analytics-trends-for-2021/> ↩
 4. <https://www.verifiedmarketresearch.com/product/graph-database-market/> ↩
 5. <https://www.globenewswire.com/news-release/2021/01/28/2165742/0/en/Global-Graph-Database-Market-Size-Share-to-Exceed-USD-4-500-Million-By-2026-Facts-Factors.html> ↩
 6. <https://www.marketsandmarkets.com/Market-Reports/graph-database-market-126230231.html> ↩
 7. <https://www.gartner.com/en/newsroom/press-releases/2019-07-01-gartner-says-the-future-of-the-database-market-is-the> ↩
 8. <https://www.amazon.com/Designing-Data-Intensive-Applications-Reliable-Maintainable/dp/1449373321> ↩
 9. I. F. Cruz, A. O. Mendelzon, and P. T. Wood. A Graphical Query Language Supporting Recursion. In Proceedings of the Association for Computing Machinery Special Interest Group on Management of Data, pages 323-330. ACM Press, May 1987. ↩↩
 10. "An overview of the recent history of Graph Query Languages". Authors: Tobias Lindaaker, U.S. National Expert. Date: 2018-05-14 ↩
 11. Gremlin是基于Apache TinkerPop开发的图语言(<https://tinkerpop.apache.org/>)。 ↩
 12. <https://docs.tigergraph.com/dev/gsql-ref> ↩
 13. <https://neo4j.com/fosdem20/> ↩
 14. <https://github.com/thinkaurelius/titan> ↩
 15. <https://github.com/JanusGraph/janusgraph> ↩
 16. <https://azure.microsoft.com/en-us/free/cosmos-db/> ↩
 17. <https://aws.amazon.com/cn/neptune/> ↩
 18. <https://www.oracle.com/database/graph/> ↩
-

最后更新: 2026年8月18日

2.3 相关技术

本节主要介绍两个和分布式图数据库关系密切的领域，数据库方面和图技术方面。

2.3.1 数据库方面

关系型数据库

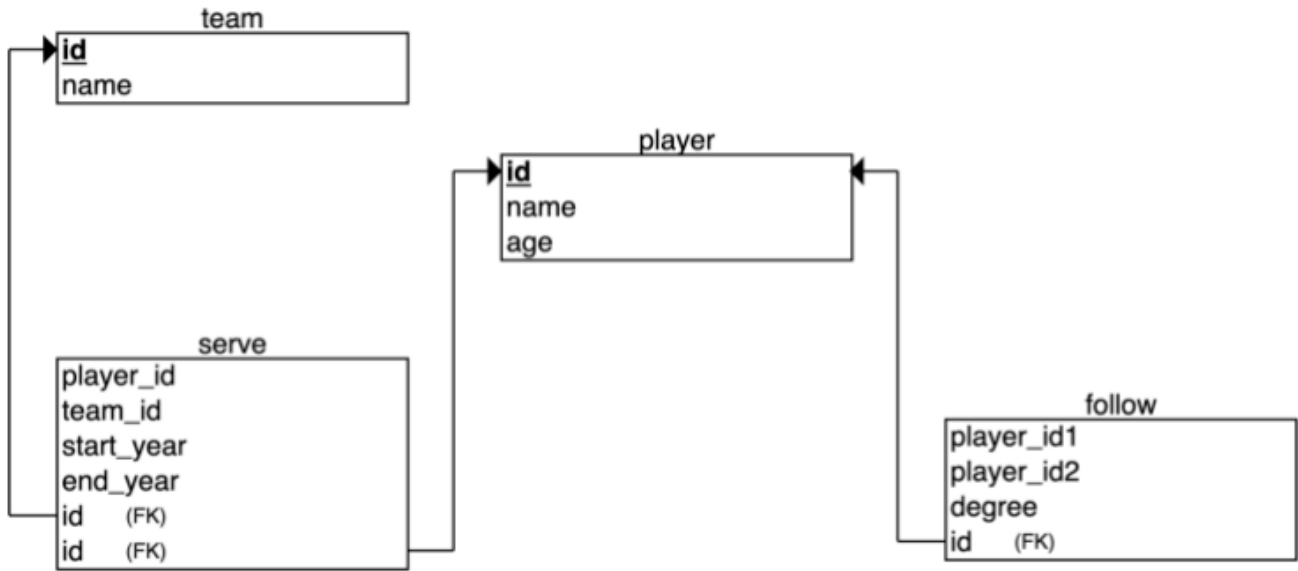
关系型数据库，是指采用了关系模型来组织数据的数据库。关系模型为二维表格模型，一个关系型数据库就是由二维表及其之间的联系所组成的一个数据组织。说到关系型数据库，大多数人都会想到 MySQL。MySQL 是目前最流行的数据库管理系统之一，支持使用最常见的结构化查询语言（SQL）进行数据库操作，并以表格、行、列的形式存储数据。这种存储数据的方法源自埃德加·科德（Edgar Frank Codd）于 1970 年提出的关系型数据模型。

在关系型数据库中，可以为待存储的每种类型的数据创建一个表。例如，球员表用来存储所有的球员信息，球队表用来存储球队信息等。SQL 表中的每行数据都必须包含一个主键（primary key）。主键是该行数据的唯一标识符。一般地，主键作为字段 ID 都是随行数自增的。关系型数据库自问世以来一直为计算机行业提供着非常好的服务，并将未来很长的时间内继续服务下去。

如果你用过 Excel、WPS 或其他类似的应用，你就会大概了解到关系数据库是如何工作的。首先设置好列，然后在对应的列下添加行数据。你可以对某一列数据进行求平均值或其他聚合操作，这与在关系型数据库 MySQL 中求平均值的操作类似。而 Excel 中的数据透视表则相当于在关系型数据库 MySQL 中使用聚合函数和 CASE 语句对数据进行查询。一个 Excel 文件可以有多张表，一张表就相当于 MySQL 的一张表。一个 Excel 文件则类似于一个 MySQL 数据库。

关系型数据库中的关系

与图数据库不同，关系型数据库（或 SQL 型的数据库）中的边也是作为实体存储在专门的边表中的。先创建两个表，球员（player）和球队（team），然后再创建表 player_team 作为边表。边表通常由相关的表 join 而成。例如，此处的边表 player_team 就由球员表和球队表 join 而成。



这种存储边的方式在关联小型数据集时问题并不大，但是当关系型数据库中的关系太多时，问题就出现了。事实上，关系型数据库是非常“反关系的”。具体来说，当你只想查询一个球员的队友时，你必须对表中的所有数据进行 join 操作，然后再过滤掉你不需要的所有数据，当你的数据集达到一定规模时，这将给关系型数据库带来巨大压力。如果你想关联多张不同的表，可能在 join 爆炸（join bombs）前系统就已经无法响应了。

关系型数据库起源

上文提到，关系型数据模型最早是由 IBM 的工程师埃德加·科德（Edgar Frank Codd）于 1970 年提出的。科德写了几篇数据库管理系统方面的论文，论述了关系型数据模型的潜力。关系型数据模型不依赖于数据链接列表（网状数据或层级数据），而是更多依赖于数据集。他使用元组演算

(tuple calculus) 的数学方法论证了这些数据集能够完成与导航数据库管理系统相同的任务。唯一的要求是，关系型数据模型需要一种合适的查询语言，以保证数据库的一致性要求。这就为后来声明型的结构化查询语言 (SQL) 提供了灵感来源。IBM 的 R 系统是关系型数据模型的最早使用者之一。然而，由前 IBM 员工拉里·埃里森创办的小公司在市场上击败了 IBM。该公司的产品就是后来为我们熟知的 Oracle。

由于“关系数据库”在当时是一个比较时髦的词汇，因此许多数据库供应商都喜欢在其产品名称中使用这个词，尽管他们的产品实际上并不是关系型的。为了防止这种情况并减少关系型数据模型的错误使用，科德提出了著名科德 12 定律 (Codd's 12 rules)。所有关系型数据系统都必须遵循科德 12 定律。

NoSQL 数据库

图数据库并不是可以克服关系型数据库缺点的唯一替代方案。现在市面上还有很多非关系型数据库的产品，这些产品都可以叫做 NoSQL。NoSQL 一词最早于上世纪 90 年代末提出，可以解释为“非 SQL”或“不仅是 SQL”，具体解释要根据语境判断。为便于理解，这里 NoSQL 可以解释成“非关系型数据库”。不同于关系型数据库，NoSQL 数据库提供的数据存储、检索机制并不是基于表关系建模的。NoSQL 数据库可以分为四类：

- 键值存储 (key-value stores)
- 列式存储 (column-family stores)
- 文件存储 (document stores)
- 图数据库 (graph databases)

下面将分别介绍这四类数据库。

键值存储

键值存储，顾名思义，就是使用键值对存储数据的数据库。不同于关系型数据库，键值存储是没有表和列的。如果一定要做类比，键值数据库本身就像一张有很多列（也就是键）的大表。在键值存储数据库中，数据（即键值对中的值）都是通过键来存储和查询的，通常用哈希列表来实现。这比传统的 SQL 数据库要简单得多，而且对于某些 web 应用来说，这就足够了。

键值模型对于 IT 系统来说优势在于简单、易部署。多数情况下，这种存储方式对非关联的数据很适用。如是只是存储数据而无需查询的话，使用这种存储方法就没有问题。但是如果 DBA 只对部分值进行查询或更新的时候，键值模型就显得效率低下了。常见的键值存储数据库有：Redis、Voldemort、Oracle BDB。

列式存储

NoSQL 数据库的列式存储与 NoSQL 数据库的键值存储有许多相似之处，因为列式存储仍然在使用键进行存储和检索。区别在于列式存储数据库中，列是最小的存储单元，每一列均由键、值以及用于版本控制和冲突解决的时间戳组成。这在分布式扩展时特别有用，因为在数据库更新时，可以使用时间戳定位过期数据。由于列式存储良好的扩展性，因此适用于非常大的数据集。常见的列式存储数据库有：HBase、Cassandra、HadoopDB 等。

文档存储

准确来说，NoSQL 数据库文档存储实际上也是基于键值的数据库，只不过对功能做了增强。数据仍然以键值的形式存储，但是文档存储中的值是结构化的文档，而不仅仅是一个字符串或单个值。也就是说，由于信息结构的增加，文档存储能够执行更优化的查询，并且使数据检索更加容易。因此，文档存储特别适合存储、索引并管理面向文档的数据或者类似的半结构化数据。

从技术上讲，作为一个半结构化的信息单元，文档存储中的文档可以是任何形式可用的文档，包括 XML、JSON、YAML 等，这取决于数据库供应商的设计。比如，JSON 就是一种常见的选择。虽然 JSON 不是结构化数据的最佳选择，但是 JSON 型的数据在前端和后端应用中都可以使用。常见的文档存储数据库有：MongoDB、CouchDB、Terrastore 等。

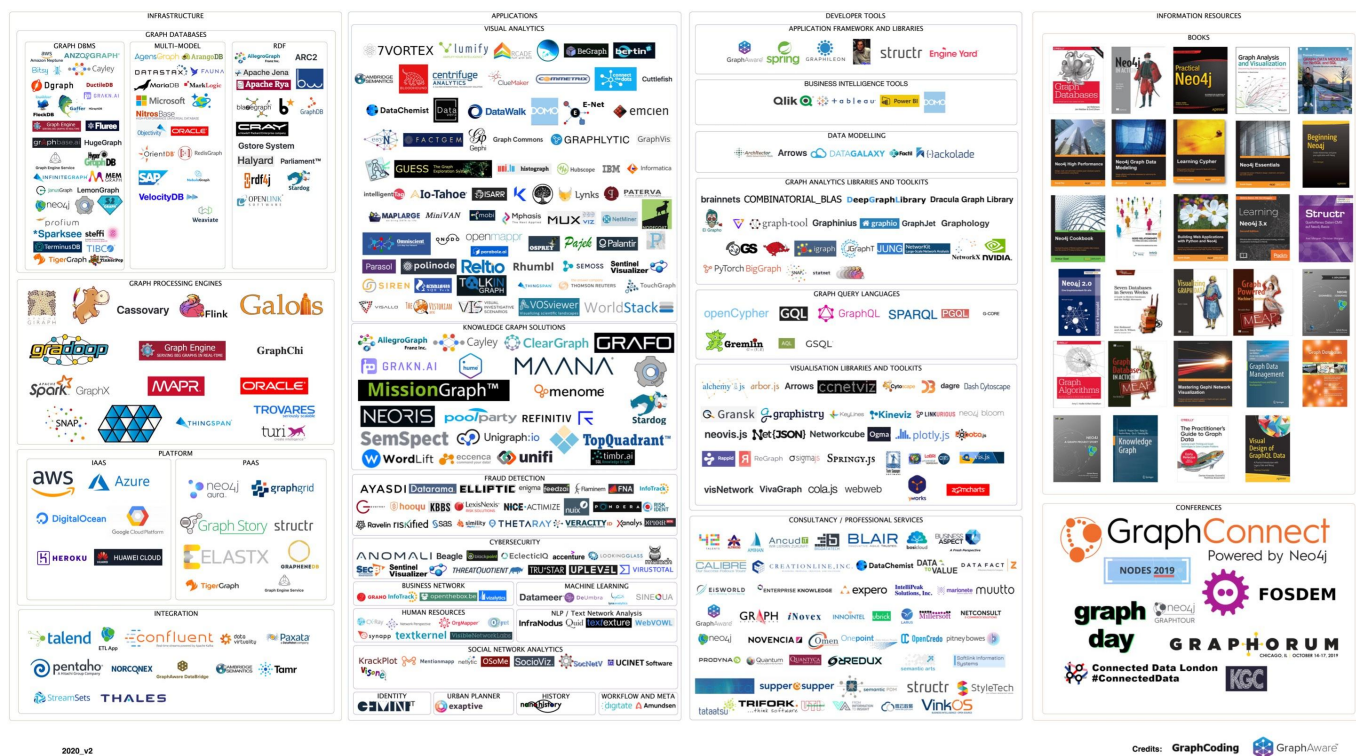
图存储

最后一类 NoSQL 数据库是图数据库。本书重点讨论的NebulaGraph也是一种图数据库。虽然同为 NoSQL 型数据库，但是图数据库与上述 NoSQL 数据库有本质上的差异。图数据库以点、边、属性的形式存储数据。其优点在于灵活性高，支持复杂的图形算法，可用于构建复杂的关系图谱。我们将在随后的章节中详细讨论图数据库。不过在本章中，你只要知道图数据库是一种 NoSQL 类型的数据库就可以了。常见的图数据库有：NebulaGraph、Neo4j、OrientDB 等。

2.3.2 图技术方面

来看一张 2020 年的图技术全景¹

GRAPH TECHNOLOGY LANDSCAPE 2020



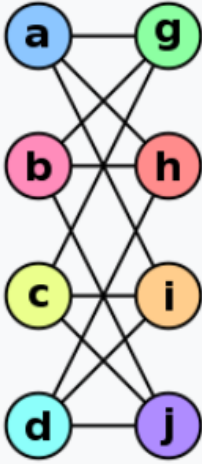
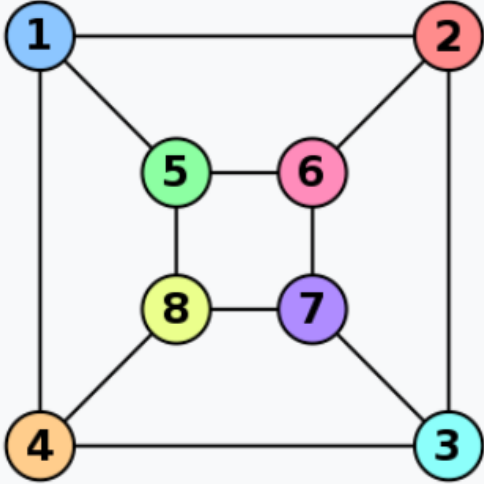
和图有关的技术有很多，可以大致分为这么几类：

- 基础设施：包括图数据库、图计算(处理)引擎、图深度学习、云服务等。
- 应用：包括可视化、知识图谱、反诈骗、网络安全、社交网络等。
- 开发工具：包括图查询语言、建模工具、开发框架和库。
- 电子书籍和会议等。

图语言

在上一节中，我们介绍了图语言的历史。在这一节中，我们对图语言的功能做一个分类：

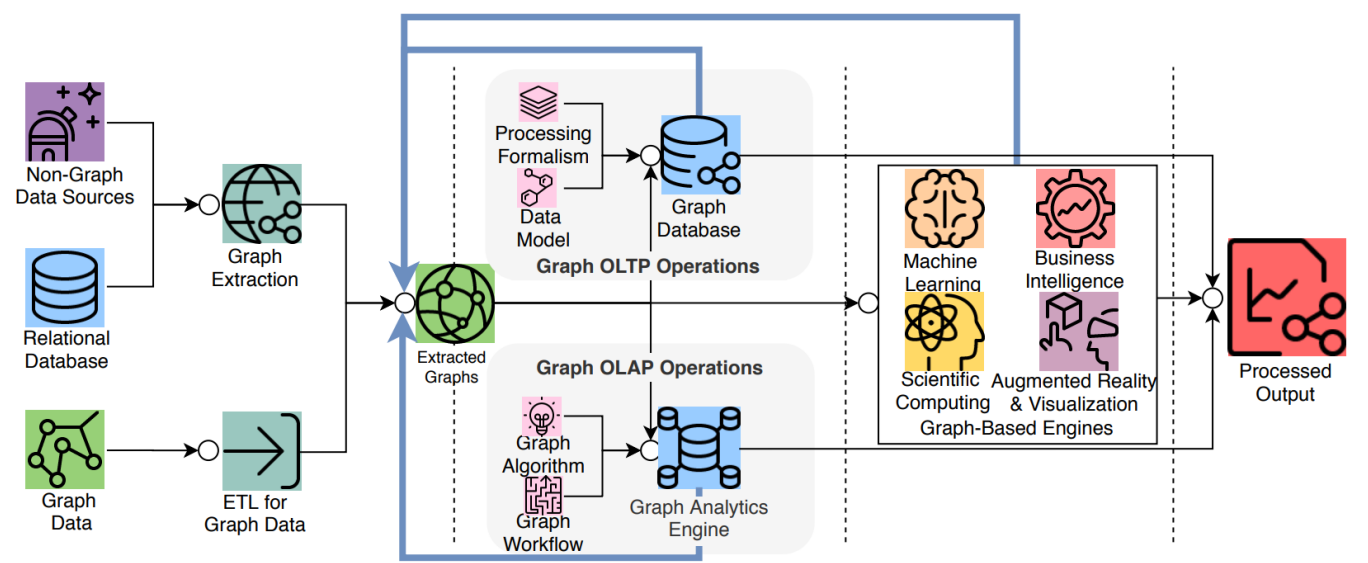
- 近邻查询：查询给定点或者边的邻边、邻点，或者是 K 跳的近邻。
- 图模式匹配(Pattern matching)：找到一个/所有的子图，满足给定的图模式；这个问题非常接近于"子图同构映射(subgraph isomorphism)"——虽然两个看上去不同的图，但其实是一模一样的²。

Graph G	Graph H	An isomorphism between G and H
		$\begin{aligned} f(a) &= 1 \\ f(b) &= 6 \\ f(c) &= 8 \\ f(d) &= 3 \\ f(g) &= 5 \\ f(h) &= 2 \\ f(i) &= 4 \\ f(j) &= 7 \end{aligned}$

- 可达性（连通性）问题：最常见的可达性问题就是最短路径问题。泛化一些，这类问题通常用正则路径(Regular Path Query)的方式来描述——一系列联通的点组构成了一条路径，而该路径需要满足某种正则表达。
- 分析型问题：通常与一些汇聚型算子相关（平均值、count、最大值、点的出入度），或者度量所有两两点之间的距离、某节点与其他节点之间的互动程度（介数中心性）等。

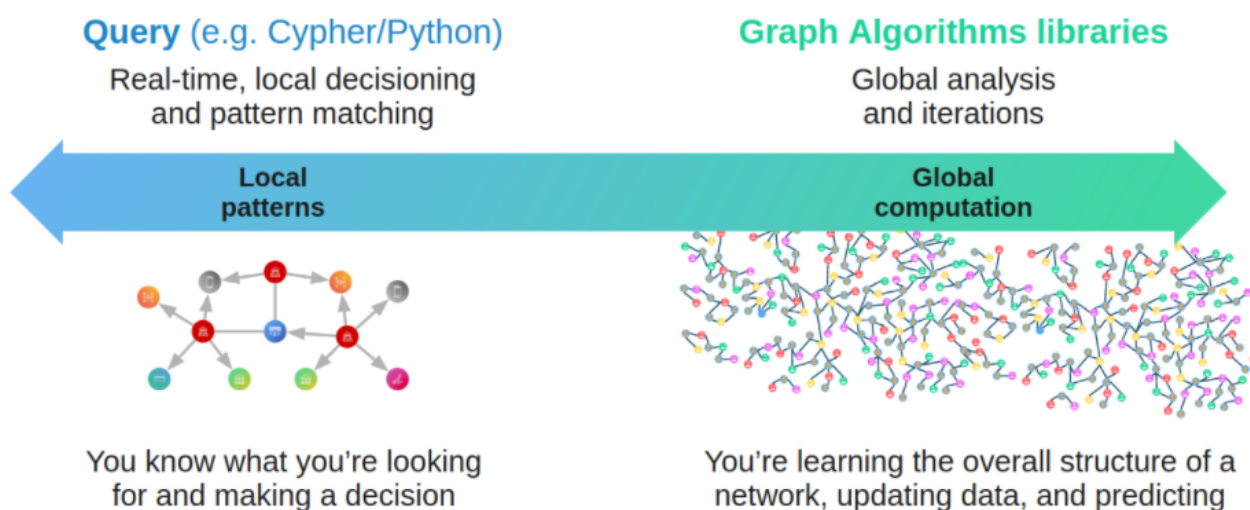
图数据库(Graph Database)与图处理(Graph processing)系统

一个图系统通常会涉及到复杂的数据流水线³，从数据源(左边)到处理输出(右边)，会经过多个数据加工处理环节和系统；大的模块可以分为 ETL模块，图数据库系统(Graph OLTP)，图处理系统(Graph OLAP)，基于图引擎的应用系统（BI、知识图谱等）。



虽然这两类系统都是与图数据和图技术相关的系统，也处理类似的目标，但是他们有着不同的起源和特长（及弱点）：

- （在线）图数据库目标是图的持久化存储管理、高效的子图操作。硬盘（及网络）是目标运行设备，物理/逻辑数据映射，数据完整性和（故障）一致性是主要目标。每一个请求通常只会涉及到全图的一小部分，通常可以在一台服务器上完成；单个请求时延通常在毫秒到秒级别，请求并发量通常在几千到几十万。早期的 Neo4j 是图数据库领域的起源之一。
- （离线）图处理系统目标是全图的大批量、并行、迭代、处理与分析，内存（及网络）是目标运行设备。每一个请求会涉及到所有的图节点，需要所有的服务器参与完成；单个请求的时延通常在分钟到小时（天），请求并发量通常为个位数。Google 的 Pregel⁴ 是图处理系统的典型起源代表，它的点中心编程抽象与BSP的运行模式构成的编程范式，相比之前 Hadoop Map-Reduce 是更为图友好的 API 抽象。



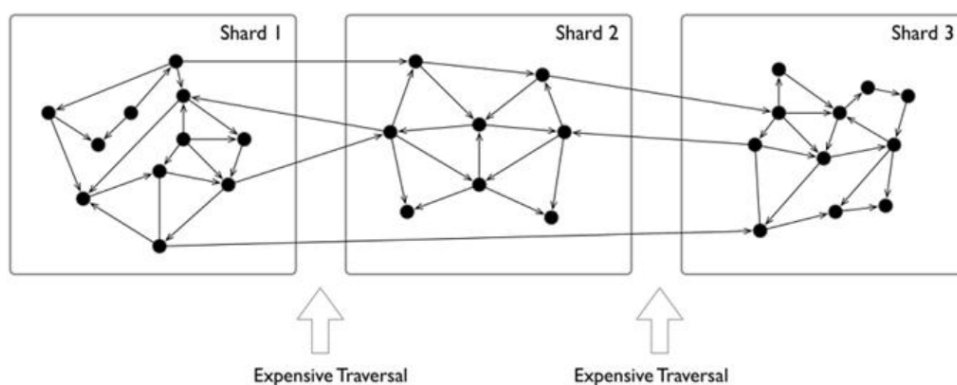
5

图的分片方式

对于一个大规模的图数据来说，是很难存放在单个服务器的内存中的，即使仅仅存放图结构本身也不够。而且通过增加单服务器的能力，其成本价格通常成指数级别上升。此外，随着数据量的增加，例如到达千亿级别的时候，已经超过了市面上所有商用服务器的容量能力。

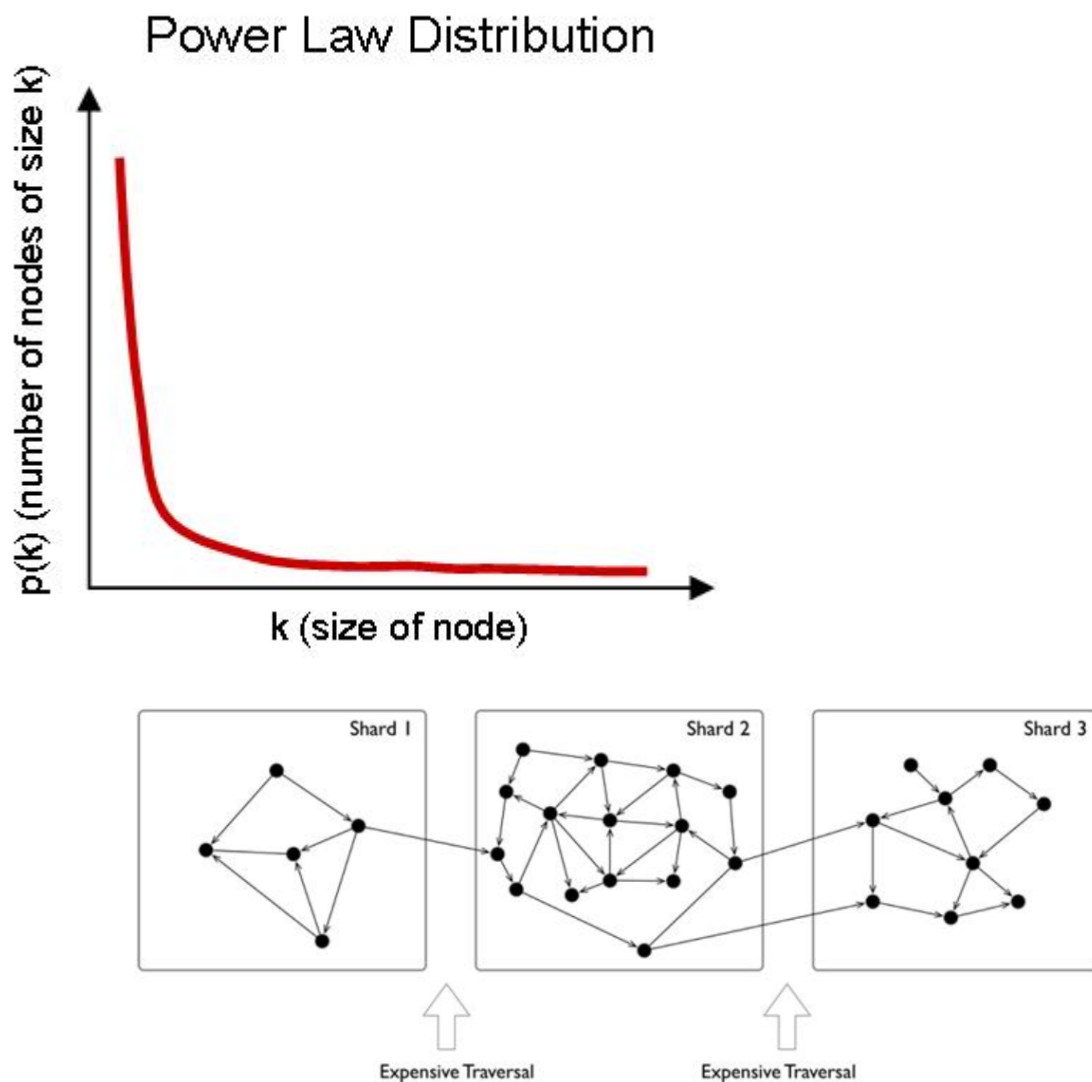
与此对应的，另外一个经常使用的方案，是对数据进行分片，并将每个分片放置在不同的服务器上（并进行冗余备份），以此来增加可靠性和性能。对于一些 NoSQL 型的系统，例如 key-value 或者文档型的系统来说，这个分片方式是比较直观和自然的；通常可以根据 key 或者 docID，来将每个记录或者数据单元(key-value, doc)放在不同的服务器上。

但是图这种数据结构的分片通常不那么直观，这是因为通常图是“全联通”的，每个点通常只要6跳就可以联通到其他任何节点；而理论上早已证明图的划分问题是 NP 的。与此同时，当把整个图数据分散到多个服务器时，跨服务器的网络访问时延10倍于同一个服务器内部的硬件(内存)访问时间；因此对于一些深度优先遍历的场景，会发生大量的跨网络访问，导致整体时延极高。



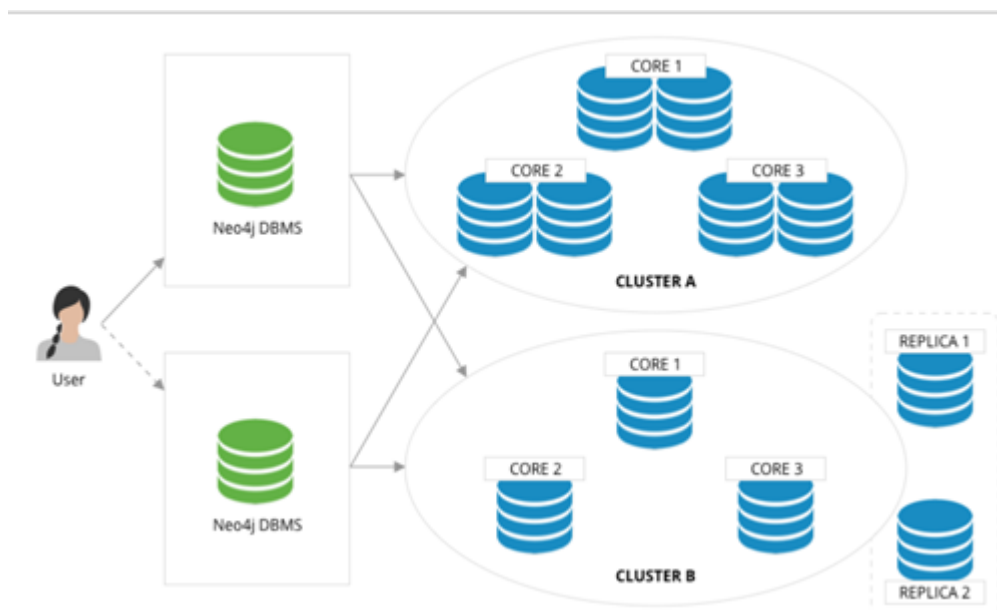
6

另一方面，通常图有着明显的幂律分布；少量节点的邻边稠密程度远大于平均的节点，虽然处理这些节点通常可以在同一台服务器内，减少了跨网络访问，但这也意味着这些服务器压力会远大于平均。

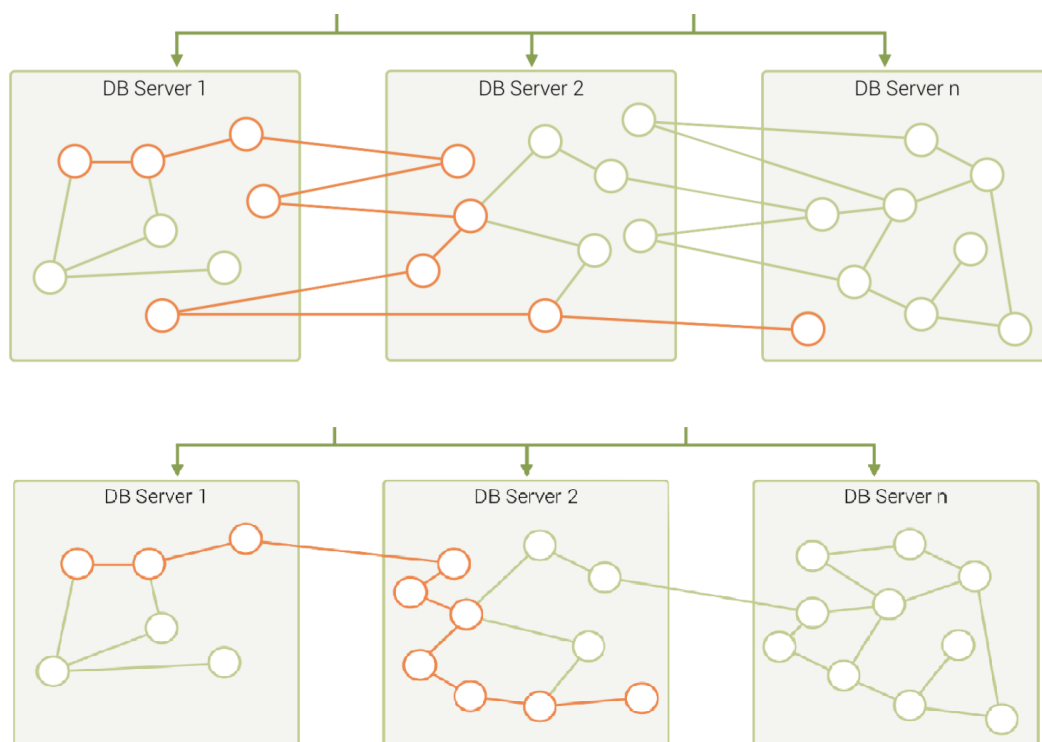


因此，常见的图分片(Sharding)方式有几类：

- 偏应用层面的分片：应用层感知并控制每个点和边应该落在哪个分片上，一般来说可以根据点和边的类型（比如业务意义来人为）判断。将一组相同类型的点放在一个分片，另一组相同类型的点放在另一个分片。当然，为了高可靠，分片本身还可以做多副本。在应用使用时，从各个分片取回所要的点和边，然后在偏应用侧（或者某个代理服务器端），将取回的数据拼装成最终的结果。其典型代表是 Neo4j 4.x 的 Fabric。



- 使用分布式的缓存层：在硬盘之上增加内存缓存层，并对重要的部分分片和数据（例如图结构）进行缓存，并预热这部分缓存。
- 增加（只读）副本或视图 (View)：为部分图分片增加只读的副本或者建立一个视图，将较重的读请求负载通过这些分片服务器承担。
- 进行细颗粒度的图划分：例如将点和边组成多个小分片 (Partition)，而不是一台服务器一个大分片 (Sharding)，再将关联性较强的 Partition 尽量放置在同一个服务器上⁷。



具体工程实践时，也会混合使用上述几种方式。通常，离线的图处理系统会通过一个ETL过程，将图进行一定程度的预处理以提高局部性；而在线图数据库系统通常会选择周期性的数据再平衡过程来提高数据局部性。

一些技术上的挑战

在文献⁸中，对于无处不在的大图和挑战做了详尽的调研，下面是其列出的十大图技术挑战：

- 可扩展性(软件可以处理更大的图规模)：包括大图的加载、更新、图计算和图遍历，触发器，超级节点；
- 可视化：可定制布局，大图的渲染，多层次展示，动态（更新）的展示
- 查询语言和编程 API：包括语言表达能力、标准兼容性、与现有系统的兼容性；子查询的设计和跨多图之间的关联查询
- 更快的图（及机器学习）算法
- 易用性（配置和使用）
- 性能指标与测试
- 更通用的图技术软件（例如，处理离线、在线、流式的计算）
- 图清洗（ETL）
- Debug 调试与测试

一些开源的单机图工具

对于图数据库通常会有一个误解，只要涉及到图结构的数据存取就需要存放在图数据库中，这是一种很大的浪费。

这就像也许你只需要一个 SQLite，却用了一个 Oracle。

当数据量并不大时，通常单机内存可以放下，例如数据量几千万的点边关系，使用一些单机的开源工具也可以取得很好的效果。



Note

下面是一些推荐的单机图库，也可以集成在你的应用程序里面。

- JGraphT⁹：一个知名的开源 Java 图论库(library)，其实现了相当多的高效图算法。
- JUNG¹⁰是BSD许可下用Java编写的开源图建模和可视化框架。该框架内置了许多布局算法，以及诸如图聚类和节点中心性度量之类的分析算法。
- igraph¹¹：一个轻量且功能强大的 Library, 支持R、python、C
- NetworkX¹²：数据科学家做图论分析第一选择，python。
- Cytoscape¹³：功能强大的可视化开源图分析工具。
- Gephi¹⁴：功能强大的可视化开源图分析工具。
- arrows.app¹⁵：非常简单的脑图工具，用于可视化生成 Cypher 语句。

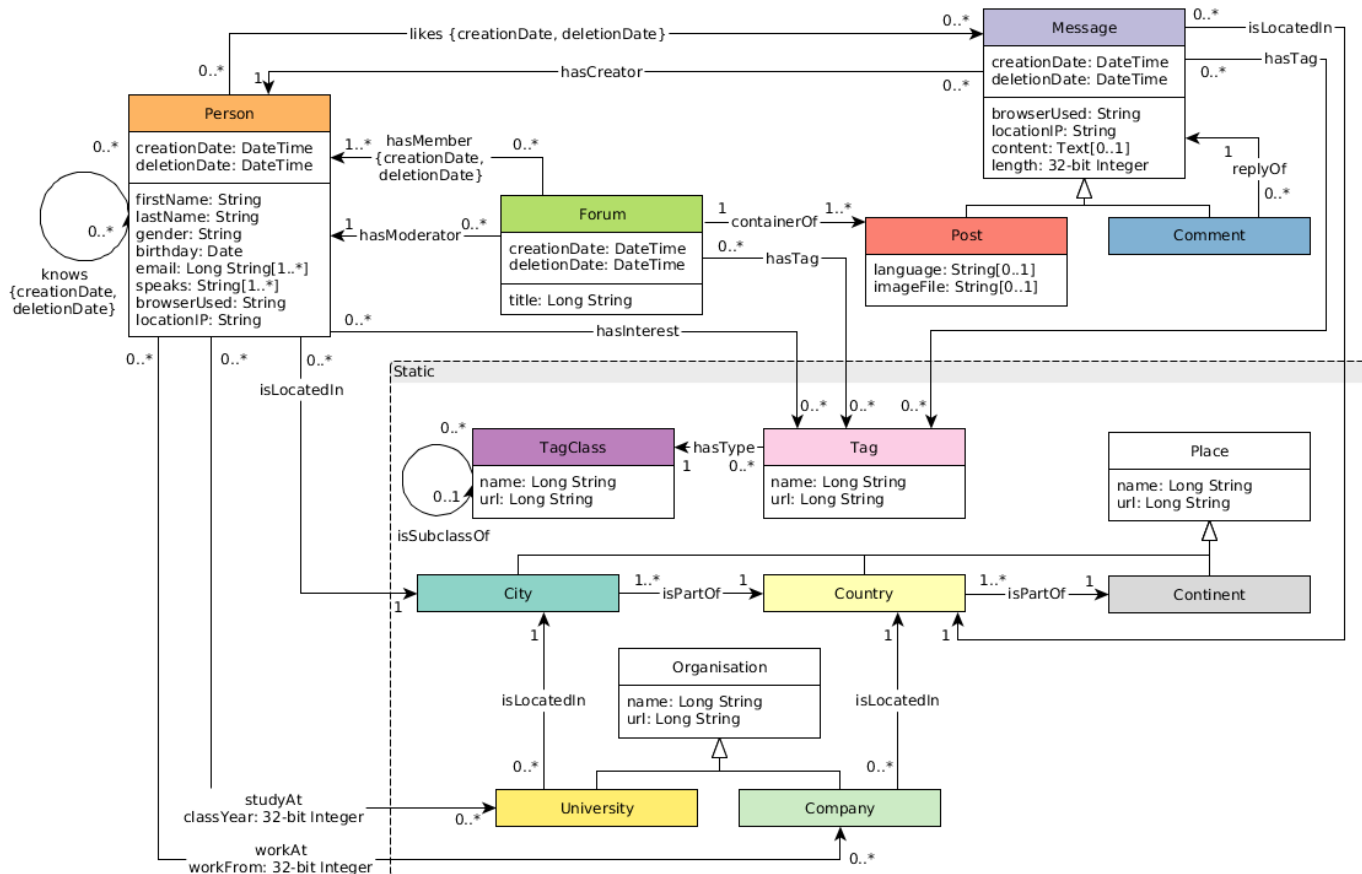
一些行业数据集和 Benchmark

LDBC

关联数据基准委员会（LDBC¹⁶, Linked Data Benchmark Council）是由Oracle、Intel等软硬件巨头和主流图数据库厂商Neo4j和TigerGraph等组成的非赢利机构，是图的基准指南制定者与测试结果发布机构，在行业内有着很高的影响力。

社交网络基准测试（SNB, Social Network Benchmark）是由关联数据基准委员会（LDBC）开发的面向图数据库的基准测试（Benchmark）之一，分为交互式查询（Interactive）和商业智能（BI）两个场景。其作用类似于 TPC-C, TPC-H 等测试在 SQL 型数据库中的功能，可以帮助用户比较多种图数据库产品的功能、性能、容量。

SNB 数据集模拟一个社交网络的人、发帖之间的关系，考虑了社交网络的分布属性、人的活跃度等等社交信息。



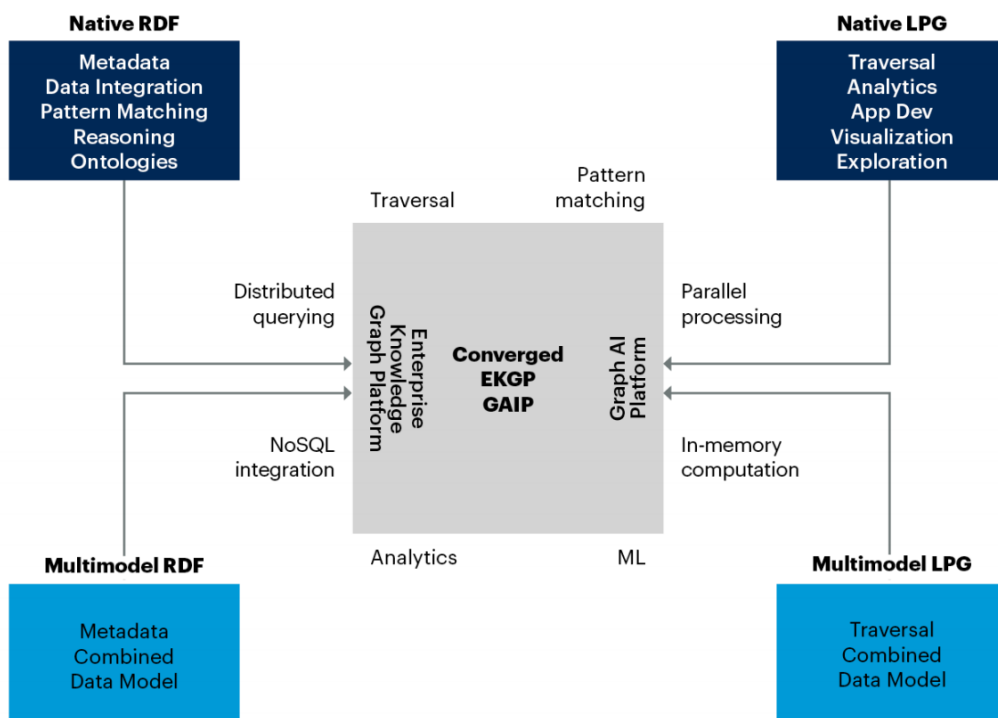
其标准数据量从 0.1 GB (scale factor 0.1) 到 1000 GB (sf1000)，也可以生成 10 TB，100 TB 等更大的数据集；其点、边数量如下表。

Scale Factor	0.1	0.3	1	3	10	30	100	300	1000
# of Persons	1.5K	3.5K	11K	27K	73K	182K	499K	1.25M	3.6M
# of nodes	327.6K	908K	3.2M	9.3M	30M	88.8M	282.6M	817.3M	2.7B
# of edges	1.5M	4.6M	17.3M	52.7M	176.6M	540.9M	1.8B	5.3B	17B

2.3.3 一些趋势

虽然图的各种技术起源和目标并不相同，但在相互借鉴和融合

Convergence of Capabilities in the Graph DBMS Landscape



Source: Gartner
737853_C

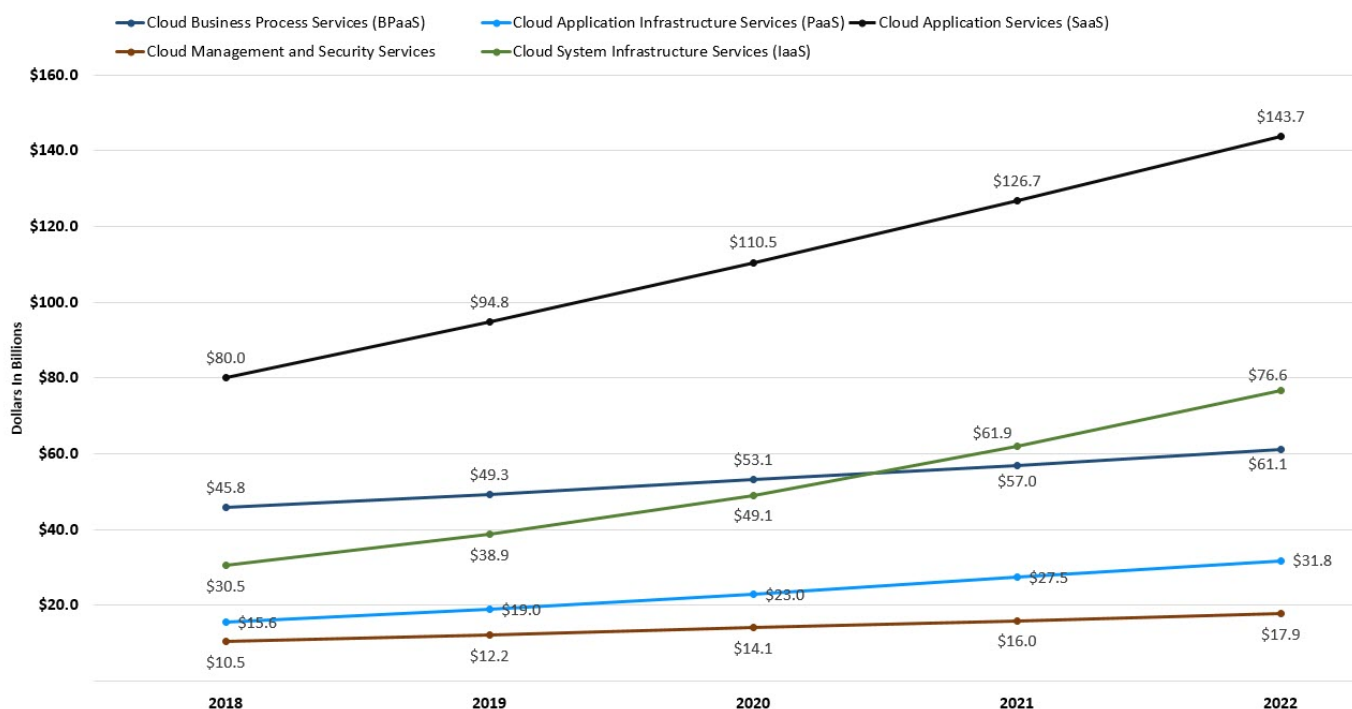
Gartner

上云的趋势在加速，对于弹性能力提出更高要求

根据 Gartner 的预计，云服务一直保持较快的增速和渗透率¹⁷。大量的商业软件，正在从 10 年前完全私有本地逐步转向基于云服务的商业模式。云服务的一大优点是其提供了近乎无限的弹性能力；这也要求各种基于云基础设施的软件必须有更好的快速弹性扩缩容能力。

Worldwide Public Cloud Service Revenue Forecast, 2018 - 2022

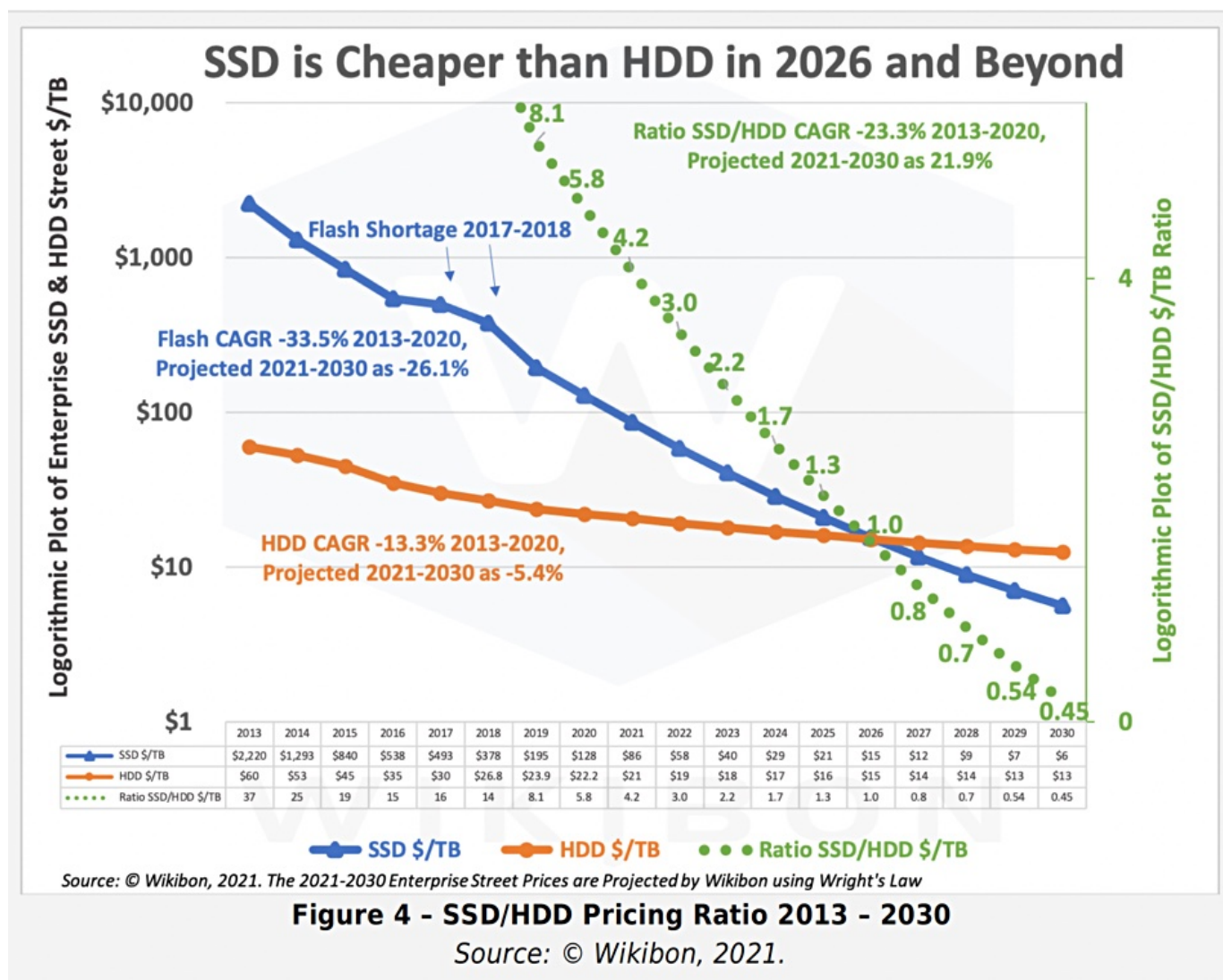
(Billions of U.S. Dollars) Source: Gartner April 2, 2019



硬件趋势，SSD 将成为主流的持久化设备

硬件决定了软件的架构——从发现摩尔定律的 50 年代到进入多核的 00 年代，硬件发展趋势和速度一直深刻的决定了软件的架构。数据库类系统大多围绕“硬盘+内存”设计，高性能计算型系统大多围绕“内存 + CPU”设计，分布式系统面对千兆、万兆和 RDMA 网卡的设计也完全不同。

图基于拓扑的遍历有着极其明显的随机访问特点，因此大多数早期图数据库系统都采用了“大内存 + HDD”的架构——通过设计常驻在内存中的一些数据结构（例如B+树、Hash表等），在内存中实现随机访问目的，以优化图的拓扑遍历，再将这些随机访问转换成 HDD 所适合的顺序读写。整套软件的架构（包括存储和计算层）都必须基于和围绕这样的 IO 流程来展开。随着 SSD 价格的快速下降¹⁸，SSD 正在替代 HDD 成为持久化设备的主流。SSD 随机访问友好、IO 队列深、按块存取的特点与 HDD 高度顺序、随机时延极高、磁道易损坏的访问特点有着明显的不同。全部的软件架构也需要重新设计，这成为沉重的历史技术负担。

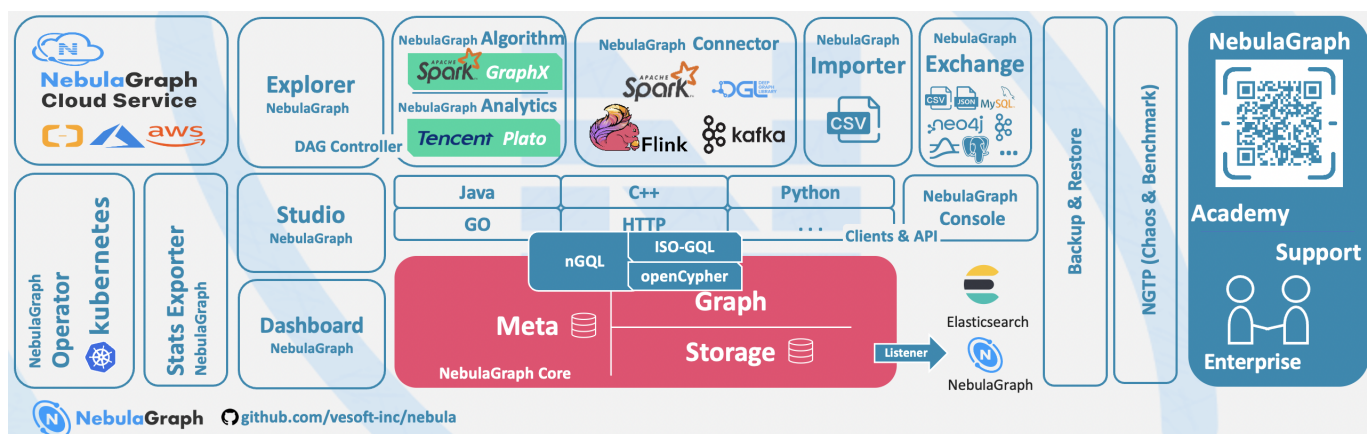


1. <https://graphaware.com/graphaware/2020/02/17/graph-technology-landscape-2020.html> ←
2. https://en.wikipedia.org/wiki/Graph_isomorphism ←
3. The Future is Big Graphs! A Community View on Graph Processing Systems. <https://arxiv.org/abs/2012.06171> ←
4. G. Malewicz, M. H. Austern, A. J. Bik, J. C. Dehnert, I. Horn, N. Leiser, and G. Czajkowski. Pregel: a system for large-scale graph processing. In Proceedings of the International Conference on Management of data (SIGMOD), pages 135-146, New York, NY, USA, 2010. ACM ←
5. <https://neo4j.com/graphacademy/training-iga-40/02-iga-40-overview-of-graph-algorithms/> ←
6. <https://livebook.manning.com/book/graph-powered-machine-learning/welcome/v-8/> ←
7. <https://www.arangodb.com/learn/graphs/using-smartgraphs-arangodb/> ←
8. <https://arxiv.org/abs/1709.03188> ←
9. <https://jgrapht.org/> ←
10. <https://github.com/jrtom/jung> ←
11. <https://igraph.org/> ←
12. <https://networkx.org/> ←
13. <https://cytoscape.org/> ←
14. <https://gephi.org/> ←
15. <https://arrows.app/> ←
16. https://github.com/ldbc/ldbc_snb_docs ←
17. <https://cloudcomputing-news.net/news/2019/apr/15/public-cloud-soaring-to-331b-by-2022-according-to-gartner/> ←
18. <https://blocksandfiles.com/2021/01/25/wikibon-ssds-vs-hard-drives-wrights-law/> ←

最后更新: 2026年8月18日

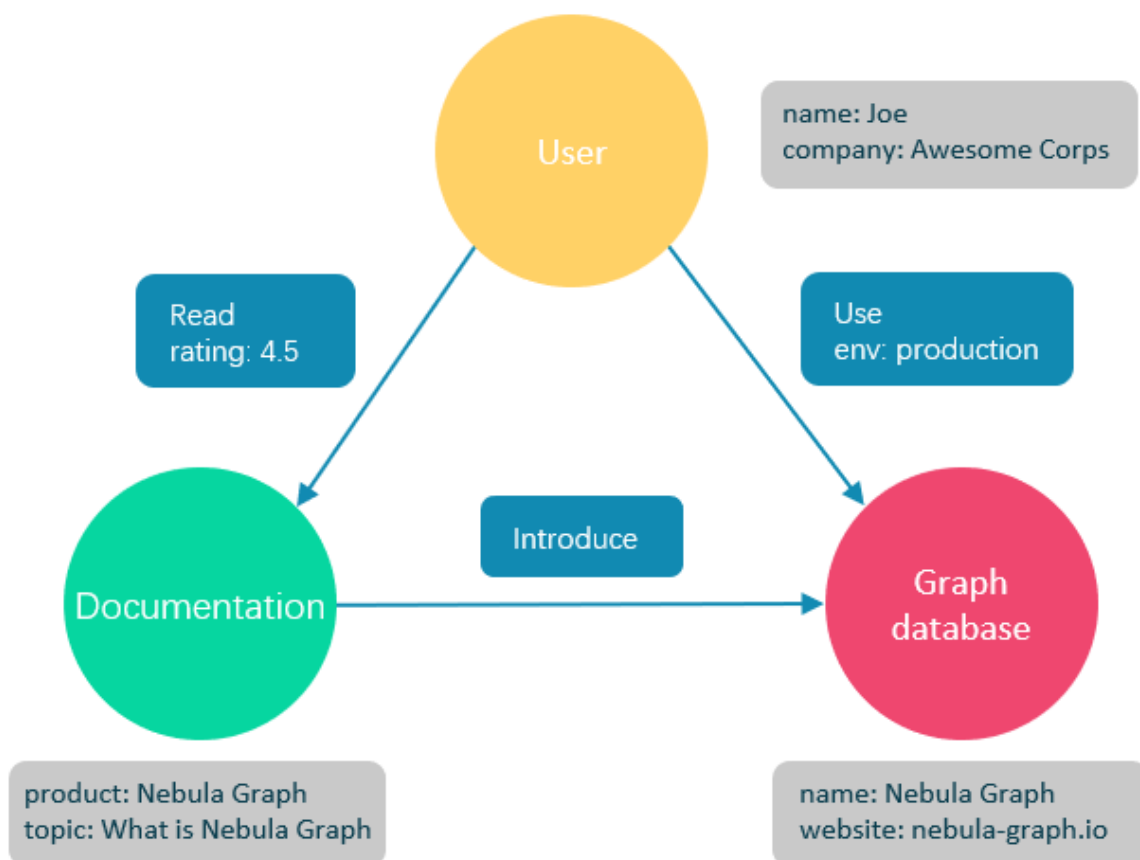
2.4 什么是NebulaGraph

NebulaGraph是一款开源的、分布式的、易扩展的原生图数据库，能够承载包含数千万个点和数万亿条边的超大规模数据集，并且提供毫秒级查询。



2.4.1 什么是图数据库

图数据库是专门存储庞大的图形网络并从中检索信息的数据库。它可以将图中的数据高效存储为点（Vertex）和边（Edge），还可以将属性（Property）附加到点和边上。



图数据库适合存储大多数从现实抽象出的数据类型。世界上几乎所有领域的事物都有内在联系，像关系型数据库这样的建模系统会提取实体之间的关系，并将关系单独存储到表和列中，而实体的类型和属性存储在其他列甚至其他表中，这使得数据管理费时费力。

NebulaGraph作为一个典型的图数据库，可以将丰富的关系通过边及其类型和属性自然地呈现。

2.4.2 NebulaGraph的优势

开源

NebulaGraph是在 Apache 2.0 条款下开发的。越来越多的人，如数据库开发人员、数据科学家、安全专家、算法工程师，都参与到 NebulaGraph的设计和开发中来，欢迎访问 [NebulaGraph GitHub 主页](#)参与开源项目。

高性能

基于图数据库的特性使用 C++ 编写的NebulaGraph，可以提供毫秒级查询。众多数据库中，NebulaGraph在图数据服务领域展现了卓越的性能，数据规模越大，NebulaGraph优势就越大。详情请参见 [NebulaGraph benchmarking 页面](#)。

易扩展

NebulaGraph采用 shared-nothing 架构，支持在不停止数据库服务的情况下扩缩容。

易开发

NebulaGraph提供 Java、Python、C++ 和 Go 等流行编程语言的客户端，更多客户端仍在开发中。详情请参见 [NebulaGraph clients](#)。

高可靠访问控制

NebulaGraph支持严格的角色访问控制和 LDAP（Lightweight Directory Access Protocol）等外部认证服务，能够有效提高数据安全性。详情请参见[验证和授权](#)。

生态多样化

NebulaGraph开放了越来越多的原生工具，例如 [NebulaGraph Studio](#)、[NebulaGraph Console](#)、[NebulaGraph Exchange](#) 等，更多工具可以查看[生态工具概览](#)。

此外，NebulaGraph还具备与 Spark、Flink、HBase 等产品整合的能力，在这个充满挑战与机遇的时代，大大增强了自身的竞争力。

兼容 openCypher 查询语言

NebulaGraph查询语言，简称为 nGQL，是一种声明性的、部分兼容 openCypher 的文本查询语言，易于理解和使用。详细语法请参见 [nGQL 指南](#)。

面向未来硬件，读写平衡

闪存型设备有着极高的性能，并且[价格快速下降](#)，NebulaGraph是一个面向 SSD 设计的产品，相比于基于 HDD + 大内存的产品，更适合面向未来的硬件趋势，也更容易做到读写平衡。

灵活数据建模

用户可以轻松地在NebulaGraph中建立数据模型，不必将数据强制转换为关系表。而且可以自由增加、更新和删除属性。详情请参见[数据模型](#)。

广受欢迎

腾讯、美团、京东、快手、360 等科技巨头都在使用NebulaGraph。详情请参见 [NebulaGraph官网](#)。

2.4.3 适用场景

NebulaGraph可用于各种基于图的业务场景。为节约转换各类数据到关系型数据库的时间，以及避免复杂查询，建议使用**NebulaGraph**。

欺诈检测

金融机构必须仔细研究大量的交易信息，才能检测出潜在的金融欺诈行为，并了解某个欺诈行为和设备的内在关联。这种场景可以通过图来建模，然后借助**NebulaGraph**，可以很容易地检测出诈骗团伙或其他复杂诈骗行为。

实时推荐

NebulaGraph能够及时处理访问者产生的实时信息，并且精准推送文章、视频、产品和服务。

知识图谱

自然语言可以转化为知识图谱，存储在**NebulaGraph**中。用自然语言组织的问题可以通过智能问答系统中的语义解析器进行解析并重新组织，然后从知识图谱中检索出问题的可能答案，提供给提问者。

社交网络

人际关系信息是典型的图数据，**NebulaGraph**可以轻松处理数十亿人和数万亿人际关系的社交网络信息，并在海量并发的情况下，提供快速的好友推荐和工作岗位查询。

2.4.4 视频

用户也可以通过视频了解什么是图数据。

- [NebulaGraph介绍视频](#)（01 分 39 秒）

2.4.5 主题演讲

查看[演讲](#)快速了解图数据库概况。

2.4.6 相关链接

- [官方网站](#)
- [文档首页](#)
- [博客首页](#)
- [论坛](#)
- [GitHub](#)

最后更新: 2026年8月18日

2.5 数据模型

本文介绍NebulaGraph的数据模型。数据模型是一种组织数据并说明它们如何相互关联的模型。

2.5.1 数据模型

NebulaGraph数据模型使用 6 种基本的数据模型：

- 图空间（Space）

图空间用于隔离不同团队或者项目的数据。不同图空间的数据是相互隔离的，可以指定不同的存储副本数、权限、分片等。

- 点（Vertex）

点用来保存实体对象，特点如下：

- 点是用点标识符（VID）标识的。VID 在同一图空间中唯一。VID 是一个 int64，或者 fixed_string(N)。
- 点可以有 0 到多个 Tag。



Compatibility

NebulaGraph 2.x 及以下版本中的点必须包含至少一个 Tag。

- 边（Edge）

边是用来连接点的，表示两个点之间的关系或行为，特点如下：

- 两点之间可以有多条边。
- 边是有方向的，不存在无向边。
- 四元组 <起点 VID、Edge type、边排序值 (rank)、终点 VID> 用于唯一标识一条边。边没有 EID。
- 一条边有且仅有一个 Edge type。
- 一条边有且仅有一个 Rank，类型为 int64，默认值为 0。



关于

Rank 可以用来区分 Edge type、起始点、目的点都相同的边。该值完全由用户自己指定。

读取时必须自行取得全部的 Rank 值后排序过滤和拼接。

不支持诸如 next(), pre(), head(), tail(), max(), min(), lessThan(), moreThan() 等函数功能，也不能通过创建索引加速访问或者条件过滤。

- 标签（Tag）

Tag 由一组事先预定义的属性构成。

- 边类型（Edge type）

Edge type 由一组事先预定义的属性构成。

- 属性（Property）

属性是指以键值对（Key-value pair）形式表示的信息。



Note

Tag 和 Edge type 的作用，类似于关系型数据库中“点表”和“边表”的表结构。

2.5.2 有向属性图

NebulaGraph使用有向属性图模型，指点和边构成的图，这些边是有方向的，点和边都可以有属性。

下表为篮球运动员数据集的结构示例，包括两种类型的点（**player**、**team**）和两种类型的边（**serve**、**follow**）。

类型	名称	属性名（数据类型）	说明
Tag	player	name (string) age (int)	表示球员。
Tag	team	name (string)	表示球队。
Edge type	serve	start_year (int) end_year (int)	表示球员的行为。 该行为将球员和球队联系起来，方向是从球员到球队。
Edge type	follow	degree (int)	表示球员的行为。 该行为将两个球员联系起来，方向是从一个球员到另一个球员。



Note

NebulaGraph中没有无向边，只支持有向边。



Compatibility

由于NebulaGraph 3.5.0 的数据模型中，允许存在“悬挂边”，因此在增删时，用户需自行保证“一条边所对应的起点和终点”的存在性。详见 **INSERT VERTEX**、**DELETE VERTEX**、**INSERT EDGE**、**DELETE EDGE**。

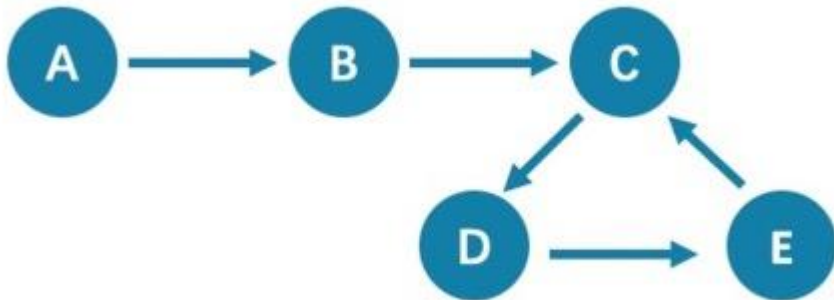
不支持 openCypher 中的 MERGE 语句。

2.6 路径

图论中一个非常重要的概念是路径，路径是指一个有限或无限的边序列，这些边连接着一系列的点。

路径的类型分为三种：`walk`、`trail`、`path`。关于路径的详细说明，请参见[维基百科](#)。

本文以下图为例进行简单介绍。



2.6.1 walk

`walk` 类型的路径由有限或无限的边序列构成。遍历时点和边可以重复。

查看示例图，由于 C、D、E 构成了一个环，因此该图包含无限个路径，例如 A->B->C->D->E、A->B->C->D->E->C、A->B->C->D->E->C->D。

Note

`GO` 语句采用的是 `walk` 类型路径。

2.6.2 trail

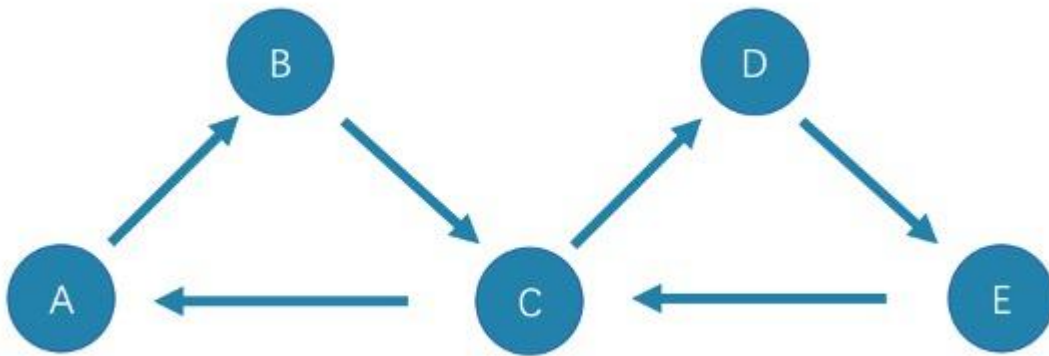
`trail` 类型的路径由有限的边序列构成。遍历时只有点可以重复，边不可以重复。柯尼斯堡七桥问题的路径类型就是 `trail`。

查看示例图，由于边不可以重复，所以该图包含有限个路径，最长路径由 5 条边组成：A->B->C->D->E->C。

Note

`MATCH`、`FIND PATH` 和 `GET SUBGRAPH` 语句采用的是 `trail` 类型路径。

在 `trail` 类型中，还有 `cycle` 和 `circuit` 两种特殊的路径类型，以下图为例对这两种特殊的路径类型进行介绍。



- cycle

`cycle` 是封闭的 `trail` 类型的路径，遍历时边不可以重复，起点和终点重复，并且没有其他点重复。在此示例图中，最长路径由三条边组成：`A->B->C->A` 或 `C->D->E->C`。

- circuit

`circuit` 也是封闭的 `trail` 类型的路径，遍历时边不可以重复，除起点和终点重复外，可能存在其他点重复。在此示例图中，最长路径为：`A->B->C->D->E->C->A`。

2.6.3 path

`path` 类型的路径由有限的边序列构成。遍历时点和边都不可以重复。

查看示例图，由于点和边都不可以重复，所以该图包含有限个路径，最长路径由 4 条边组成：`A->B->C->D->E`。

2.6.4 视频

用户也可以观看视频了解路径的相关概念。

Path (03 分 09 秒)

最后更新: 2026年8月18日

2.7 点 VID

在一个图空间中，一个点由点的 ID 唯一标识，即 VID 或 Vertex ID。

2.7.1 VID 的特点

- VID 数据类型只可以为定长字符串 `FIXED_STRING(<N>)` 或 `INT64`。一个图空间只能选用其中一种 VID 类型。
- VID 在一个图空间中必须唯一，其作用类似于关系型数据库中的主键（索引+唯一约束）。但不同图空间中的 VID 是完全独立无关的。
- 点 VID 的生成方式必须由用户自行指定，系统不提供自增 ID 或者 UUID。
- VID 相同的点，会被认为是同一个点。例如：
- VID 相当于一个实体的唯一标号，例如一个人的身份证号。**Tag** 相当于实体所拥有的类型，例如"滴滴司机"和"老板"。不同的 **Tag** 又相应定义了两组不同的属性，例如"驾照号、驾龄、接单量、接单小号"和"工号、薪水、债务额度、商务电话"。
- 同时操作相同 VID 并且相同 Tag 的两条 `INSERT` 语句（均无 `IF NOT EXISTS` 参数），晚写入的 `INSERT` 会覆盖先写入的。
- 同时操作包含相同 VID 但是两个不同 TAG A 和 TAG B 的两条 `INSERT` 语句，对 TAG A 的操作不会影响 TAG B。
- VID 通常会被（LSM-tree 方式）索引并缓存在内存中，因此直接访问 VID 的性能最高。

2.7.2 VID 使用建议

- NebulaGraph 1.x 只支持 VID 类型为 `INT64`，从 2.x 开始支持 `INT64` 和 `FIXED_STRING(<N>)`。在 `CREATE SPACE` 中通过参数 `vid_type` 可以指定 VID 类型。
- 可以使用 `id()` 函数，指定或引用该点的 VID。
- 可以使用 `LOOKUP` 或者 `MATCH` 语句，来通过属性索引查找对应的 VID。
- 性能上，直接通过 VID 找到点的语句性能最高，例如 `DELETE xxx WHERE id(xxx) == "player100"`，或者 `GO FROM "player100"` 等语句。通过属性先查找 VID，再进行图操作的性能会变差，例如 `LOOKUP | GO FROM $-.ids` 等语句，相比前者多了一次内存或硬盘的随机读（`LOOKUP`）以及一次序列化（`|`）。

2.7.3 VID 生成建议

VID 的生成工作完全交给应用端，有一些通用的建议：

- （最优）通过有唯一性的主键或者属性来直接作为 VID；属性访问依赖于 VID；
- 通过有唯一性的属性组合来生成 VID，属性访问依赖于属性索引。
- 通过 `snowflake` 等算法生成 VID，属性访问依赖于属性索引。
- 如果个别记录的主键特别长，但绝大多数记录的主键都很短的情况，不要将 `FIXED_STRING(<N>)` 的 N 设置成超大，这会浪费大量内存和硬盘，也会降低性能。此时可通过 `BASE64`，`MD5`，`hash` 编码加拼接的方式来生成。
- 如果用 `hash` 方式生成 `int64` VID：在有 10 亿个点的情况下，发生 `hash` 冲突的概率大约是 1/10。边的数量与碰撞的概率无关。

2.7.4 定义和修改 VID 与其数据类型

VID 的数据类型必须在创建图空间时定义，且一旦定义无法修改。

VID 必须在插入点时设置，且一旦设置无法修改。

2.7.5 "查询起始点"(start vid) 与全局扫描

绝大多数情况下，NebulaGraph的查询语句（MATCH、GO、LOOKUP）的执行计划，必须要通过一定方式找到查询起始点的 VID（start vid）。

定位 start vid 只有两种方式：

1. 例如 `GO FROM "player100" OVER` 是在语句中显式的指明 start vid 是 "player100"；
 2. 例如 `LOOKUP ON player WHERE player.name == "Tony Parker"` 或者 `MATCH (v:player {name:"Tony Parker"})`，是通过属性 player.name 的索引来定位到 start vid；
-

最后更新: 2026年8月18日

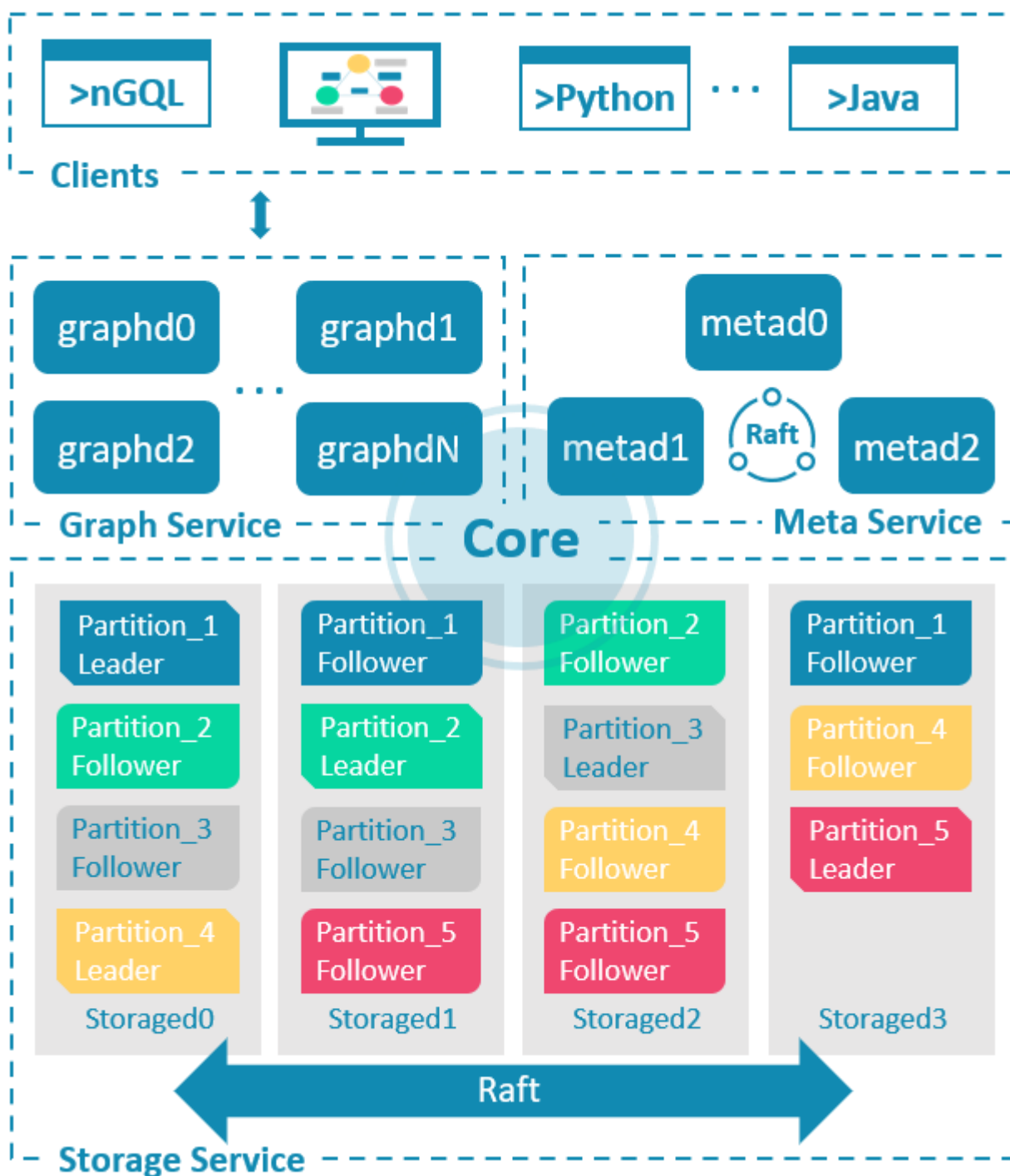
2.8 服务架构

2.8.1 NebulaGraph架构总览

NebulaGraph由三种服务构成：Graph 服务、Meta 服务和 Storage 服务，是一种存储与计算分离的架构。

每个服务都有可执行的二进制文件 and 对应进程，用户可以使用这些二进制文件在一个或多个计算机上部署NebulaGraph集群。

下图展示了NebulaGraph集群的经典架构。



Meta 服务

在NebulaGraph架构中，Meta 服务是由 `nebula-metad` 进程提供的，负责数据管理，例如 Schema 操作、集群管理和用户权限管理等。

Meta 服务的详细说明，请参见 [Meta 服务](#)。

Graph 服务和 Storage 服务

NebulaGraph采用计算存储分离架构。Graph 服务负责处理计算请求，Storage 服务负责存储数据。它们由不同的进程提供，Graph 服务是由 `nebula-graphd` 进程提供，Storage 服务是由 `nebula-storaged` 进程提供。计算存储分离架构的优势如下：

- 易扩展

分布式架构保证了 Graph 服务和 Storage 服务的灵活性，方便扩容和缩容。

- 高可用

如果提供 Graph 服务的服务器有一部分出现故障，其余服务器可以继续为客户端提供服务，而且 Storage 服务存储的数据不会丢失。服务恢复速度较快，甚至能做到用户无感知。

- 节约成本

计算存储分离架构能够提高资源利用率，而且可根据业务需求灵活控制成本。

- 更多可能性

基于分离架构的特性，Graph 服务将可以在更多类型的存储引擎上单独运行，Storage 服务也可以为多种目的的计算引擎提供服务。

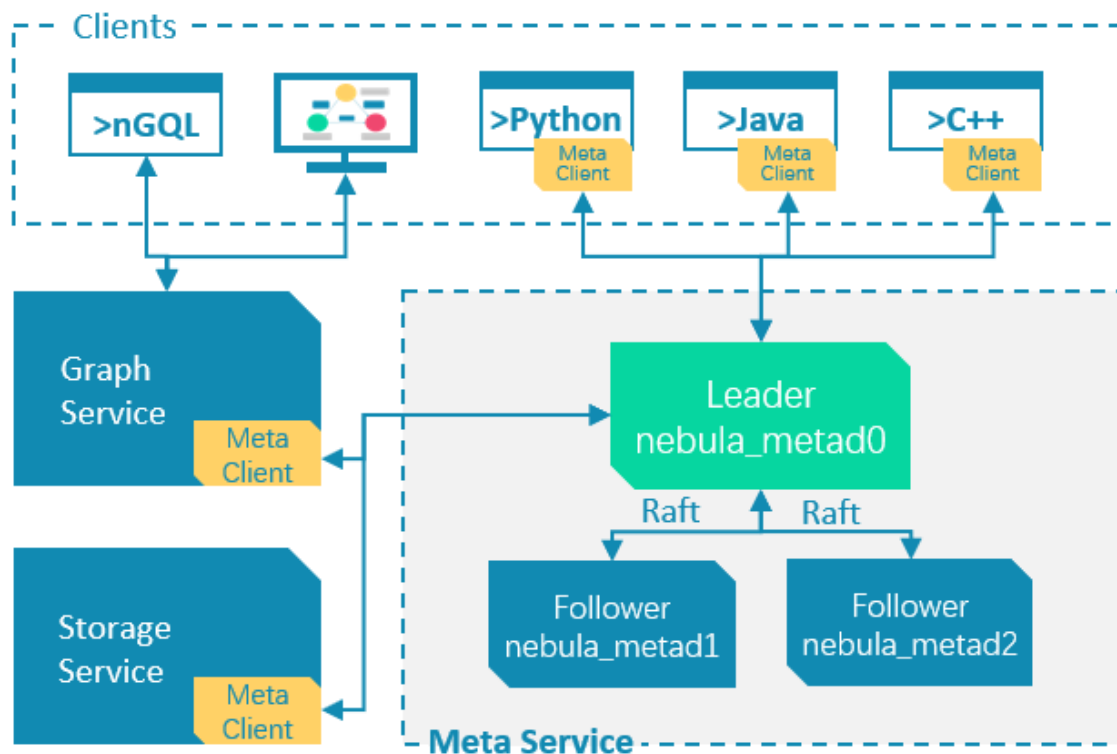
Graph 服务和 Storage 服务的详细说明，请参见 [Graph 服务](#)和 [Storage 服务](#)。

最后更新: 2026年8月18日

2.8.2 Meta 服务

本文介绍 Meta 服务的架构和功能。

Meta 服务架构



Meta 服务是由 `nebula-metad` 进程提供的，用户可以根据场景配置 `nebula-metad` 进程数量：

- 测试环境中，用户可以在 NebulaGraph 集群中部署 1 个或 3 个 `nebula-metad` 进程。如果要部署 3 个，用户可以将它们部署在 1 台机器上，或者分别部署在不同的机器上。
- 生产环境中，建议在 NebulaGraph 集群中部署 3 个 `nebula-metad` 进程。请将这些进程部署在不同的机器上以保证高可用。

所有 `nebula-metad` 进程构成了基于 Raft 协议的集群，其中一个进程是 leader，其他进程都是 follower。

leader 是由多数派选举出来，只有 leader 能够对客户端或其他组件提供服务，其他 follower 作为候补，如果 leader 出现故障，会在所有 follower 中选举出新的 leader。

Note

leader 和 follower 的数据通过 Raft 协议保持一致，因此 leader 故障和选举新 leader 不会导致数据不一致。更多关于 Raft 的介绍见 [Storage 服务](#)。

Meta 服务功能

管理用户账号

Meta 服务中存储了用户的账号和权限信息，当客户端通过账号发送请求给 Meta 服务，Meta 服务会检查账号信息，以及该账号是否有对应的请求权限。

更多NebulaGraph的访问控制说明，请参见[身份验证](#)。

管理分片

Meta 服务负责存储和管理分片的位置信息，并且保证分片的负载均衡。

管理图空间

NebulaGraph支持多个图空间，不同图空间内的数据是安全隔离的。**Meta** 服务存储所有图空间的元数据（非完整数据），并跟踪数据的变更，例如增加或删除图空间。

管理 SCHEMA 信息

NebulaGraph是强类型图数据库，它的 **Schema** 包括 **Tag**、**Edge type**、**Tag** 属性和 **Edge type** 属性。

Meta 服务中存储了 **Schema** 信息，同时还负责 **Schema** 的添加、修改和删除，并记录它们的版本。

更多NebulaGraph的 **Schema** 信息，请参见[数据模型](#)。

管理 TTL 信息

Meta 服务存储 **TTL**（Time To Live）定义信息，可以用于设置数据生命周期。数据过期后，会由 **Storage** 服务进行处理，具体过程参见 [TTL](#)。

管理作业

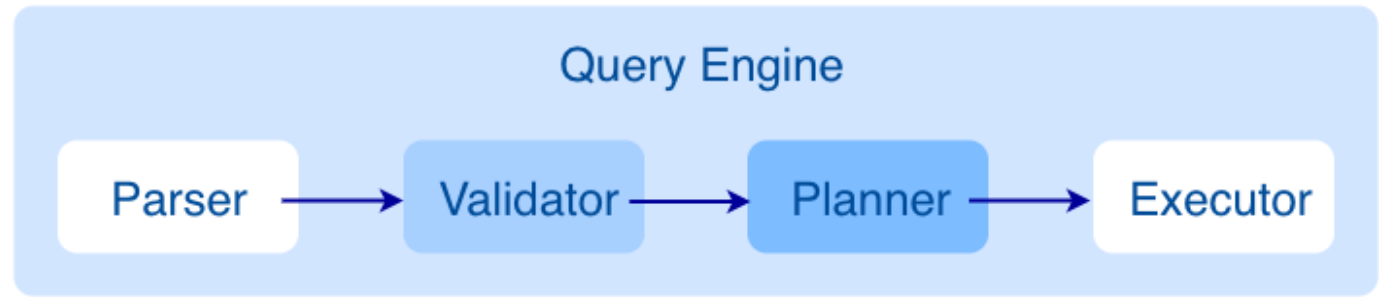
Meta 服务中的作业管理模块负责作业的创建、排队、查询和删除。

最后更新: 2026年8月18日

2.8.3 Graph 服务

Graph 服务主要负责处理查询请求，包括解析查询语句、校验语句、生成执行计划以及按照执行计划执行四个大步骤，本文将基于这些步骤介绍 Graph 服务。

Graph 服务架构



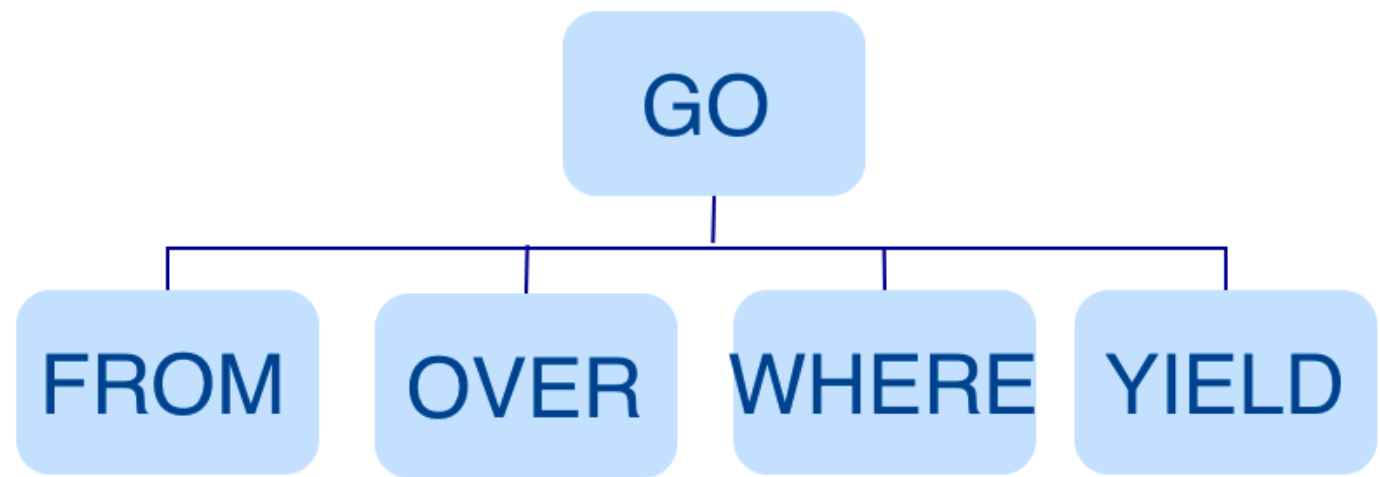
查询请求发送到 Graph 服务后，会由如下模块依次处理：

- 1. **Parser**: 词法语法解析模块。
- 2. **Validator**: 语义校验模块。
- 3. **Planner**: 执行计划与优化器模块。
- 4. **Executor**: 执行引擎模块。

Parser

Parser 模块收到请求后，通过 Flex（词法分析工具）和 Bison（语法分析工具）生成的词法语法解析器，将语句转换为抽象语法树（AST），在语法解析阶段会拦截不符合语法规则的语句。

例如 `GO FROM "Tim" OVER Like WHERE properties(edge).likeness > 8.0 YIELD dst(edge)` 语句转换的 AST 如下。



Validator

Validator 模块对生成的 AST 进行语义校验，主要包括：

- 校验元数据信息

校验语句中的元数据信息是否正确。

例如解析 `OVER`、`WHERE` 和 `YIELD` 语句时，会查找 Schema 校验 `Edge type`、`Tag` 的信息是否存在，或者插入数据时校验插入的数据类型和 Schema 中的是否一致。

- 校验上下文引用信息

校验引用的变量是否存在或者引用的属性是否属于变量。

例如语句 `$var = GO FROM "Tim" OVER Like YIELD dst(edge) AS ID; GO FROM $var.ID OVER serve YIELD dst(edge)`，Validator 模块首先会检查变量 `var` 是否定义，其次再检查属性 `ID` 是否属于变量 `var`。

- 校验类型推断

推断表达式的结果类型，并根据子句校验类型是否正确。

例如 `WHERE` 子句要求结果是 `bool`、`null` 或者 `empty`。

- 校验 `*` 代表的信息

查询语句中包含 `*` 时，校验子句时需要将 `*` 涉及的 Schema 都进行校验。

例如语句 `GO FROM "Tim" OVER * YIELD dst(edge), properties(edge).likeness, dst(edge)`，校验 `OVER` 子句时需要校验所有的 `Edge type`，如果 `Edge type` 包含 `Like` 和 `serve`，该语句会展开为 `GO FROM "Tim" OVER Like,serve YIELD dst(edge), properties(edge).likeness, dst(edge)`。

- 校验输入输出

校验管道符 (`|`) 前后的一致性。

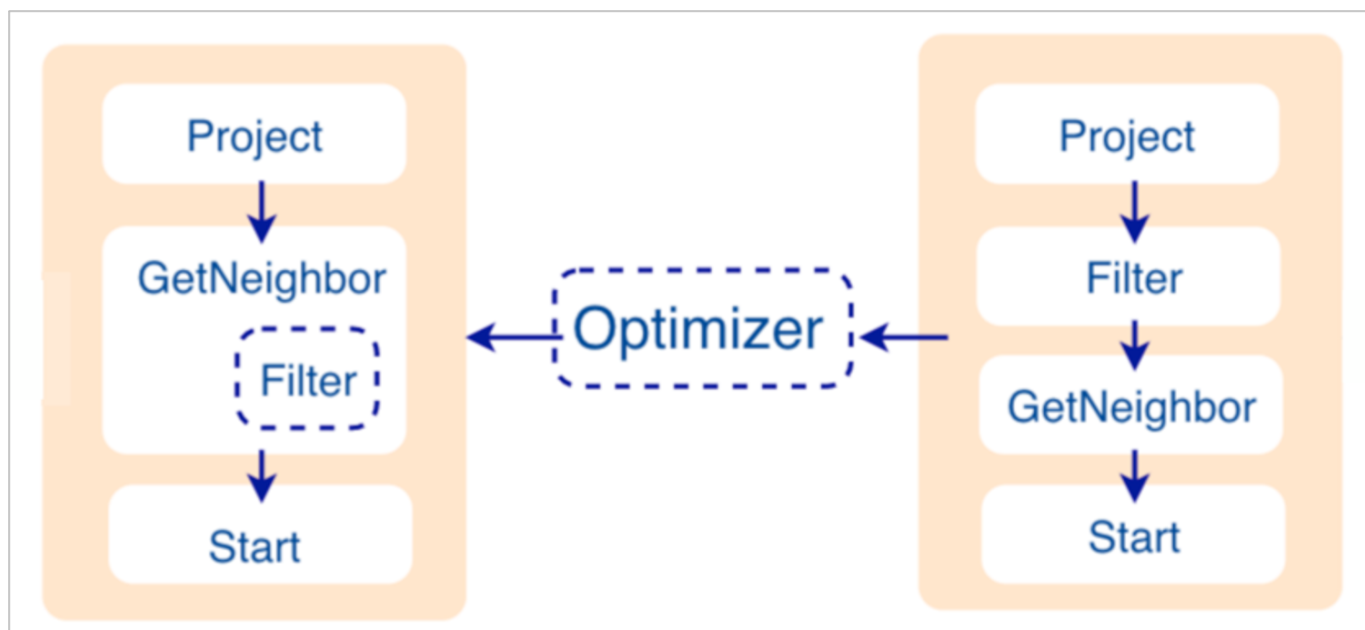
例如语句 `GO FROM "Tim" OVER Like YIELD dst(edge) AS ID | GO FROM $-.ID OVER serve YIELD dst(edge)`，Validator 模块会校验 `$-.ID` 在管道符左侧是否已经定义。

校验完成后，Validator 模块还会生成一个默认可执行，但是未进行优化的执行计划，存储在目录 `src/planner` 内。

Planner

如果配置文件 `nebula-graphd.conf` 中 `enable_optimizer` 设置为 `false`，Planner 模块不会优化 Validator 模块生成的执行计划，而是直接交给 Executor 模块执行。

如果配置文件 `nebula-graphd.conf` 中 `enable_optimizer` 设置为 `true`，Planner 模块会对 Validator 模块生成的执行计划进行优化。如下图所示。



• 优化前

如上图右侧未优化的执行计划，每个节点依赖另一个节点，例如根节点 **Project** 依赖 **Filter**、**Filter** 依赖 **GetNeighbor**，最终找到叶子节点 **Start**，才能开始执行（并非真正执行）。

在这个过程中，每个节点会有对应的输入变量和输出变量，这些变量存储在一个哈希表中。由于执行计划不是真正执行，所以哈希表中每个 **key** 的 **value** 值都为空（除了 **Start** 节点，起始数据会存储在该节点的输入变量中）。哈希表定义在仓库 **nebula-graph** 内的 **src/context/ExecutionContext.cpp** 中。

例如哈希表的名称为 **ResultMap**，在建立 **Filter** 这个节点时，定义该节点从 **ResultMap["GN1"]** 中读取数据，然后将结果存储在 **ResultMap["Filter2"]** 中，依次类推，将每个节点的输入输出都确定好。

• 优化过程

Planner 模块目前的优化方式是 **RBO**（rule-based optimization），即预定义优化规则，然后对 **Validator** 模块生成的默认执行计划进行优化。新的优化规则 **CBO**（cost-based optimization）正在开发中。优化代码存储在仓库 **nebula-graph** 的目录 **src/optimizer/** 内。

RBO 是一个自底向上的探索过程，即对于每个规则而言，都会由执行计划的根节点（示例是 **Project**）开始，一步步向下探索到最底层的节点，在过程中查看是否可以匹配规则。

如上图所示，探索到节点 **Filter** 时，发现依赖的节点是 **GetNeighbor**，匹配预先定义的规则，就会将 **Filter** 融入到 **GetNeighbor** 中，然后移除节点 **Filter**，继续匹配下一个规则。在执行阶段，当算子 **GetNeighbor** 调用 **Storage** 服务的接口获取一个点的邻边时，**Storage** 服务内部会直接将不符合条件的边过滤掉，这样可以极大地减少传输的数据量，该优化称为过滤下推。

Executor

Executor 模块包含调度器（**Scheduler**）和执行器（**Executor**），通过调度器调度执行计划，让执行器根据执行计划生成对应的执行算子，从叶子节点开始执行，直到根节点结束。如下图所示。



每一个执行计划节点都一一对应一个执行算子，节点的输入输出在优化执行计划时已经确定，每个算子只需要拿到输入变量中的值进行计算，最后将计算结果放入对应的输出变量中即可，所以只需要从节点 `Start` 一步步执行，最后一个算子的输出变量会作为最终结果返回给客户端。

代码结构

NebulaGraph的代码层次结构如下：

```
|--src
|  |--graph
|  |  |--context    //校验期和执行期上下文
|  |  |--executor   //执行算子
|  |  |--gc         //垃圾收集器
|  |  |--optimizer  //优化规则
|  |  |--planner    //执行计划结构
|  |  |--scheduler  //调度器
|  |  |--service    //对外服务管理
|  |  |--session    //会话管理
|  |  |--stats      //运行指标
|  |  |--util       //基础组件
|  |  |--validator  //语句校验
|  |  |--visitor    //visitor表达式
```

视频

用户也可以通过视频全方位了解NebulaGraph的查询引擎。

- [nMeetup·上海 | 全面解析 Query Engine](#) (33 分 30 秒)

最后更新: 2026年8月18日

2.8.4 Storage 服务

NebulaGraph 的存储包含两个部分，一个是 **Meta** 相关的存储，称为 **Meta** 服务，在前文已有介绍。

另一个是具体数据相关的存储，称为 **Storage** 服务。其运行在 `nebula-storaged` 进程中。本文仅介绍 **Storage** 服务的架构设计。

优势

- 高性能（自研 KVStore）
- 易水平扩展（Shared-nothing 架构，不依赖 NAS 等硬件设备）
- 强一致性（Raft）
- 高可用性（Raft）
- 支持向第三方系统进行同步（例如[全文索引](#)）

Storage 服务架构

image

Storage 服务是由 `nebula-storaged` 进程提供的，用户可以根据场景配置 `nebula-storaged` 进程数量，例如测试环境 1 个，生产环境 3 个。

所有 `nebula-storaged` 进程构成了基于 **Raft** 协议的集群，整个服务架构可以分为三层，从上到下依次为：

• Storage interface 层

Storage 服务的最上层，定义了一系列和图相关的 **API**。**API** 请求会在这一层被翻译成一组针对分片的 **KV** 操作，例如：

- `getNeighbors`：查询一批点的出边或者入边，返回边以及对应的属性，并且支持条件过滤。
- `insert vertex/edge`：插入一条点或者边及其属性。
- `getProps`：获取一个点或者一条边的属性。

正是这一层的存在，使得 **Storage** 服务变成了真正的图存储，否则 **Storage** 服务只是一个 **KV** 存储服务。

• Consensus 层

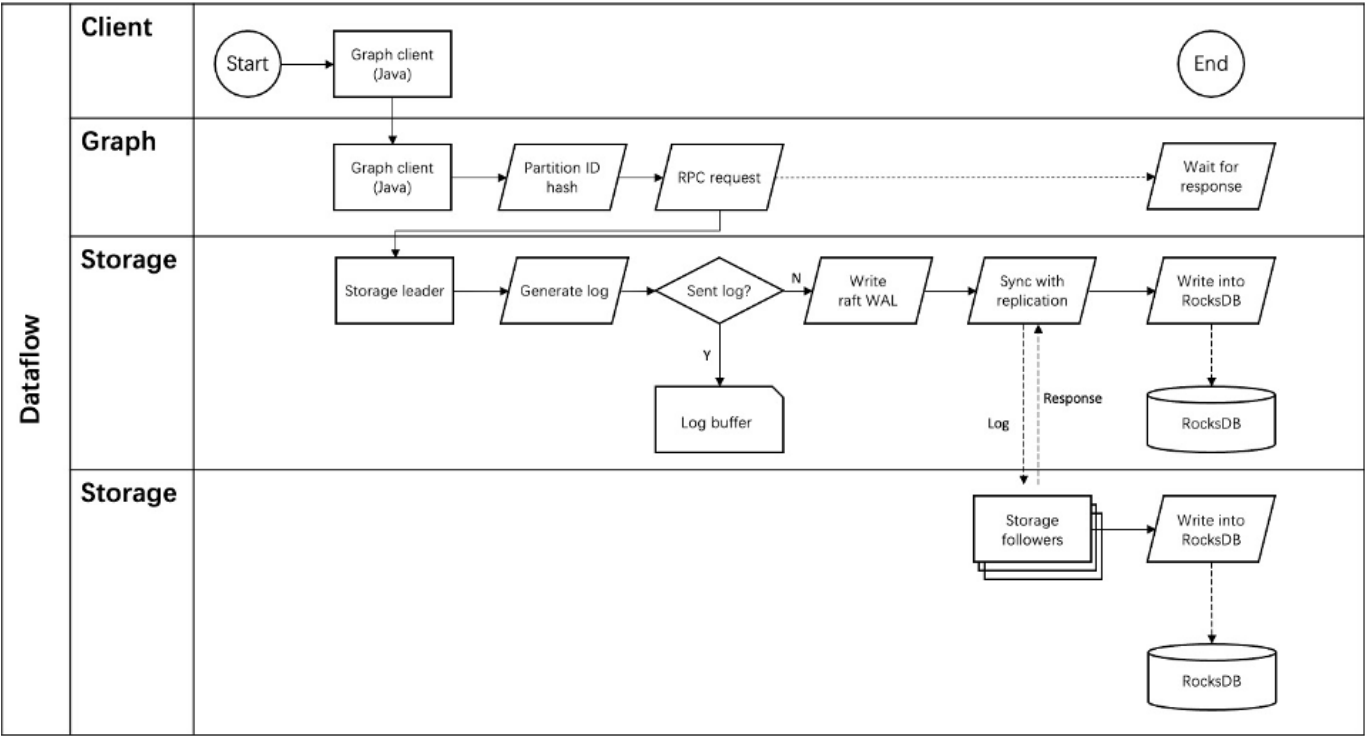
Storage 服务的中间层，实现了 **Multi Group Raft**，保证强一致性和高可用性。

• Store Engine 层

Storage 服务的最底层，是一个单机本地存储引擎，提供对本地数据的 `get`、`put`、`scan` 等操作。相关接口存储在 `KVStore.h` 和 `KVEngine.h` 文件，用户可以根据业务需求定制开发相关的本地存储插件。

下文将基于架构介绍 **Storage** 服务的部分特性。

Storage 写入流程



KVStore

NebulaGraph使用自行开发的 KVStore，而不是其他开源 KVStore，原因如下：

- 需要高性能 KVStore。
- 需要以库的形式提供，实现高效计算下推。对于强 Schema 的NebulaGraph来说，计算下推时如何提供 Schema 信息，是高效的关键。
- 需要数据强一致性。

基于上述原因，NebulaGraph使用 RocksDB 作为本地存储引擎，实现了自己的 KVStore，有如下优势：

- 对于多硬盘机器，NebulaGraph只需配置多个不同的数据目录即可充分利用多硬盘的并发能力。
- 由 Meta 服务统一管理所有 Storage 服务，可以根据所有分片的分布情况和状态，手动进行负载均衡。



不支持自动负载均衡是为了防止自动数据搬迁影响线上业务。

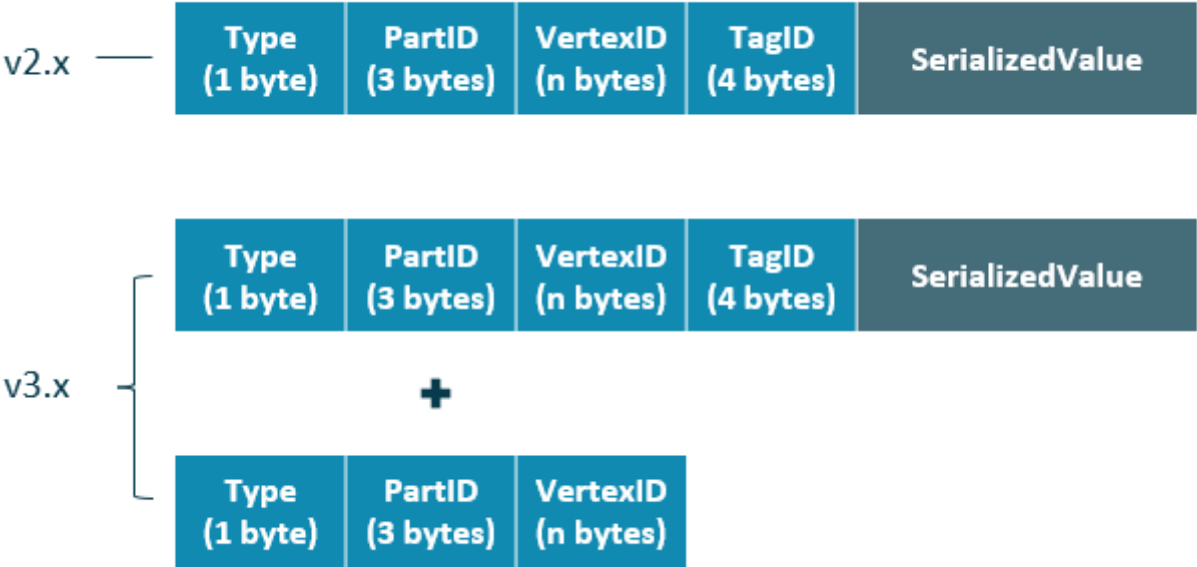
- 定制预写日志（WAL），每个分片都有自己的 WAL。
- 支持多个图空间，不同图空间相互隔离，每个图空间可以设置自己的分片数和副本数。

数据存储格式

图存储的主要数据是点和边，NebulaGraph将点和边的信息存储为 **key**，同时将点和边的属性信息存储在 **value** 中，以便更高效地使用属性过滤。

- 点数据存储格式

相比NebulaGraph 2.x 版本，3.x 版本在开启无 **Tag** 的点配置后，每个点多了一个不含 TagID 字段并且无 value 的 key。



字段	说明
Type	key 类型。长度为 1 字节。
PartID	数据分片编号。长度为 3 字节。此字段主要用于 Storage 负载均衡（ balance ）时方便根据前缀扫描整个分片的数据。
VertexID	点 ID。当点 ID 类型为 int 时，长度为 8 字节；当点 ID 类型为 string 时，长度为创建图空间时指定的 fixed_string 长度。
TagID	点关联的 Tag ID。长度为 4 字节。
SerializedValue	序列化的 value ，用于保存点的属性信息。

- 边数据存储格式

Type (1 byte)	PartID (3 bytes)	VertexID (n bytes)	EdgeType (4 bytes)	Rank (8 bytes)	VertexID (n bytes)	Placeholder (1 byte)	SerializedValue
------------------	---------------------	-----------------------	-----------------------	-------------------	-----------------------	-------------------------	-----------------

字段	说明
Type	key 类型。长度为 1 字节。
PartID	数据分片编号。长度为 3 字节。此字段主要用于 Storage 负载均衡（balance）时方便根据前缀扫描整个分片的数据。
VertexID	点 ID。前一个 VertexID 在出边里表示起始点 ID，在入边里表示目的点 ID；后一个 VertexID 出边里表示目的点 ID，在入边里表示起始点 ID。
Edge type	边的类型。大于 0 表示出边，小于 0 表示入边。长度为 4 字节。
Rank	用来处理两点之间有多同类型边的情况。用户可以根据自己的需求进行设置，例如存放交易时间、交易流水号等。长度为 8 字节，
Placeholder	预留字段。长度为 1 字节。
SerializedValue	序列化的 value，用于保存边的属性信息。

属性说明

NebulaGraph使用强类型 Schema。

对于点或边的属性信息，NebulaGraph会将属性信息编码后按顺序存储。由于定长属性的长度是固定的，查询时可以根据偏移量快速查询。在解码之前，需先从 Meta 服务中查询具体的 Schema 信息（并缓存）。同时为了支持在线变更 Schema，在编码属性时，会加入对应的 Schema 版本信息。

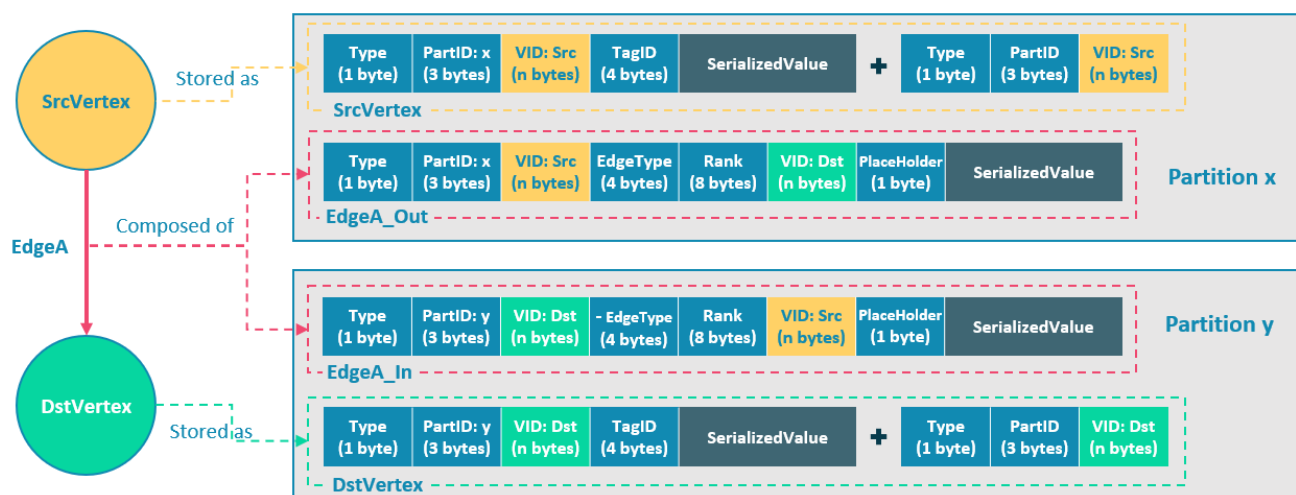
数据分片

由于超大规模关系网络的节点数量高达百亿到千亿，而边的数量更会高达万亿，即使仅存储点和边两者也远大于一般服务器的容量。因此需要有方法将图元素切割，并存储在不同逻辑分片（Partition）上。NebulaGraph采用边分割的方式。

data partitioning

切边与存储放大

NebulaGraph中逻辑上的一条边对应着硬盘上的两个键值对（key-value pair），在边的数量和属性较多时，存储放大现象较明显。边的存储方式如下图所示。



上图以最简单的两个点和一条边为例，起点 **SrcVertex** 通过边 **EdgeA** 连接目的点 **DstVertex**，形成路径 (SrcVertex)-[EdgeA]->(DstVertex)。这两个点和一条边会以 6 个键值对的形式保存在存储层的两个不同分片，即 **Partition x** 和 **Partition y** 中，详细说明如下：

- 点 **SrcVertex** 的键值保存在 **Partition x** 中。
- 边 **EdgeA** 的第一份键值，这里用 **EdgeA_Out** 表示，与 **SrcVertex** 一同保存在 **Partition x** 中。key 的字段有 **Type**、**PartID** (x)、**VID** (Src，即点 **SrcVertex** 的 ID)、**EdgeType** (符号为正，代表边方向为出)、**Rank** (0)、**VID** (Dst，即点 **DstVertex** 的 ID) 和 **Placeholder**。SerializedValue 即 Value，是序列化的边属性。
- 点 **DstVertex** 的键值保存在 **Partition y** 中。
- 边 **EdgeA** 的第二份键值，这里用 **EdgeA_In** 表示，与 **DstVertex** 一同保存在 **Partition y** 中。key 的字段有 **Type**、**PartID** (y)、**VID** (Dst，即点 **DstVertex** 的 ID)、**EdgeType** (符号为负，代表边方向为入)、**Rank** (0)、**VID** (Src，即点 **SrcVertex** 的 ID) 和 **Placeholder**。SerializedValue 即 Value，是序列化的边属性，与 **EdgeA_Out** 中该部分的完全相同。

EdgeA_Out 和 **EdgeA_In** 以方向相反的两条边的形式存在于存储层，二者组合成了逻辑上的一条边 **EdgeA**。**EdgeA_Out** 用于从起点开始的遍历请求，例如 (a)-[]->(); **EdgeA_In** 用于指向目的点的遍历请求，或者说从目的点开始，沿着边的方向逆序进行的遍历请求，例如例如 ()-[]->(a)。

如 **EdgeA_Out** 和 **EdgeA_In** 一样，**NebulaGraph** 冗余了存储每条边的信息，导致存储边所需的实际空间翻倍。因为边对应的 key 占用的硬盘空间较小，但 value 占用的空间与属性值的长度和数量成正比，所以，当边的属性值较大或数量较多时候，硬盘空间占用量会比较大。

分片算法

分片策略采用静态 **Hash** 的方式，即对点 **VID** 进行取模操作，同一个点的所有 **Tag**、出边和入边信息都会存储到同一个分片，这种方式极大地提升了查询效率。

Note

创建图空间时需指定分片数量，分片数量设置后无法修改，建议设置时提前满足业务将来的扩容需求。

多机集群部署时，分片分布在集群内的不同机器上。分片数量在 **CREATE SPACE** 语句中指定，此后不可更改。

如果需要将某些点放置在相同的分片（例如在一台机器上），可以参考[公式或代码](#)。

下文用简单代码说明 **VID** 和分片的关系。

```
// 如果 ID 长度为 8，为了兼容 1.0，将数据类型视为 int64。
uint64_t vid = 0;
if (id.size() == 8) {
    memcpy(static_cast<void*>(&vid), id.data(), 8);
} else {
    MurmurHash2 hash;
    vid = hash(id.data());
}
```

```
}
PartitionID pId = vid % numParts + 1;
```

简单来说，上述代码是将一个固定的字符串进行哈希计算，转换成数据类型为 `int64` 的数字（`int64` 数字的哈希计算结果是数字本身），将数字取模，然后加 1，即：

```
pId = vid % numParts + 1;
```

示例的部分参数说明如下。

参数	说明
<code>%</code>	取模运算。
<code>numParts</code>	VID 所在图空间的分片数，即 <code>CREATE SPACE</code> 语句中的 <code>partition_num</code> 值。
<code>pId</code>	VID 所在分片的 ID。

例如有 100 个分片，VID 为 1、101 和 1001 的三个点将会存储在相同的分片。分片 ID 和机器地址之间的映射是随机的，所以不能假定任何两个分片位于同一台机器上。

Raft

关于 **RAFT** 的简单介绍

分布式系统中，同一份数据通常会有多个副本，这样即使少数副本发生故障，系统仍可正常运行。这就需要一定的技术手段来保证多个副本之间的一致性。

基本原理：**Raft** 就是一种用于保证多副本一致性的协议。**Raft** 采用多个副本之间竞选的方式，赢得“超过半数”副本投票的（候选）副本成为 **Leader**，由 **Leader** 代表所有副本对外提供服务；其他 **Follower** 作为备份。当该 **Leader** 出现异常后（通信故障、运维命令等），其余 **Follower** 进行新一轮选举，投票出一个新的 **Leader**。**Leader** 和 **Follower** 之间通过心跳的方式相互探测是否存活，并以 **Raft-wal** 的方式写入硬盘，超过多个心跳仍无响应的副本会被认为发生故障。

Note

因为 **Raft-wal** 需要定期写硬盘，如果硬盘写能力瓶颈会导致 **Raft** 心跳失败，导致重新发起选举。硬盘 IO 严重堵塞情况下，会导致长期无法选举出 **Leader**。

读写流程：对于客户端的每个写入请求，**Leader** 会将该写入以 **Raft-wal** 的方式，将该条同步给其他 **Follower**，并只有在“超过半数”副本都成功收到 **Raft-wal** 后，才会返回客户端该写入成功。对于客户端的每个读取请求，都直接访问 **Leader**，而 **Follower** 并不参与读请求服务。

故障流程：场景 1：考虑一个配置为单副本（图空间）的集群；如果系统只有一个副本时，其自身就是 **Leader**；如果其发生故障，系统将完全不可用。场景 2：考虑一个配置为 3 副本（图空间）的集群；如果系统有 3 个副本，其中一个副本是 **Leader**，其他 2 个副本是 **Follower**；即使原 **Leader** 发生故障，剩下两个副本仍可投票出一个新的 **Leader**（以及一个 **Follower**），此时系统仍可使用；但是当这 2 个副本中任一者再次发生故障后，由于投票人数不足，系统将完全不可用。

Note

Raft 多副本的方式与 **HDFS** 多副本的方式是不同的，**Raft** 基于“多数派”投票，因此副本数量不能是偶数。

MULTI GROUP RAFT

由于 **Storage** 服务需要支持集群分布式架构，所以基于 **Raft** 协议实现了 **Multi Group Raft**，即每个分片的所有副本共同组成一个 **Raft group**，其中一个副本是 **leader**，其他副本是 **follower**，从而实现强一致性和高可用性。**Raft** 的部分实现如下。

由于 **Raft** 日志不允许空洞，**NebulaGraph** 使用 **Multi Group Raft** 缓解此问题，分片数量较多时，可以有效提高 **NebulaGraph** 的性能。但是分片数量太多会增加开销，例如 **Raft group** 内部存储的状态信息、**WAL** 文件，或者负载过低时的批量操作。

实现 Multi Group Raft 有 2 个关键点:

- 共享 Transport 层

每一个 Raft group 内部都需要向对应的 peer 发送消息, 如果不能共享 Transport 层, 会导致连接的开销巨大。

- 共享线程池

如果不共享一组线程池, 会造成系统的线程数过多, 导致大量的上下文切换开销。

批量 (BATCH) 操作

NebulaGraph中, 每个分片都是串行写日志, 为了提高吞吐, 写日志时需要做批量操作, 但是由于NebulaGraph利用 WAL 实现一些特殊功能, 需要对批量操作进行分组, 这是NebulaGraph的特色。

例如无锁 CAS 操作需要之前的 WAL 全部提交后才能执行, 如果一个批量写入的 WAL 里包含了 CAS 类型的 WAL, 就需要拆分成粒度更小的几个组, 还要保证这几组 WAL 串行提交。

LEADER 切换 (TRANSFER LEADERSHIP)

leader 切换对于负载均衡至关重要, 当把某个分片从一台机器迁移到另一台机器时, 首先会检查分片是不是 leader, 如果是的话, 需要先切换 leader, 数据迁移完毕之后, 通常还要重新均衡 leader 分布。

对于 leader 来说, 提交 leader 切换命令时, 就会放弃自己的 leader 身份, 当 follower 收到 leader 切换命令时, 就会发起选举。

成员变更

为了避免脑裂, 当一个 Raft group 的成员发生变化时, 需要有一个中间状态, 该状态下新旧 group 的多数派需要有重叠的部分, 这样就防止了新的 group 或旧的 group 单方面做出决定。为了更加简化, Diego Ongaro 在自己的博士论文中提出每次只增减一个 peer 的方式, 以保证新旧 group 的多数派总是有重叠。NebulaGraph也采用了这个方式, 只不过增加成员和移除成员的实现有所区别。具体实现方式请参见 Raft Part class 里 addPeer/removePeer 的实现。

与 HDFS 的区别

Storage 服务基于 Raft 协议实现的分布式架构, 与 HDFS 的分布式架构有一些区别。例如:

- Storage 服务本身通过 Raft 协议保证一致性, 副本数量通常为奇数, 方便进行选举 leader, 而 HDFS 存储具体数据的 DataNode 需要通过 NameNode 保证一致性, 对副本数量没有要求。
- Storage 服务只有 leader 副本提供读写服务, 而 HDFS 的所有副本都可以提供读写服务。
- Storage 服务无法修改副本数量, 只能在创建图空间时指定副本数量, 而 HDFS 可以调整副本数量。
- Storage 服务是直接访问文件系统, 而 HDFS 的上层 (例如 HBase) 需要先访问 HDFS, 再访问到文件系统, 远程过程调用 (RPC) 次数更多。

总而言之, Storage 服务更加轻量级, 精简了一些功能, 架构没有 HDFS 复杂, 可以有效提高小块存储的读写性能。

最后更新: 2026年8月18日

3. 快速入门

3.1 基于 Docker 快速部署

NebulaGraph 提供了基于 Docker 的快速部署方式，可以在几分钟内完成部署。

使用 **Docker Desktop**

使用 **Docker Compose**

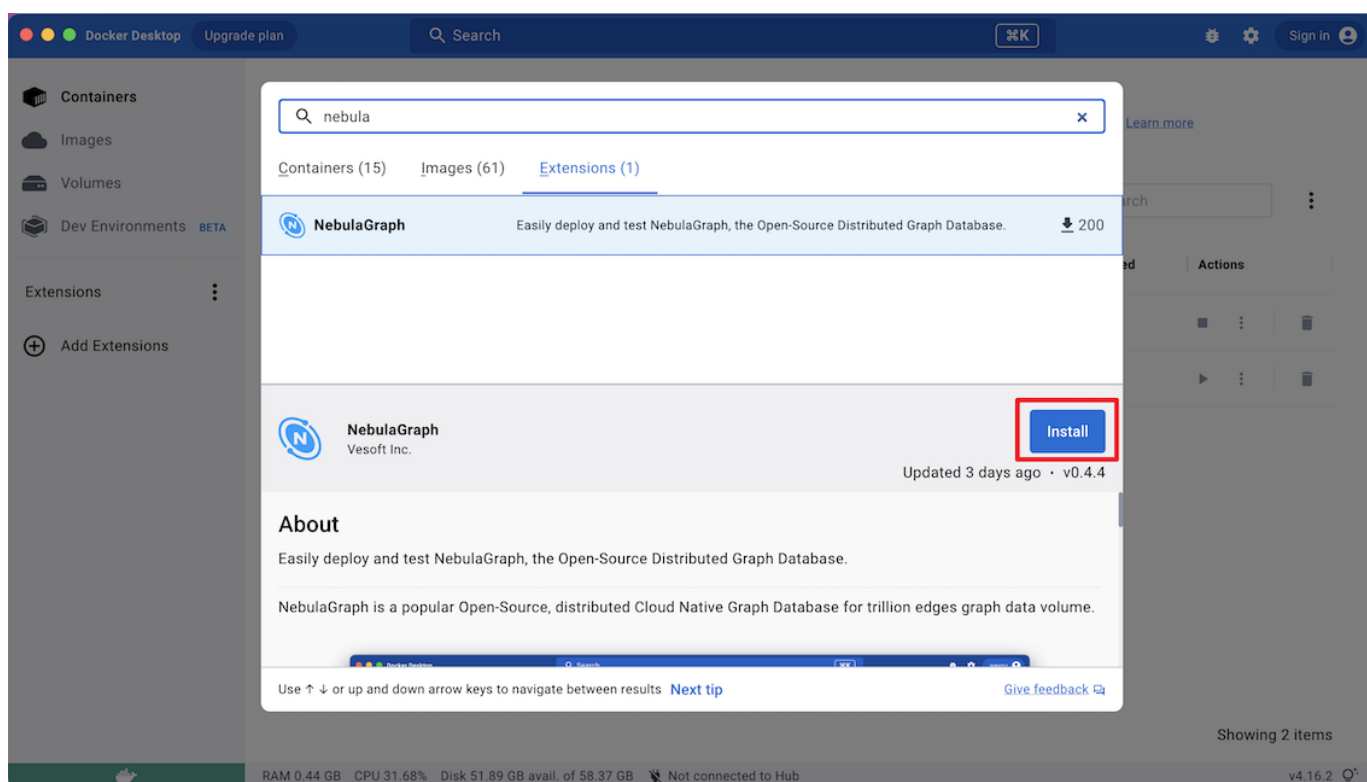
按照以下步骤可以快速在 Docker Desktop 中部署 NebulaGraph。

1. 安装 **Docker Desktop**。

Caution

如果在 Windows 端安装 Docker Desktop 需安装 **WSL 2**。

2. 在仪表盘中单击 Extensions 或 Add Extensions 打开 Extensions Marketplace 搜索 NebulaGraph，也可以点击 **NebulaGraph** 在 Docker Desktop 打开。
3. 导航到 NebulaGraph 的扩展市场。
4. 点击 Install 下载 NebulaGraph。



5. 在有更新的时候，可以点击 Update 更新到最新版本。

